

Instituto Politécnico Nacional

Escuela Superior de Cómputo

ESCOM

Trabajo Terminal I

Desarrollo de un General Game Player (GGP) utilizando un lenguaje de descripción de reglas para juegos de información perfecta

2027-A178

Ingeniería en Sistemas Computacionales

Presentan

Sánchez Catarino Edgar
Linares Villegas Gustavo

Director

Dr. en C. Enrique Escalante Notario



2026

INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
SUBDIRECCIÓN ACADÉMICA

No. de TT: 2027-A178 12 de mayo de 2026

Documento técnico

Desarrollo de un General Game Player (GGP) utilizando un lenguaje de descripción de reglas para juegos de información perfecta

2027-A178

Presentan

Sánchez Catarino Edgar

Linares Villegas Gustavo

Director

Dr. en C. Enrique Escalante Notario

RESUMEN

Este Trabajo Terminal propone diseñar e implementar un General Game Player (GGP) para juegos discretos, deterministas, por turnos y de información perfecta. La idea es separar las reglas de cada juego del motor de decisión del agente mediante un lenguaje de descripción externo. A partir de ese lenguaje, el sistema interpretará la estructura del juego, construirá una representación interna, generará los movimientos legales, aplicará transiciones, reconocerá los estados terminales y elegirá acciones con algoritmos generales de búsqueda. El proyecto se validará con el Gato, como caso funcional completo, y con el Ajedrez, como caso de mayor complejidad cuyo diseño experimental aún debe definirse, para observar qué tanto generaliza el enfoque sin modificar el núcleo algorítmico del agente.

Palabras clave: General Game Playing, lenguaje de descripción de juegos, inteligencia artificial, juegos de información perfecta, árboles de búsqueda, Monte Carlo Tree Search.

Advertencia

"Este documento contiene información desarrollada por la Escuela Superior de Cómputo del Instituto Politécnico Nacional, a partir de datos y documentos sujetos a derechos de propiedad. Por lo tanto, su uso queda restringido a las aplicaciones que explícitamente se convengan."

Cualquier aplicación no convenida exime a la Escuela Superior de Cómputo de responsabilidad técnica y podrá dar lugar a las consecuencias legales que, para tal efecto, correspondan.

Para obtener información adicional sobre este reporte técnico, podrá dirigirse a:

La Subdirección Académica de la Escuela Superior de Cómputo del Instituto Politécnico Nacional.

Índice general

Advertencia II

Glosario X

Abreviaturas XIII

1 Introducción 1
- 1.1 Contexto y motivación . . . 1
- 1.2 Planteamiento del problema . . . 2
- 1.3 Justificación . . . 3
- 1.4 Objetivos . . . 4
  - 1.4.1 Objetivo general . . . 4
  - 1.4.2 Objetivos específicos . . . 4
- 1.5 Alcance y limitaciones . . . 4
- 1.6 Metodología y planeación del desarrollo . . . 5
  - 1.6.1 Enfoque iterativo e incremental . . . 5
  - 1.6.2 Etapas de desarrollo . . . 5
  - 1.6.3 Productos esperados . . . 6
  - 1.6.4 Matriz de seguimiento y evidencias propuestas . . . 6
  - 1.6.5 Diseño de la evaluación: métricas y muestra . . . 7
    - Métricas y procedimiento de medición . . . 7
    - Selección de la muestra y de los casos . . . 8
    - Condiciones de comparación y registro . . . 9
  - 1.6.6 Distribución del trabajo y calendario . . . 9

2 Estado del arte 11
- 2.1 Evolución de los sistemas de juego en inteligencia artificial . . . 11
- 2.2 Surgimiento y fundamentos de General Game Playing . . . 12
- 2.3 Generaciones y agentes emblemáticos de General Game Playing . . . 12
  - 2.3.1 Primera generación: Análisis estático de reglas y Minimax heurístico . . . 12
  - 2.3.2 Segunda generación: La revolución de Monte Carlo Tree Search . . . 13
  - 2.3.3 Tercera generación: PropNets optimizadas y heurísticas de simulación . . . 14
- 2.4 Plataformas y lenguajes modernos de descripción de juegos . . . 15
  - 2.4.1 Extensiones de GDL e información imperfecta . . . 15
  - 2.4.2 El sistema Ludii . . . 15
  - 2.4.3 Regular Boardgames . . . 15

2.4.4 Entornos generales para aprendizaje: OpenSpiel 16
2.5 Tendencias contemporáneas en aprendizaje profundo y modelos unificados . . . 16
2.6 Síntesis comparativa y posicionamiento del Trabajo Terminal 16

3 Marco Teórico 18
3.1 Inteligencia artificial en juegos 18
3.2 Juegos de información perfecta 18
3.2.1 Espacio de observaciones y función de observación (O_i) 19
3.2.2 Relación de indistinguibilidad ($\sim_i$) 19
3.2.3 La indistinguibilidad como relación de equivalencia 20
3.2.4 Conjuntos de información y clases de observación 20
3.2.5 Ejemplos comparativos 21
3.2.6 Implicación para el modelo formal Γ 22
3.3 General Game Playing 23
3.4 Modelo formal de juego 23
3.4.1 Conjunto de roles (R) 24
3.4.2 Espacio de estados (S) y semántica de fluents 24
Ejemplo ilustrativo en el juego del Gato 25
3.4.3 Estado inicial (s_0) 26
3.4.4 Acciones individuales (A_i) y espacio conjunto de acciones ($\mathbf{A}$) 26
3.4.5 Función de legalidad (L_i) 26
3.4.6 Función de transición dinámica (T) 26
3.4.7 Conjunto de estados terminales (F) 27
3.4.8 Función de utilidad o meta (G_i) 27
3.4.9 Axiomas de admisibilidad y juegos bien formados 27
3.4.10 Distancia en el grafo de estados y estimación hacia configuraciones terminales 28
3.4.11 Modelado síncrono de juegos por turnos alternados 30
3.4.12 Interfaz computacional para el motor de decisión 30
3.5 Lenguaje de descripción de juegos (GDL) 30
3.5.1 Características fundamentales de GDL 31
3.5.2 Concepto y estructura de un predicado lógico 31
Construcción de reglas de inferencia (<=) 32
3.5.3 Predicados reservados del sistema en GDL 32
3.5.4 Ejemplificación mediante código de especificación: El juego del Gato . 32
Roles y estado inicial 32
Reglas de legalidad y alternancia de turnos 33
Transición de estado y persistencia inercial 34
Condiciones terminales y asignación de recompensas 34
3.6 GDL, Ludii y lenguajes de dominio específico 34
3.7 Intérprete y representación interna 36
3.8 Árboles de juego y explosión combinatoria 36
3.9 Minimax 37
3.10 Poda alfa-beta 38
3.11 Funciones heurísticas generales 38

3.12 Monte Carlo Tree Search . . . 39
3.13 UCT . . . 40
3.14 Aprendizaje por refuerzo y aprendizaje profundo . . . 40
3.15 AlphaZero como antecedente moderno . . . 41
3.16 Síntesis del marco teórico . . . 41

4 Análisis **43**
4.1 Representación formal de juegos y niveles de descripción . . . 43
4.1.1 Delimitación del objeto descrito . . . 43
4.1.2 Representación extensional y representación factorizada . . . 44
4.1.3 Espacio de estados y espacio de partidas . . . 46
Definición de partida . . . 46
Conjuntos en que viven los estados y las partidas . . . 47
Número de estados del Gato . . . 48
Árbol de juego, transposiciones y número de partidas . . . 50
Relación con la distancia entre estados: conteo de trayectorias . . . 52
Comparación entre el número de partidas y el número de estados . . . 55
Estimación para el Ajedrez . . . 58
4.1.4 Niveles de abstracción de una descripción . . . 61
4.1.5 Arquitectura del intérprete y flujo de procesamiento . . . 62
4.2 Definición conceptual de un lenguaje de descripción de juegos . . . 63
4.2.1 Propiedades fundamentales de un lenguaje de descripción . . . 64
4.3 Sintaxis del *Game Description Language* . . . 66
4.3.1 Fundamentos sintácticos en KIF y en las expresiones simbólicas . . . 66
4.3.2 Alfabeto y especificación léxica . . . 66
4.3.3 Signatura y sintaxis de términos, átomos y reglas . . . 67
Reificación de fluents y acciones como términos . . . 69
4.3.4 Universo y base de Herbrand . . . 70
4.3.5 Conjunto de palabras reservadas . . . 71
Relaciones reservadas de entrada, de salida y estáticas . . . 72
Declaración de dominios mediante `base` e `input` . . . 73
Extensiones del conjunto reservado en los dialectos de GDL . . . 73
4.3.6 Gramática libre de contexto de GDL . . . 74
Restricciones contextuales y sintaxis abstracta . . . 75
4.4 Condiciones de validez de una descripción GDL . . . 76
4.4.1 Seguridad de las reglas . . . 76
4.4.2 Grafo de dependencias y estratificación . . . 77
4.4.3 Restricción de la recursión . . . 79
4.4.4 Restricciones de uso de las relaciones reservadas . . . 80
4.4.5 Caracterización del conjunto de descripciones válidas . . . 81
4.5 Semántica formal de GDL . . . 82
4.5.1 Modelo estándar de un programa estratificado . . . 82
4.5.2 Función semántica $[[\cdot]]$ . . . 84
4.5.3 Clase de juegos objetivo y juegos bien formados . . . 86
4.6 Análisis semántico del juego de la resta . . . 88

4.6.1 Validación estática . . . 88
4.6.2 Modelo estático . . . 89
4.6.3 Legalidad en el estado inicial s_0 . . . 89
4.6.4 Evaluación de la función de transición . . . 89
4.6.5 Juego denotado y valor Minimax . . . 90
4.7 Análisis semántico del juego del Gato . . . 91
4.7.1 Evaluación de la legalidad en el estado inicial . . . 92
4.7.2 Evaluación de la transición $s_0 \rightarrow s_1$ . . . 92
4.7.3 Terminación y utilidades en estados terminales . . . 93
4.7.4 Dimensión del juego denotado . . . 93
4.8 Clasificación de descripciones defectuosas . . . 93
4.9 Expresividad y no inyectividad de la semántica . . . 96
4.10 Complejidad computacional de las consultas . . . 97
4.11 Análisis comparativo de lenguajes de descripción de juegos . . . 98
4.12 Requisitos derivados para el intérprete . . . 99

5 Diseño **103**
5.1 Arquitectura conceptual propuesta . . . 103
5.2 Prototipos de interfaz aprobados . . . 105

6 Desarrollo **111**

7 Pruebas **112**

8 Conclusiones y trabajo a futuro **113**

Referencias **114**

Anexos **119**
Cronogramas . . . 119
Sánchez Catarino Edgar . . . 119
Linares Villegas Gustavo . . . 120

Índice de figuras

1.1 Comparación estructural entre un motor especializado tradicional y la arquitectura desacoplada de un General Game Player. 3

2.1 Comparación esquemática entre la inferencia lógica interpretada recursivamente y la evaluación acelerada por propagación de señales en redes proposicionales (*PropNets*). 13
2.2 Organización expositiva de antecedentes y generaciones algorítmicas en *General Game Playing* (GGP). 14

3.1 Clases de estados compatibles con una observación. En (a), dos estados producen la misma observación y pertenecen a una clase no unitaria; en (b), la observación inyectiva identifica cada estado. La figura ilustra observación de estados, no conjuntos de información sobre historias. 22
3.2 Árbol de juego adversarial con poda alfa-beta. Tras evaluarse el nodo con valor 2, el nodo MIN asegura un valor a lo sumo de 2, el cual es estrictamente inferior al valor $\alpha = 3$ ya garantizado por MAX en la rama izquierda; en consecuencia, se interrumpe la exploración del resto de los sucesores. 38
3.3 Ciclo iterativo de cuatro fases en Monte Carlo Tree Search (MCTS): Selección de nodos mediante la cota UCT, Expansión de nuevas acciones legales, Simulación estocástica (*rollout*) hasta un estado terminal y Retropropagación de recompensas hacia la raíz. 39
3.4 Flujo conceptual de ejecución y arquitectura desacoplada del General Game Player. El flujo detallado se presenta en la Figura 5.1. 42

4.1 Árbol de juego del Gato. Cada nodo del árbol es una historia; un mismo estado puede corresponder a varios nodos, como muestra la transposición del panel (b). . 51
4.2 Dos familias de juegos que muestran que no hay un orden general entre el número de partidas y el número de estados. En la cadena no hay transposiciones y cada estado tiene una única historia; en la escalera, cada estado del nivel j tiene 2^{j-1} historias, y las partidas crecen exponencialmente mientras los estados crecen linealmente. 58
4.3 Niveles de abstracción de una descripción de juego. Cada función es parcial; los dominios de definición se van estrechando de izquierda a derecha hasta quedar únicamente las descripciones con significado. 61
4.4 Grafo de dependencias de la descripción del Gato. Las aristas continuas son positivas y las discontinuas, negativas. Ninguna arista negativa forma parte de un ciclo, por lo que el programa es estratificado con dos estratos. 79

4.5 Grafo de juego de $\Gamma_{D_{\text{resta}}}$. Las aristas se etiquetan con la acción del rol activo; la del otro rol es siempre `noop`. Los nodos verdes son terminales. 91

5.1 Arquitectura propuesta: carga de reglas, consultas de búsqueda y actualización de la partida. Las dos filas principales forman un único flujo; las entradas, salidas y condiciones de terminación se muestran alrededor del núcleo. 104
5.2 Prototipo aprobado de carga y revisión de una descripción externa de Gato. La validación y el contenido del archivo son ilustrativos; la sintaxis del lenguaje permanece por definir. 106
5.3 Prototipo aprobado de una partida de Gato entre agentes. El tablero, historial, acciones legales y próxima decisión representan un estado ilustrativo. 107
5.4 Prototipo aprobado de una partida de Ajedrez entre agentes. La posición y las acciones son ilustrativas. 108
5.5 Prototipo aprobado de preparación de una evaluación táctica de primera jugada en Ajedrez. Las cantidades, el presupuesto y el estado de ejecución son ilustrativos y no describen un experimento realizado. 109
5.6 Prototipo aprobado del reporte de resultados de una evaluación. Las métricas y el detalle por caso muestran datos simulados, sin atribuir una ejecución real ni desempeño al agente. 110

Índice de tablas

1.1 Matriz propuesta de seguimiento del desarrollo. 6
1.2 Métricas y procedimientos propuestos de evaluación. 8

2.1 Comparación de paradigmas y sistemas representativos en juegos e inteligencia artificial. 17

3.1 Predicados reservados de GDL y componente de Γ que describen. 32

4.1 Estados, historias y partidas del Gato según el número k de marcas en el tablero. B_k: tableros compatibles con el invariante (4.6); R_k: estados alcanzables; F_k: estados terminales alcanzables; H_k: nodos del árbol de juego; P_k: partidas de longitud k. 52
4.2 Número de estados y de partidas en los juegos de este capítulo. 58
4.3 Conjuntos en que vive una descripción en cada nivel de abstracción. 62
4.4 Categorías léxicas de GDL. La notación X^+ indica una o más repeticiones de elementos de X; $\cdot$ denota concatenación. 67
4.5 Palabras reservadas de GDL, su aridad, su significado formal y la componente de Γ que determinan. La notación M se refiere al modelo estándar de la Sección 4.5. 72
4.6 Palabras reservadas añadidas por cada dialecto de la familia GDL y clase de juegos que habilitan. 74
4.7 Gramática $\mathcal{G}_{\text{GDL}}$ en notación EBNF. Las categorías *Id* y *Var* son las de la Tabla 4.4; $Id^\circ = Id \setminus K_{\log}$. 75
4.8 Restricciones de uso de las relaciones reservadas y razón formal de cada una. "Depende" se entiende en el sentido de la Definición 4.15. 81
4.9 Representación explícita completa de $\Gamma_{D_{\text{resta}}}$: estados no terminales, acciones conjuntas legales y sucesores. 90
4.10 Evaluación de las reglas de transición del Gato para s_0 y $\mathbf{a} = (\texttt{mark}(2,2), \texttt{noop})$. 92
4.11 Cifras del juego denotado por D_{Gato}. La intersección $F \cap \text{Reach}(\Gamma_D)$ es el conjunto de estados que son a la vez terminales y alcanzables; la suma de la penúltima fila cuenta las aristas del grafo de juego, como en la Ecuación (4.2). 94
4.12 Contraejemplos de cada restricción, nivel de detección y consecuencia. 95
4.13 Comparación de lenguajes y entornos de descripción de juegos según las componentes de la Definición 4.5. 100
4.14 Requisitos derivados del análisis del lenguaje de descripción. 101

Glosario

Término	Definición
Acción legal	Movimiento que un jugador puede hacer en un estado concreto del juego, según las reglas de la descripción formal del dominio.
Acción nula (*noop*)	Acción pasiva (*no-operation*) que ejecutan los jugadores que no tienen el turno. Sirve para representar la alternancia de turnos dentro de un modelo formal síncrono y con varios participantes.
Agente inteligente	Sistema que percibe un entorno, razona sobre sus posibles acciones y elige una de acuerdo con un objetivo, una recompensa o un criterio de utilidad.
Árbol de juego	Grafo acíclico con las secuencias posibles de estados y acciones de una partida. Cada nodo es una configuración y cada arista, una acción conjunta que lleva a un nuevo estado.
Asunción de mundo cerrado (*CWA*)	Principio de la programación lógica según el cual todo hecho o *fluent* que no pueda deducirse como verdadero en el estado actual se considera falso.
Conjunto de información de Kuhn	Conjunto de nodos de decisión o historias que un jugador no distingue con la información recibida hasta ese momento. No equivale automáticamente a una clase de estados con igual observación actual.
Criterio de selección UCT	Regla de selección de Monte Carlo Tree Search basada en la cota superior de confianza UCB1. Busca equilibrar la explotación de las jugadas que han funcionado con la exploración de las ramas poco visitadas.
Descripción de juego	Especificación externa y declarativa que define los roles, el estado inicial, las acciones legales, las transiciones, los estados terminales y las recompensas, sin programar esas reglas dentro del motor del agente.
Distancia dirigida entre estados	Longitud mínima de una trayectoria legal dirigida entre estados. Vale cero de un estado a sí mismo e infinito si no hay trayectoria; no es una métrica ordinaria porque puede ser asimétrica y no finita.
Estado	Configuración completa del juego en un instante discreto. Formalmente, es el conjunto finito de *fluents* atómicos verdaderos bajo la asunción de mundo cerrado.
Estado terminal	Configuración en la que la partida termina y se asigna una recompensa numérica a cada participante.
Explosión combinatoria	Crecimiento exponencial del número de configuraciones y de nodos del árbol de búsqueda a medida que se profundiza, provocado por el factor de ramificación del dominio.

Continúa en la siguiente página...

Término	**Definición**
Factor de ramificación	Número promedio de acciones legales que salen de un nodo del árbol de búsqueda de un juego.
Fluent	Proposición lógica atómica cuyo valor de verdad cambia con el tiempo o con el estado. Representa hechos del entorno que pueden aparecer, mantenerse o desaparecer tras cada transición.
Función de observación	Función que asigna a cada estado del juego la observación que realmente recibe un jugador, es decir, la vista de la configuración real que le llega.
Función heurística	Función que estima qué tan bueno es un estado no terminal, o qué tan cerca está de la victoria, cuando la explosión combinatoria impide explorar el árbol hasta el final.
General Game Player	Agente autónomo que recibe en tiempo de ejecución la descripción formal de un juego que no conocía y lo juega bien, sin ajustes manuales en su código para ese dominio.
Información perfecta	Propiedad de los juegos cuyos conjuntos de información en los nodos de decisión son unitarios: quien decide conoce la historia completa hasta ese nodo. No implica por sí sola determinismo ni ausencia de azar.
Intérprete	Módulo de software que analiza el archivo de reglas externo, valida su estructura y construye en memoria la representación del juego.
Juego bien formado (*well-formed game*)	Juego formal que cumple a la vez con la jugabilidad (no hay bloqueos o *deadlocks*) y con la terminación garantizada en un número finito de jugadas.
Lenguaje de descripción de juegos	Notación formal y declarativa de dominio específico (como GDL o Ludii) con la que se codifican la dinámica, la legalidad y las metas de un juego, separadas del algoritmo de decisión.
Minimax	Algoritmo clásico de decisión para juegos de dos jugadores y suma constante: elige la acción que maximiza la utilidad mínima que puede asegurar frente a un adversario racional.
Motor de decisión	Núcleo del agente que explora las jugadas posibles y elige una acción mediante búsqueda en árboles, podas o simulación estocástica.
Negación estratificada	Propiedad de la semántica de Datalog que prohíbe la recursión cruzada con negación, lo que asegura que el modelo lógico de las reglas sea único, finito y decidible.
Poda alfa-beta	Optimización de Minimax que descarta subárboles completos cuya exploración no puede cambiar la decisión óptima en la raíz.
Recompensa	Valor numérico que recibe cada rol al llegar a una configuración terminal y que mide qué tan bien le fue en la partida.
Relación de indistinguibilidad	Relación de equivalencia sobre los estados que vincula dos configuraciones cuando un participante recibe exactamente la misma observación de ambas.
Relajación de efectos negativos (*delete relaxation*)	Relajación de planificación que elimina efectos de borrado. Para STRIPS proposicional explícito su alcanzabilidad es polinomial, pero el plan relajado óptimo sigue siendo NP-hard; las aproximaciones no son siempre admisibles.

Continúa en la siguiente página...

Término	**Definición**
Retropropagación (*backpropagation*)	Fase del ciclo de MCTS en la que la recompensa obtenida en una hoja o en una simulación se propaga hacia la raíz y actualiza las estadísticas de visitas y el valor medio.
Simulación estocástica (*rollout*)	Fase de MCTS en la que se juega una partida rápida, con movimientos elegidos al azar, desde un nodo expandido hasta una condición terminal.
Transición	Función que, a partir del estado actual y de un vector de acciones conjuntas, determina el único estado sucesor.

Abreviaturas

Abreviatura	Significado
AI	Artificial Intelligence (Inteligencia Artificial)
CWA	Closed World Assumption (Asunción de Mundo Cerrado)
DSL	Domain-Specific Language (Lenguaje de Dominio Específico)
GDL	Game Description Language (Lenguaje de Descripción de Juegos)
GGP	General Game Playing o General Game Player, según el contexto
IA	Inteligencia Artificial
MCTS	Monte Carlo Tree Search (Búsqueda de Árbol Monte Carlo)
RL	Reinforcement Learning (Aprendizaje por Refuerzo)
TT	Trabajo Terminal
UCT	Upper Confidence Bounds applied to Trees

Capítulo 1

Introducción

1.1. Contexto y motivación

Los juegos acompañan a la inteligencia artificial desde sus primeros años. Sirven como banco de pruebas porque tienen reglas explícitas, espacios de estados discretos y resultados que cualquiera puede verificar, algo que el mundo físico rara vez ofrece [1]. En la década de 1950, Arthur L. Samuel escribió un programa para jugar a las damas inglesas que combinaba búsqueda Minimax con una evaluación heurística y que mejoraba con la experiencia acumulada [2]. Con él se popularizó el término *machine learning*, aunque la idea ya la había explorado en teoría Alan Turing [3]; lo que hizo Samuel fue llevarla a la práctica.

Las décadas siguientes dejaron varios hitos frente a jugadores humanos. En 1997, Deep Blue, la supercomputadora de IBM, derrotó al campeón mundial Garry Kasparov [4]. Lo logró con hardware masivamente paralelo y una búsqueda alfa-beta que examinaba hasta doscientos millones de posiciones por segundo, guiada por una función de evaluación pensada y ajustada solo para el ajedrez. Después, el sistema Chinook resolvió el juego de damas [5] y, en años más recientes, AlphaGo alcanzó un desempeño destacado en Go [6]. AlphaZero extendió un mismo algoritmo de aprendizaje y búsqueda a Ajedrez, Shogi y Go, con entrenamiento separado para cada juego [7].

Estos avances tienen distintos alcances de generalidad. Deep Blue se diseñó para Ajedrez y AlphaGo para Go; transferirlos a otro juego requiere adaptar su representación y sus reglas. AlphaZero comparte un algoritmo entre tres dominios, pero utiliza reglas, representaciones de acciones y una instancia entrenada para cada uno [7]. La propuesta de este TT estudiará otra forma de generalidad: recibir reglas externas y construir un modelo consultable sin reprogramar el núcleo de decisión.

Para atacar esta limitación, a partir de 2005 la Universidad de Stanford y la *Association for the Advancement of Artificial Intelligence* (AAAI) impulsaron la línea de investigación conocida como *General Game Playing* (GGP) [8], [9]. Aquí el objetivo ya no es construir el mejor programa para un juego concreto, sino un agente que reciba la descripción formal de un juego que nunca ha visto y lo juegue bien, sin apoyarse en algoritmos o heurísticas escritos para ese dominio [10]. El trabajo de fondo pasa así del programador al agente, que debe interpretar la lógica del juego, armar sus propias representaciones internas y decidir en tiempo real.

1.2. Planteamiento del problema

Los agentes de juego tradicionales dependen demasiado de las reglas del entorno para el que fueron escritos. Sus estructuras de datos se diseñan a la medida del tablero y sus funciones heurísticas incorporan conocimiento empírico del dominio, como el valor relativo de las piezas en el ajedrez o tablas de posiciones ganadoras calculadas de antemano.

En General Game Playing el agente tiene que manejar el espacio de estados $\mathcal{S}$ de cualquier juego que le toque, sin saber de antemano cómo funciona. Desde la teoría de juegos, un juego de información perfecta se modela con una tupla formal: los jugadores, los estados válidos, las acciones permitidas, la función de transición y las utilidades terminales. El problema es el tamaño. Las configuraciones posibles crecen de forma exponencial y a veces ni siquiera se pueden recorrer completas.

Un ejemplo permite distinguir las configuraciones representables de las que satisfacen restricciones. Para las ocho damas se considerarán aquí **configuraciones completas**: un vector $v = (v_1, \ldots, v_8)$ indica la fila de la dama en cada columna de un tablero de 8×8. El espacio de representación es $\mathcal{S}_{\text{rep}} = \{0, \ldots, 7\}^8$, con $8^8 = 16\,777\,216$ elementos. Si se exige además una dama por fila, el subconjunto $\mathcal{S}$ contiene las $8! = 40\,320$ permutaciones. Las soluciones $\mathcal{S}_{\text{solución}} \subset \mathcal{S}$ satisfacen también $|v_i - v_j| \neq |i - j|$ para $i \neq j$, evitando ataques diagonales. Son 92 configuraciones [11]; al agruparlas por rotaciones y reflexiones del cuadrado se obtienen 12 clases. Esta convención clasifica configuraciones completas; no define los estados parciales ni las transiciones de un algoritmo de colocación. En el agente propuesto, las restricciones del dominio deberán proceder de la descripción externa.

En Ajedrez deben distinguirse el espacio de posiciones y el árbol de variantes. Shannon estimó un orden de 10^{43} configuraciones de piezas mediante un conteo combinatorio y un orden de 10^{120} variantes bajo supuestos de ramificación y duración medias [12]. No son un censo exacto de posiciones legales alcanzables ni del árbol completo. Estas estimaciones ilustran por qué el agente deberá generar sucesores a solicitud y decidir bajo límites de búsqueda, en vez de enumerar todo el espacio.

El problema que atiende este Trabajo Terminal es cómo construir una arquitectura desacoplada, basada en reglas externas y en un intérprete, para reconocer acciones legales, aplicar transiciones y seleccionar jugadas bajo un presupuesto de búsqueda. Al cargar otro juego cambiarán la descripción y el modelo derivado de ella, conservando el código del núcleo. La calidad de las decisiones se estudiará con las métricas propuestas, sin anticipar fuerza competitiva ni optimalidad con búsqueda limitada.

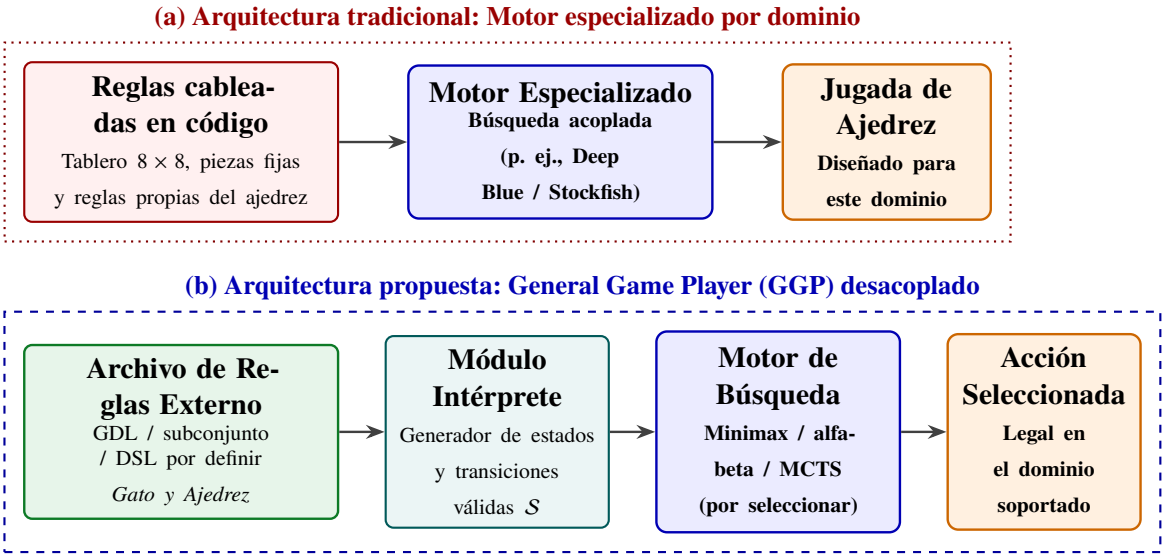


Figura 1.1: Comparación estructural entre un motor especializado tradicional y la arquitectura desacoplada de un General Game Player.

1.3. Justificación

El punto de partida de este trabajo es una incomodidad concreta: casi todos los agentes de juego quedan atados a un solo dominio. Tanto en la industria como en la academia lo habitual es dejar fijos, dentro del código fuente, las reglas, el tablero y las mecánicas. Un General Game Player que no dependa del dominio ofrece otra salida, porque el mismo programa se reutiliza con juegos distintos.

Lo distintivo de la propuesta es que el agente no sabe nada de ningún juego. No juega bien por consejos que dejó un experto humano, sino porque lee la descripción formal de las reglas y aplica procesos generales de búsqueda heurística y simulación. Las transiciones válidas y las condiciones de victoria salen del código ejecutable y pasan a un archivo externo estructurado; con eso se busca que el motor de inteligencia artificial quede separado por completo de la lógica de cada juego.

Desde la Ingeniería en Sistemas Computacionales, el desarrollo reúne varias áreas. Hace falta teoría de lenguajes formales y autómatas para diseñar el análisis léxico, sintáctico y semántico del intérprete de reglas; también modelado matemático y estructuras de datos eficientes para guardar en memoria espacios de estados enormes; y, por último, algoritmos de decisión adversarial, tanto clásicos como modernos, como Minimax con poda alfa-beta y Monte Carlo Tree Search (MCTS).

La viabilidad del proyecto se plantea dentro del ciclo anual de Trabajo Terminal en la Escuela Superior de Cómputo del Instituto Politécnico Nacional, con un alcance académico y búsqueda acotada. Se propone comprobar la reutilización de una arquitectura modular que procese reglas declarativas; igualar los resultados publicados de sistemas de aprendizaje profundo queda fuera de este alcance.

1.4. Objetivos

1.4.1. Objetivo general

Diseñar e implementar un General Game Player (GGP) para juegos de información perfecta, capaz de interpretar las reglas de distintos entornos a partir de un lenguaje de descripción estructurado, basado en un modelo formal de juego. El agente debe ajustar su toma de decisiones a la especificación que se le cargue, sin modificar el código fuente de su motor algorítmico base.

1.4.2. Objetivos específicos

1. Diseñar la sintaxis y la semántica del lenguaje de descripción de reglas, es decir, el formato del archivo.

2. Desarrollar el intérprete y el generador de estados.

3. Implementar el motor de toma de decisiones del agente.

4. Validar la generalización del agente y del formato de archivo con dominios de reglas muy distintos, en particular el Gato (*Tic-Tac-Toe*) y el Ajedrez, de modo que el cambio de comportamiento dependa únicamente del archivo de especificación.

5. Evaluar el desempeño táctico del agente en ajedrez mediante partidas contra un motor especializado con ELO configurable, y obtener métricas como tasa de victorias, longitud de las partidas y tiempo de respuesta por jugada.

1.5. Alcance y limitaciones

Este Trabajo Terminal se limita a juegos que cumplen cuatro condiciones a la vez: información perfecta (el estado completo de la partida y el historial de jugadas son conocidos por todos los participantes en todo momento), estados y acciones discretos, dinámica determinista, es decir, sin azar, y turnos que se alternan de forma secuencial. Con esta frontera el esfuerzo se concentra en el lenguaje de reglas, el intérprete y los algoritmos de búsqueda general, sin sumar la complejidad de los modelos probabilísticos ni de los conjuntos de información parcialmente observables.

Elegimos dos casos que se parecen poco entre sí. El Gato (*Tic-Tac-Toe*) es tan pequeño que puede revisarse entero, y por eso ahí es posible comprobar de forma analítica que el intérprete y las transiciones funcionan bien. El Ajedrez está en el otro extremo: con tantas jugadas posibles y una táctica tan profunda, muestra qué le pasa al motor cuando el problema es grande.

La validación no buscará superar en fuerza de juego a motores especializados como Stockfish, un motor libre y de código abierto para Ajedrez [13]. Tampoco se propone reproducir el entrenamiento profundo de AlphaZero o Student of Games [7], [14]. Los recursos empleados para entrenar esos sistemas y los de su ejecución posterior son aspectos diferentes; aquí se estudiará la generalidad operativa mediante reglas externas y búsqueda acotada, sin atribuir al prototipo un desempeño aún no medido.

1.6. Metodología y planeación del desarrollo

La metodología y el calendario de trabajo se tomarán del protocolo del TT 2027-A178, en particular de sus apartados de productos esperados, metodología y cronograma [15]. Se conservarán las actividades, los responsables y los meses allí establecidos. Para facilitar el seguimiento, se propone complementar esa base con entregables verificables, dependencias y evidencias de aceptación. Estos criterios operativos se distinguirán de los compromisos del protocolo y no se interpretarán como resultados ya obtenidos.

1.6.1. Enfoque iterativo e incremental

El desarrollo se organizará mediante ciclos de trabajo en los que se definirá un objetivo acotado, se elaborará un entregable y se revisará su correspondencia con las reglas y el modelo formal del juego. Las observaciones de cada revisión se utilizarán para ajustar el siguiente ciclo. De esta manera, la integración del sistema se realizará de forma gradual y permitirá identificar inconsistencias entre la descripción de reglas, la interpretación del estado y la selección de acciones.

En cada ciclo se propone registrar la versión del entregable, las comprobaciones realizadas y las limitaciones pendientes. Una actividad se considerará concluida únicamente cuando exista evidencia revisable de su cumplimiento. La presencia de una actividad en el cronograma, por sí sola, no acreditará su ejecución ni un porcentaje de avance.

1.6.2. Etapas de desarrollo

Se recuperarán las cuatro etapas metodológicas del protocolo [15]:

1. **Diseño del lenguaje de descripción de reglas.** Se definirán la sintaxis, la gramática y la semántica necesarias para expresar estados, acciones, transiciones y condiciones terminales. La elección entre GDL completo, un subconjunto o un DSL propio deberá documentarse antes de cerrar la especificación.

2. **Construcción del módulo intérprete.** Se desarrollará y validará el procesamiento del archivo de reglas y la representación interna del juego. Se verificará que las consultas de legalidad, transición y terminalidad correspondan a la especificación definida.

3. **Implementación del agente.** Se desarrollará la toma de decisiones sobre el modelo que exponga el intérprete. La selección de algoritmos, las funciones heurísticas y los presupuestos de búsqueda se documentarán sin introducir reglas particulares de Gato o Ajedrez dentro del núcleo de decisión.

4. **Integración y validación experimental.** Se integrarán el intérprete, el modelo del juego y el agente. Se prepararán los archivos de los dos dominios y se ejecutarán los experimentos que se definan para comprobar legalidad, funcionamiento y desempeño bajo condiciones reproducibles.

Estas etapas expresarán dependencias de trabajo y podrán revisitarse durante las iteraciones. No implicarán que se deba enumerar en memoria la totalidad del espacio de estados de Ajedrez ni que el agente seleccione una jugada óptima cuando la búsqueda sea limitada.

1.6.3. Productos esperados

Se conservarán los cinco productos establecidos en el protocolo [15]:

1. **Código fuente del General Game Player:** incluirá el intérprete del archivo de reglas y el motor algorítmico del agente.
2. **Especificación formal del lenguaje de reglas:** documentará la sintaxis, la semántica y la gramática del formato adoptado.
3. **Archivos de entorno de Gato y Ajedrez:** describirán los dos dominios que se utilizarán en la validación, con las restricciones que se acuerden para cada uno.
4. **Reporte de pruebas y experimentos:** presentará los escenarios, las condiciones de ejecución, las métricas y los resultados efectivamente obtenidos.
5. **Manual de usuario:** explicará cómo ejecutar el agente y cómo escribir descripciones compatibles con el lenguaje especificado.

1.6.4. Matriz de seguimiento y evidencias propuestas

La Tabla 1.1 relaciona las etapas con sus entregables y dependencias. Las evidencias de aceptación son una propuesta operativa para el seguimiento; no se atribuyen literalmente al protocolo ni constituyen pruebas ya realizadas. Los umbrales cuantitativos se definirán al diseñar los experimentos.

Tabla 1.1: Matriz propuesta de seguimiento del desarrollo.

Etapa	Actividad	Entregable	Dependencia	Evidencia de aceptación propuesta
1. Lenguaje	Definir gramática y semántica de las reglas.	Especificación y ejemplos de descripciones.	Modelo formal y alcance de juegos definidos.	Se explicará cómo se representan estados, acciones, legalidad, transiciones, terminales y utilidades, sin reglas implícitas en el agente.
2. Intérprete	Procesar reglas y construir el modelo consultable.	Intérprete y casos de comprobación.	Especificación de la etapa 1.	Se contrastarán las acciones legales, los estados sucesores y la terminalidad con casos cuyo resultado esperado se haya documentado previamente.

Continúa en la siguiente página.

Tabla 1.1 (continuación).

Etapa	Actividad	Entregable	Dependencia	Evidencia de aceptación propuesta
3. Agente	Seleccionar acciones mediante búsqueda general.	Motor de decisión y registro de ejecución.	Consultas del intérprete verificadas.	Se comprobará que las acciones elegidas sean legales y que la decisión respete el presupuesto de búsqueda definido, registrando tiempo y nodos explorados.
4. Validación	Integrar módulos y ejecutar experimentos.	Archivos de juegos, reporte y manual.	Integración funcional y diseño experimental definido.	Se documentará qué cambia al cargar otro juego; se verificará que el núcleo se conserve y se reportarán resultados, condiciones de ejecución y limitaciones.

1.6.5. Diseño de la evaluación: métricas y muestra

Se propone evaluar la corrección de las reglas, la calidad de decisión, el costo de búsqueda y la generalidad operativa de la arquitectura de la Sección 5.1. Se documentarán los casos, las referencias de comparación y el procedimiento antes de realizar las mediciones. Este diseño complementará los objetivos existentes y mantendrá pendiente el acuerdo sobre el experimento definitivo de Ajedrez. Las posiciones funcionales y los problemas de primera jugada serán comprobaciones complementarias: no sustituirán las partidas contra un motor con ELO configurable previstas en el objetivo específico. Su motor, nivel, número de partidas y presupuesto continuarán pendientes de acuerdo.

Métricas y procedimiento de medición

Para cada comprobación se preparará una entrada y un resultado esperado obtenido de una referencia independiente. Para los indicadores de acierto de movimientos, transiciones, terminalidad, utilidad y decisión se utilizará la proporción de la Ecuación (1.1):

$$P_k = 100\,\frac{C_k}{N_k}, \qquad N_k > 0, \tag{1.1}$$

donde N_k será el número de casos evaluados del indicador k y C_k el número de coincidencias. Los denominadores se reportarán por separado: posiciones, pares estado–acción, estados terminales o decisiones, según corresponda. El tiempo, la profundidad y los nodos se expresarán en sus unidades propias; la generalidad se comprobará mediante evidencia de conservación del núcleo. La Tabla 1.2 precisa cada comprobación.

Tabla 1.2: Métricas y procedimientos propuestos de evaluación.

Aspecto	Procedimiento e indicador
Movimientos legales	Porcentaje de posiciones cuyo conjunto completo de acciones coincida con la referencia. Se registrarán por separado las acciones ilegales generadas y las legales omitidas; igualar únicamente la cantidad no será suficiente.
Transiciones	Porcentaje de pares estado–acción cuyo sucesor coincida con el esperado. Se compararán la configuración, el turno y los datos adicionales exigidos por las reglas, incluidos los antecedentes pertinentes.
Terminalidad y utilidad	Dos porcentajes independientes: clasificación correcta de terminalidad sobre todos los estados evaluados y asignación correcta del vector de utilidades sobre los estados terminales.
Calidad de decisión	En Gato, porcentaje de decisiones pertenecientes al conjunto óptimo calculado por un solucionador independiente con Minimax. En Ajedrez, porcentaje de aciertos de primera jugada: la acción inicial deberá pertenecer al conjunto de respuestas aceptadas y entregarse dentro del presupuesto acordado. Las acciones inválidas y los tiempos agotados contarán como fallos.
Costo de búsqueda	Tiempo por decisión en milisegundos, resumido mediante mediana y percentil 95; número de nodos expandidos y profundidad máxima del árbol explorado, contada en medias jugadas desde la raíz (profundidad cero). Se contará la profundidad de los nodos incorporados al árbol; el horizonte completado de Minimax y la longitud de las simulaciones de MCTS se reportarán aparte cuando correspondan. Se medirá desde la solicitud de acción sobre un estado disponible hasta la respuesta; la carga del archivo se registrará aparte.
Generalidad operativa	Se ejecutarán ambos dominios con la misma versión del intérprete y del motor. Se registrarán los componentes modificados y se revisará que las reglas particulares permanezcan en las descripciones externas. El criterio será conservar el núcleo al cambiar de juego y verificar su funcionamiento.

Para Ajedrez se podrá utilizar una implementación independiente, como *python-chess*, para contrastar movimientos, sucesores y condiciones terminales [16]. Su uso como referencia de pruebas no sustituirá las consultas del intérprete propuesto. Se fijarán su versión y las reglas comparadas. Se propone exigir coincidencia del 100 % en los casos funcionales incluidos; esta condición no demostrará corrección fuera del banco. Los umbrales de calidad y tiempo se justificarán después del piloto.

Selección de la muestra y de los casos

En Gato se propone una revisión exhaustiva de los estados alcanzables desde el tablero vacío, con X al inicio y detención al aparecer una victoria o un empate. Se incluirán los estados terminales para comprobar sus resultados y los no terminales para evaluar decisiones. Se conservarán las configuraciones simétricas como casos distintos y se verificará la enumeración mediante una referencia independiente.

Para Ajedrez se propone un banco piloto de **60 posiciones funcionales**, organizado inicialmente en seis familias de diez casos: movimientos y capturas, jaques y clavadas, enroques, promociones, captura al paso y terminalidad. Cada caso tendrá una familia principal; se cubrirán ambos colores y situaciones que admitan o impidan la acción. La composición y el tamaño se ajustarán cuando se acuerden las reglas soportadas.

Se propone además reservar **30 problemas tácticos para evaluar la primera jugada**, seleccionados por tema y dificultad, con respuestas verificadas. Una fuente posible será la base abierta de Lichess [17]; se preparará la posición aplicando a su FEN el primer movimiento proporcionado, pues la solución comienza con el segundo. Se documentará por caso el conjunto de primeras jugadas aceptadas, incluidas las alternativas válidas. El piloto y los ajustes de parámetros utilizarán casos distintos de estos 30. El indicador medirá el acierto inicial, sin atribuir la resolución de la secuencia completa.

Las cantidades de 60 y 30 son propuestas exploratorias de cobertura, no tamaños que garanticen representatividad estadística de todo Ajedrez. Cada caso conservará identificador, procedencia, posición, regla evaluada y resultado esperado. Las repeticiones de una misma posición se contabilizarán como ejecuciones y no como nuevas posiciones.

Condiciones de comparación y registro

Las configuraciones se compararán sobre los mismos casos y bajo un presupuesto de búsqueda acordado. Se documentarán el algoritmo, los parámetros, la versión de las reglas, los criterios de parada y el tratamiento de los estados terminales. Se contará una expansión cuando se generen sucesores de un nodo; las expansiones y las simulaciones tendrán contadores separados. Si se utiliza aleatoriedad, se repetirán las ejecuciones con semillas registradas; el número definitivo de repeticiones se justificará con la variabilidad observada en el piloto.

Los registros incluirán caso, configuración, repetición, acción elegida, resultado de la comprobación, tiempo, nodos y motivo de terminación. En calidad de decisión, cada combinación de posición y ejecución será un ensayo; se informarán tanto el número de posiciones distintas como el de ensayos por configuración. Los fallos y agotamientos de tiempo permanecerán en el informe, sin excluirlos del denominador de decisiones. La evidencia de generalidad se limitará a los dominios y reglas efectivamente evaluados.

1.6.6. Distribución del trabajo y calendario

Se mantendrá la distribución del protocolo: Sánchez Catarino Edgar se ocupará de la fundamentación del motor, la representación matemática, las reglas y la generación de movimientos, la implementación e integración del agente y el archivo de descripción de Ajedrez. Linares Villegas Gustavo se ocupará del estado del arte y la definición del problema, el diseño de árboles de búsqueda y la función heurística, los experimentos, el análisis estadístico y las conclusiones. Ambos participarán en la redacción de capítulos técnicos y manuales, la revisión final y la preparación de la defensa [15].

Los cronogramas individuales se presentan en el apartado de Cronogramas, dentro de Anexos. Se conservará la secuencia de agosto a junio y el sombreado de las actividades del documento base, sin agregar años, fechas de entrega ni porcentajes de avance que no estén confirmados.

Se deberá revisar una dependencia del calendario: los experimentos de Gustavo están previstos para febrero y marzo, mientras que el archivo de Ajedrez de Edgar está previsto para abril. Antes de ejecutar la validación, se determinará qué experimentos podrán realizarse con los recursos disponibles y cuáles requerirán ese archivo. Esta observación no modifica los periodos del protocolo ni supone que ya exista una descripción parcial de Ajedrez.

Capítulo 2

Estado del arte

2.1. Evolución de los sistemas de juego en inteligencia artificial

En ciencias de la computación, el estudio de los juegos ha seguido dos caminos. El primero, y el más antiguo, es construir motores muy especializados que rindan al máximo en un solo dominio. El segundo, que llegó después, busca agentes generales que entiendan y jueguen distintos juegos a partir de descripciones abstractas [1].

Al principio de esta historia, en 1959, Arthur Samuel mostró que una computadora podía mejorar en las damas inglesas mediante aprendizaje inductivo y una evaluación heurística de posiciones apoyada en Minimax [2]. Su trabajo abrió una larga tradición en la que cada juego se atacaba por separado, afinando a mano las funciones de evaluación y explorando árboles de juego de forma masiva.

Ese enfoque alcanzó su cima en 1997, cuando Deep Blue venció al campeón mundial Garry Kasparov [4]. Combinaba chips hechos a la medida para generar jugadas de ajedrez con una búsqueda alfa-beta muy paralelizada, que revisaba hasta doscientos millones de posiciones por segundo. Su criterio, sin embargo, lo habían calibrado a mano Grandes Maestros: pesos amarrados a las sesenta y cuatro casillas del tablero y al valor de cada pieza. Con Chinook pasó algo parecido en las damas, que resolvió en 2007 apoyándose en bases de datos exhaustivas de finales [5]. Y TD-Gammon llegó a nivel de maestro en backgammon con redes neuronales entrenadas por diferencias temporales.

Desde 2016 llegó otro salto, con AlphaGo [6], AlphaZero [7] y MuZero [18]. AlphaZero mostró que bastaban las reglas básicas para dominar el ajedrez, el Shogi y el Go. Lo hizo uniendo la búsqueda en árboles Monte Carlo (MCTS) con redes neuronales profundas entrenadas por autojuego, sin aperturas ni conocimiento humano experto.

En esta revisión se distinguirá la reutilización de un algoritmo de aprendizaje de la interpretación de reglas externas. AlphaZero comparte aprendizaje y búsqueda entre juegos, pero emplea reglas y codificaciones de estados y acciones propias de cada dominio, además de entrenar una instancia por juego [7]. GGP plantea recibir una descripción formal y razonar sobre ella [8]. Esta comparación precisa el sentido de generalidad que interesa al TT, sin negar la aportación de AlphaZero ni atribuirle una interfaz declarativa que su publicación no propone.

2.2. Surgimiento y fundamentos de General Game Playing

Para atacar de raíz la hiperespecialización, en 2005 la Universidad de Stanford y la Asociación para el Avance de la Inteligencia Artificial (AAAI) lanzaron la iniciativa de *General Game Playing* (GGP) y convocaron a la primera competencia internacional abierta [8]. El documento fundacional, de Genesereth, Love y Pell, planteaba construir agentes autónomos que reciban la especificación formal de un juego cualquiera, desconocido de antemano, y lo jueguen bien sin supervisión humana ni cambios en su código.

Lo que hizo posible este esquema fue el *Game Description Language* (GDL) [9], [10]. El lenguaje se inspira en la programación lógica declarativa, en concreto en el subconjunto de Datalog con negación estratificada, y establece un compromiso claro: todo juego discreto, finito, síncrono y de información perfecta puede describirse como una máquina de estados abstracta usando unos pocos predicados lógicos.

Así transcurre una partida típica de GGP. El agente se conecta a un servidor central, que le manda el archivo de reglas unos minutos antes de empezar. Ese margen lo marca un reloj de preparación (*start clock*) y sirve para procesar la gramática, analizar el juego y armar las representaciones internas. Una vez que arranca la partida, cada turno tiene un reloj de jugada (*play clock*) para explorar el espacio de estados y responder. Con estas condiciones no hay lugar para heurísticas escritas de antemano: el agente tiene que derivar qué movimientos son legales, prever qué provocan y estimar qué tan cerca está del final, todo con la semántica de las reglas que recibió.

2.3. Generaciones y agentes emblemáticos de General Game Playing

Para organizar esta revisión se agrupan los agentes en tres generaciones expositivas: análisis de reglas, búsqueda por simulación y optimización de la simulación. Esta división servirá para comparar enfoques; no se atribuye a la literatura una taxonomía única ni fronteras cronológicas estrictas. Los trabajos originales sustentan las técnicas descritas [19]-[22].

2.3.1. Primera generación: Análisis estático de reglas y Minimax heurístico

En los primeros años, la idea fue llevar al caso general las técnicas clásicas de búsqueda adversarial, Minimax y poda alfa-beta. El problema es que, sin conocer el juego, el agente no tenía una función heurística con la cual evaluar los estados donde el árbol se corta por profundidad.

ClunePlayer, de Jim Clune en la Universidad de California en Los Ángeles, fue el primer campeón de GGP, en la competencia de la AAAI de 2005 [19]. Su aporte fue analizar la estructura sintáctica de las reglas GDL durante el tiempo de preparación para construir automáticamente una función de evaluación posicional. Identificaba heurísticas generales: la estabilidad de hechos (fluents que, una vez alcanzados, ya no pueden destruirse), la movilidad (cuántas jugadas legales tiene cada jugador) y la distancia simbólica a las condiciones de victoria definidas en las metas.

En 2006 el campeón fue Fluxplayer, de Stephan Schiffel y Michael Thielscher, en la Universidad Técnica de Dresde [20]. En lugar de inferencia lógica directa, usaba un razonador basado en el cálculo de fluents (*Fluent Calculus*) junto con aproximaciones de lógica difusa. Así podía

medir cuán cerca estaba un estado cualquiera de cumplir las condiciones de victoria, y esas evaluaciones continuas alimentaban una búsqueda alfa-beta con profundización iterativa. Ambos sistemas tenían, sin embargo, un mismo punto débil: cuando las reglas eran muy combinatorias o no lineales, analizar las cláusulas resultaba muy costoso y producía heurísticas engañosas que empeoraban las decisiones.

2.3.2. Segunda generación: La revolución de Monte Carlo Tree Search

El paso a la segunda generación cambió el campo por completo. Como era difícil inferir heurísticas estáticas confiables, los investigadores empezaron a probar métodos basados en simulación estocástica y en árboles de decisión que se adaptan sobre la marcha.

En 2007, Hilmar Finnsson y Yngvi Björnsson, de la Universidad de Reikiavik, presentaron CadiaPlayer [21]. CadiaPlayer dejó de lado el Minimax guiado por heurísticas sintácticas y aplicó por primera vez en GGP el algoritmo *Monte Carlo Tree Search* (MCTS) con la política de balance UCT (*Upper Confidence Bounds applied to Trees*) [23]. Lo interesante es que el agente no necesitaba entender a fondo las reglas para valorar una posición: simulaba cientos de partidas aleatorias desde el nodo actual hasta el final y promediaba las recompensas para decidir hacia dónde explorar. Ganó el campeonato mundial en 2007, 2008 y de nuevo en 2012, y desde entonces MCTS domina en General Game Playing.

En 2009 el primer lugar fue para Ary, de Jean Méhat en la Universidad de París 8, que implementó un motor MCTS muy paralelizado: corría simulaciones en hilos concurrentes y podaba de forma dinámica las ramas que resultaban estadísticamente desfavorables durante el muestreo.

En esa época apareció también una innovación clave en la representación interna de los estados: las redes proposicionales o PropNets. En vez de resolver consultas lógicas en cada nodo del árbol con un motor de Datalog, las reglas se compilaban en la fase de preparación como una red de compuertas booleanas (`AND`, `OR`, `NOT`) alojada directamente en memoria. Calcular el siguiente estado dejaba de ser un problema de resolución lógica y pasaba a ser la propagación de bits por un circuito, con lo que un motor MCTS podía correr varios órdenes de magnitud más simulaciones por segundo. La Figura 2.1 contrasta la deducción lógica interpretada con la aceleración por propagación de compuertas booleanas.

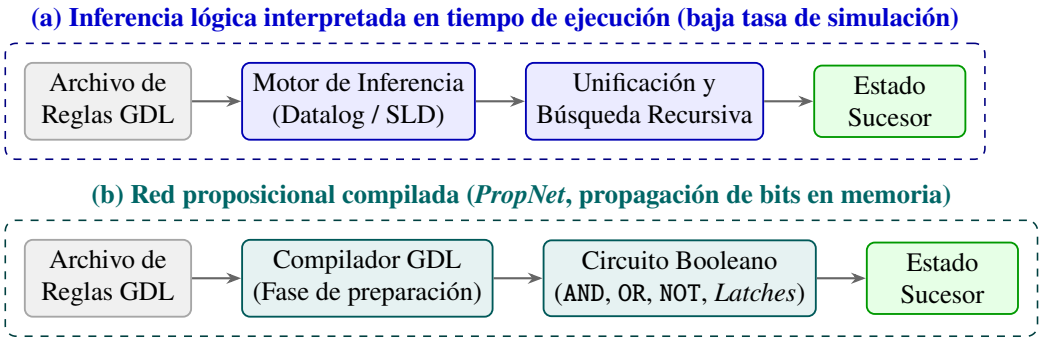


Figura 2.1: Comparación esquemática entre la inferencia lógica interpretada recursivamente y la evaluación acelerada por propagación de señales en redes proposicionales (*PropNets*).

2.3.3. Tercera generación: PropNets optimizadas y heurísticas de simulación

La tercera generación se dedicó a pulir el motor de simulación. El objetivo era superar la lentitud de las partidas puramente aleatorias en juegos con espacios de búsqueda inmensos o ramas terminales muy profundas.

Sancho, desarrollado por Steve Draper y Andrew Rose, ganó la competencia internacional de GGP de 2014, según la página de sus autores [24]. Este antecedente se distinguirá de las técnicas que se describen en publicaciones independientes: MAST utiliza estadísticas históricas de acciones para orientar las simulaciones [21], mientras RAVE reutiliza estimaciones de acciones observadas durante la búsqueda [25]. Su estudio no implica que ambas técnicas estuvieran presentes en Sancho ni que se implementarán en este TT.

La Figura 2.2 resume cómo se pasa de una generación a otra y cómo todas desembocan en los formalismos actuales.

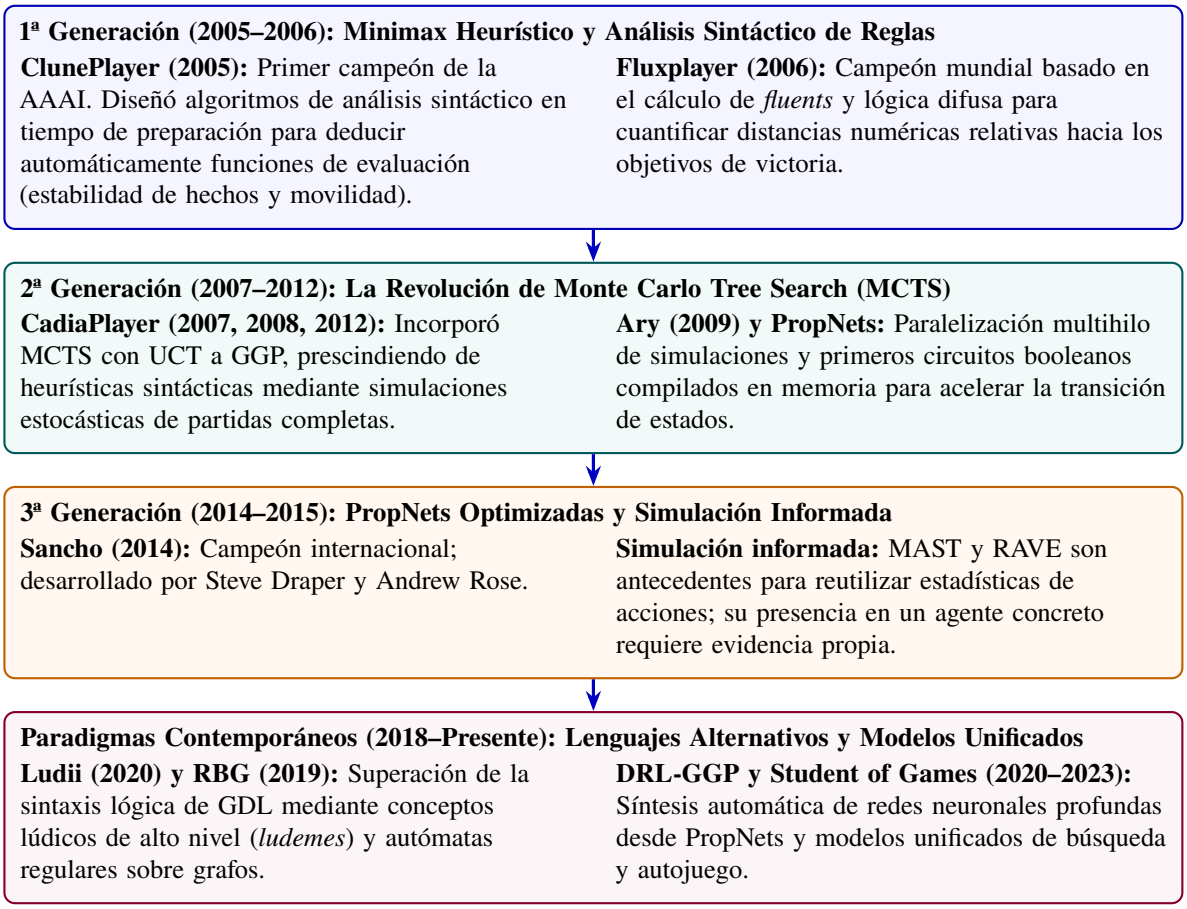


Figura 2.2: Organización expositiva de antecedentes y generaciones algorítmicas en *General Game Playing* (GGP).

2.4. Plataformas y lenguajes modernos de descripción de juegos

GDL clásico llegó a ser el estándar académico para juegos discretos de información perfecta. Con su sintaxis prefija, heredada de KIF, y su semántica puramente declarativa, sin embargo, se volvía lento en tableros grandes o con mecánicas complejas. En la última década han aparecido plataformas y lenguajes alternativos que responden a esa limitación y han ampliado lo que se investiga.

2.4.1. Extensiones de GDL e información imperfecta

Para cubrir juegos con azar y con información oculta, Michael Thielscher formalizó en 2010 la extensión GDL-II (*Game Description Language for Incomplete Information and Stochastic Games*) [26]. Añadió al lenguaje los eventos estocásticos, mediante un rol especial llamado `random` que emite jugadas con distribución uniforme, como al tirar un dado, y la visibilidad parcial, mediante el predicado `sees`, que modela la función de observación $O_i : S \to \Omega_i$ para ocultar cartas o piezas al adversario. La expresividad creció bastante, pero también la complejidad del razonador, pues los agentes deben rastrear conjuntos de información de Kuhn y sostener creencias probabilísticas sobre los estados que no ven.

2.4.2. El sistema Ludii

Ludii se desarrolló en la Universidad de Maastricht bajo la dirección de Cameron Browne y es uno de los avances más notables en la descripción general de juegos de mesa modernos [27], [28]. En lugar de la lógica de predicados pura de GDL, usa *ludemes*, unidades conceptuales que agrupan información lúdica de alto nivel.

El lenguaje incorpora conceptos de tableros, piezas, movimientos y condiciones de fin. Piette y colaboradores compararon Ludii con un razonador GDL basado en PropNet en los juegos de Tiltyard: reportaron mayor velocidad de simulación para las descripciones y condiciones evaluadas, sin demostrar una superioridad para todo juego o implementación [27]. Soemers y colaboradores documentan más de mil descripciones en la biblioteca de su estudio de 2024 [28]; esa cifra corresponde a la publicación, no a un inventario actualizado del sistema.

2.4.3. Regular Boardgames

Regular Boardgames (RBG) es una propuesta de Jakub Kowalski, Maksymilian Mika, Jakub Sutowicz y Marek Szykuła, presentada en la conferencia AAAI de 2019. Se trata de un formalismo para juegos de tablero que se construye sobre lenguajes regulares y autómatas finitos [29].

En RBG, las reglas de movimiento y las condiciones de legalidad de cada pieza se escriben como expresiones regulares que describen trayectorias válidas sobre una cuadrícula vista como un grafo conexo. Los autores probaron que RBG es universal para juegos finitos, deterministas, por turnos y de información perfecta. Lo más llamativo es su rendimiento: los analizadores basados en autómatas generan movimientos legales y aplican transiciones entre dos y tres órdenes de

magnitud más rápido que los razonadores lógicos de GDL, lo que lo hace muy adecuado para experimentos intensivos de búsqueda y de aprendizaje por refuerzo.

2.4.4. Entornos generales para aprendizaje: OpenSpiel

En el campo del aprendizaje automático actual, DeepMind liberó en 2019 la plataforma de código abierto OpenSpiel [30]. No es un lenguaje declarativo de reglas pensado para usuarios finales, sino un marco algorítmico en C++ y Python con una colección común de juegos e interfaces estándar para investigar aprendizaje por refuerzo, teoría de juegos algorítmica y búsqueda en árboles. Su valor está en ofrecer implementaciones de referencia de algoritmos modernos, como *Counterfactual Regret Minimization*, *Policy Gradients* y MCTS, con lo que se pueden comparar agentes en condiciones metodológicas homogéneas.

2.5. Tendencias contemporáneas en aprendizaje profundo y modelos unificados

Últimamente la investigación intenta juntar dos cosas: la descripción declarativa de juegos y las técnicas de aprendizaje profundo que transformaron a los motores especializados.

Goldwaser y Thielscher estudiaron el aprendizaje por refuerzo profundo en GGP [31]. Su modelo recibe valores de nodos de una PropNet y construye automáticamente capas completamente conectadas, con salidas de política y valor por rol. El algoritmo 1 de su artículo determina tamaños de capas a partir de las entradas y acciones; no reproduce la topología de compuertas como una red convolucional. Los autores explican que las convoluciones requieren un orden geométrico que GDL no proporciona necesariamente. El entrenamiento utiliza autojuego y MCTS; se trata de un antecedente y no de una arquitectura neuronal seleccionada para este TT.

Student of Games, publicado por Schmid y colaboradores en 2023, combina búsqueda, aprendizaje por autojuego y razonamiento de teoría de juegos para estudiar Ajedrez, Go, póker y Scotland Yard [14]. La publicación distingue configuraciones de entrenamiento y de evaluación. Sus recursos y tiempos experimentales no se presentarán como un requisito universal para ejecutar cualquier agente basado en ese enfoque. En este TT sirve como antecedente de generalidad algorítmica; su aprendizaje profundo no forma parte de la implementación comprometida.

2.6. Síntesis comparativa y posicionamiento del Trabajo Terminal

Después de este recorrido se pueden comparar los enfoques principales de la literatura, como se resume en la Tabla 2.1.

Tabla 2.1: Comparación de paradigmas y sistemas representativos en juegos e inteligencia artificial.

Sistema / Paradigma	**Entrada de reglas**	**Motor de decisión**	**Tipo de juego**	**Dependencia de infraestructura**
Deep Blue [4]	Código nativo cableado	Alfa-beta especializado	Ajedrez (info. perfecta)	Hardware dedicado a la medida
AlphaZero [7]	Reglas y codificación por dominio	MCTS + Redes profundas	Varios (info. perfecta)	Entrenamiento y evaluación con recursos distintos
ClunePlayer [19]	Reglas lógicas (GDL)	Minimax con heurística aprendida	General GGP (info. perfecta)	Estación de trabajo estándar
CadiaPlayer [21]	Reglas lógicas (GDL)	MCTS con UCT sin heurística	General GGP (info. perfecta)	Estación de trabajo estándar
Sancho [24]	Reglas lógicas (GDL)	Agente de competencia GGP	General GGP (info. perfecta)	Estación de trabajo estándar
Ludii [27], [28]	Sintaxis de ludemes	MCTS / UCT / Alpha-beta	General (tableros y cartas)	Entorno Java de escritorio
RBG [29]	Expresiones regulares	Búsqueda con autómatas finitos	General (tableros)	Entorno C++ de alto rendimiento
Student of Games [14]	Reglas y codificación por dominio	Búsqueda + Teoría de juegos	Perfecta e imperfecta	Configuraciones experimentales por juego
Presente TT (Propuesta)	**Lenguaje declarativo (DSL/GDL)**	**Minimax / Alfa-Beta / MCTS desacoplado**	**General (información perfecta)**	**Estación de trabajo estándar**

Del análisis se desprende dónde se ubica este Trabajo Terminal.

Primero, se propone aplicar el principio de General Game Playing: el motor de decisión deberá operar sobre el modelo construido a partir de reglas externas, sin variables ni heurísticas codificadas para un dominio particular.

Segundo, el TT se centrará en la interpretación declarativa y la búsqueda acotada. La comparación con AlphaZero y Student of Games no implica que sus costos de entrenamiento sean los mismos que los de tomar decisiones con un modelo entrenado. La generalidad operativa del sistema propuesto se comprobará conservando el núcleo y cambiando la descripción externa.

Por último, se propone una representación interna consultable y una separación entre reglas y decisión. El lenguaje y los algoritmos definitivos requerirán justificar sus alternativas; la presencia de GDL, Minimax o MCTS en esta revisión no los convierte en decisiones de implementación. Con ello se busca fundamentar un agente general dentro del alcance documentado.

Capítulo 3

Marco Teórico

3.1. Inteligencia artificial en juegos

Los juegos se han usado una y otra vez como banco de pruebas para la inteligencia artificial, porque tienen reglas explícitas, objetivos medibles y resultados que se pueden verificar. Desde los primeros trabajos de Arthur Samuel sobre aprendizaje automático en el juego de damas [2], han servido para estudiar búsqueda, planeación, toma de decisiones, aprendizaje y evaluación de estrategias.

Hay otra razón por la que los juegos importan: en un entorno controlado reúnen problemas que aparecen también fuera de ellos. Hay que elegir acciones bajo restricciones, anticipar consecuencias, competir con otros agentes, repartir el cómputo y decidir sin haber explorado todas las posibilidades. En un juego pequeño como el Gato todavía se puede recorrer buena parte del espacio de estados; en el Ajedrez, el número de estados y de secuencias de jugadas crece de forma combinatoria y sin búsqueda ni evaluación no hay manera de avanzar.

Históricamente, muchos de los sistemas más exitosos han sido motores especializados. Rinden bien porque llevan dentro conocimiento del dominio: reglas programadas para un juego, funciones de evaluación ajustadas a sus piezas o posiciones, representaciones hechas a la medida y optimizaciones para un tablero concreto. Eso da agentes fuertes, pero rígidos. Un programa hecho para jugar Ajedrez no puede, sin más, jugar Damas, Go o Gato, y no por falta de capacidad de cálculo, sino porque lo que sabe está pegado al juego para el que se construyó.

De ahí nace el interés por agentes que jueguen más de un juego sin reprogramar el motor, y esa línea de investigación es *General Game Playing*. Una parte del trabajo intelectual se mueve del programador al agente: ya no se espera que alguien codifique cada regla a mano, sino que el agente sepa leer la descripción formal del juego y razonar sobre ella.

3.2. Juegos de información perfecta

En teoría de juegos y en inteligencia artificial [1], [32], [33], algo que separa a los juegos es cuánto ven los jugadores del entorno. En un juego de información perfecta, quien debe decidir conoce la historia completa del juego hasta ese nodo de decisión. Sus conjuntos de información son unitarios. Esta propiedad no exige por sí sola estados discretos, ausencia de azar ni una

determinada alternancia; esas restricciones se adoptan adicionalmente en el alcance de este proyecto. El Ajedrez, las Damas y el Gato son los ejemplos de siempre.

Para formalizar esta idea con el modelo de juego (Γ) de la Sección 3.4, consideremos un conjunto de jugadores $R = \{r_1, r_2, \ldots, r_n\}$ y un conjunto de estados posibles S, donde cada estado $s \in S$ es la configuración completa y real del juego en un instante dado.

3.2.1. Espacio de observaciones y función de observación (O_i)

Conviene distinguir entre lo que el entorno es realmente y lo que cada jugador alcanza a percibir de él. Llamamos **estado real** ($s \in S$) a la configuración completa y verdadera del juego, que incluye todo lo que existe en ese instante: la posición de cada pieza, las cartas que cada participante tiene en la mano (también las que los demás no ven), el tiempo que queda en el reloj y a quién le toca mover. La **observación** ($o \in \Omega_i$), por su parte, es la información, parcial o total, que el jugador r_i recibe de ese estado, es decir, lo que ve en su pantalla o sobre la mesa. El conjunto Ω_i reúne todas las observaciones posibles para ese jugador.

La relación entre la realidad del juego y lo que ve el participante se modela con la **función de observación** [1], [26]:

$$O_i : S \to \Omega_i \tag{3.1}$$

que toma el estado real $s \in S$ y devuelve la observación $O_i(s) \in \Omega_i$ que ve el jugador r_i. Se puede pensar en O_i como un filtro colocado desde el punto de vista del jugador: aunque el juego esté en la configuración completa s, solo pasa lo que las reglas le permiten ver. En un juego de cartas, por ejemplo, el filtro borra las cartas del adversario y deja las propias y las comunitarias.

3.2.2. Relación de indistinguibilidad ($\sim_i$)

A partir de la función de observación se puede establecer con precisión cuándo dos situaciones distintas del juego resultan indiscernibles para un participante [33].

Definición 3.1: Relación de indistinguibilidad

Dado un conjunto de estados S, un espacio de observaciones Ω_i y una función de observación $O_i : S \to \Omega_i$ [26], [32], [33], la **relación de indistinguibilidad** $\sim_i \subseteq S \times S$ para el jugador $r_i \in R$ se define como:

$$s \sim_i s' \iff O_i(s) = O_i(s') \tag{3.2}$$

Dos estados $s, s' \in S$ son indistinguibles para el jugador r_i si y solo si producen exactamente la misma proyección perceptual en su codominio de observaciones.

Que dos estados s y s' sean indistinguibles para un jugador quiere decir que lo que los diferencia queda fuera de su campo de visión. Piénsese en un juego de cartas: en el estado s el adversario tiene un As oculto y en s' tiene un Rey oculto. El adversario los distingue sin problema, porque ve su propia mano. Para el jugador r_i, en cambio, la carta del rival está boca abajo y recibe exactamente la misma vista en ambos casos, $O_i(s) = O_i(s')$, de modo que no puede saber en cuál de las dos situaciones se encuentra.

3.2.3. La indistinguibilidad como relación de equivalencia

Para agrupar los estados que un jugador confunde entre sí hay que demostrar que $\sim_i$ es una relación de equivalencia sobre el espacio de estados [33].

Proposición 3.1. *Para todo jugador $r_i \in R$, la relación binaria de indistinguibilidad $\sim_i$ es una relación de equivalencia sobre el conjunto de estados S.*

Demostración (véase [33]). Se debe verificar que $\sim_i$ satisface las tres propiedades fundamentales:

1. **Reflexividad ($\forall s \in S : s \sim_i s$):** Sea $s \in S$ un estado arbitrario. La observación que produce dicho estado es idéntica a sí misma:

   $$O_i(s) = O_i(s)$$

   Por tanto, $s \sim_i s$ para todo $s \in S$.

   Un estado es indistinguible de sí mismo bajo la misma función de observación.

2. **Simetría ($\forall s, s' \in S : s \sim_i s' \implies s' \sim_i s$):** Sean $s, s' \in S$ tales que $s \sim_i s'$. Por definición, ambos estados producen la misma observación: $O_i(s) = O_i(s')$. En virtud de la simetría de la igualdad matemática, se tiene:

   $$O_i(s') = O_i(s)$$

   lo que implica que $s' \sim_i s$.

   La confusión es recíproca: si la vista de s no permite descartar que el juego esté en s', la vista de s' tampoco permite descartar que esté en s.

3. **Transitividad ($\forall s, s', s'' \in S : (s \sim_i s' \land s' \sim_i s'') \implies s \sim_i s''$):** Sean $s, s', s'' \in S$ tales que $s \sim_i s'$ y $s' \sim_i s''$. Esto significa que:

   $$O_i(s) = O_i(s') \quad \text{y} \quad O_i(s') = O_i(s'')$$

   Por la propiedad transitiva de la igualdad matemática:

   $$O_i(s) = O_i(s'')$$

   lo cual demuestra que $s \sim_i s''$.

   Si s se ve igual que s' y s' se ve igual que s'', los tres estados se ven idénticos para el jugador, y por tanto s y s'' son indistinguibles.

Al satisfacerse la reflexividad, simetría y transitividad, queda formalmente demostrado que $\sim_i$ es una relación de equivalencia sobre S. ■

3.2.4. Conjuntos de información y clases de observación

En un juego en forma extensiva, una historia h identifica una secuencia de acciones y resultados hasta un nodo. Sea H_i el conjunto de historias en las que debe decidir r_i. Un conjunto de información $I_i(h) \subseteq H_i$ agrupa los nodos que ese jugador no distingue a partir de la información que ha recibido, incluido su historial pertinente [32], [33]. No se define, en general, únicamente por la observación del estado presente.

Definición 3.2: Juego de información perfecta

Un juego en forma extensiva es de **información perfecta** cuando cada conjunto de información de cada jugador en sus nodos de decisión es unitario [32], [33]:

$$\forall r_i \in R, \quad \forall h \in H_i : \quad I_i(h) = \{h\}. \tag{3.3}$$

El jugador conoce en qué nodo está al decidir; esto no elimina la incertidumbre sobre las decisiones futuras del adversario ni sobre resultados futuros de azar, si el juego los admite.

Para ilustrar la observación de estados, la relación $\sim_i$ definida antes induce una partición distinta:

$$C_i = S/\sim_i = \{[s]_{\sim_i} \mid s \in S\}. \tag{3.4}$$

Definición 3.3: Clase de estados compatibles con una observación

Para $s \in S$, se denota por $C_i(s)$ la clase de estados que producen la misma observación actual:

$$C_i(s) = [s]_{\sim_i} = \{s' \in S \mid O_i(s') = O_i(s)\}. \tag{3.5}$$

Esta clase describe compatibilidad con una observación de estado; no debe identificarse automáticamente con $I_i(h)$. Historias diferentes pueden llegar a una misma configuración, y un jugador puede conservar información que no aparece en su observación actual.

Proposición 3.2 (Observación completa del estado). *La función $O_i : S \to \Omega_i$ es inyectiva si y solo si $C_i(s) = \{s\}$ para todo $s \in S$.*

Demostración. Si O_i es inyectiva, la igualdad $O_i(s') = O_i(s)$ implica $s' = s$, por lo que la clase contiene únicamente s. Recíprocamente, si todas las clases son unitarias y dos estados producen la misma observación, ambos pertenecen a la misma clase y deben coincidir. ■

La condición de inyectividad se expresa como:

$$\forall s, s' \in S : \quad O_i(s) = O_i(s') \implies s = s'. \tag{3.6}$$

En ese caso, cada observación efectivamente producida identifica un único estado. Para $o \in O_i(S)$ existe una inversa sobre la imagen:

$$s = \left(O_i\big|_{S \to O_i(S)}\right)^{-1}(o). \tag{3.7}$$

Esta inversa no se extiende a observaciones ajenas a la imagen. En el modelo del proyecto se supondrá que el estado y los antecedentes públicos necesarios para aplicar las reglas están disponibles. La observación completa de esa representación se distinguirá de la definición sobre historias de la Ecuación (3.3).

3.2.5. Ejemplos comparativos

Dos ejemplos ayudan a ver la diferencia.

En el Gato ($R = \{\mathrm{x}, \mathrm{o}\}$), que es un juego de información perfecta, el tablero de 3×3 está completamente a la vista de ambos jugadores. Si el estado real es s (por ejemplo, una ficha en el centro), la función de observación de cada participante devuelve exactamente ese tablero: $O_{\mathrm{x}}(s) = s$ y $O_{\mathrm{o}}(s) = s$. Sus clases de observación del estado son, por tanto,

$$C_{\mathrm{x}}(s) = \{s\}, \quad C_{\mathrm{o}}(s) = \{s\} \tag{3.8}$$

Ninguno duda de qué casillas están marcadas; los dos ven toda la realidad del juego a través de un mapeo inyectivo directo. Lo mismo vale para el Ajedrez.

La Batalla Naval, en cambio, es de información imperfecta. Supongamos que el jugador 1 coloca su barco en la coordenada $(1, 1)$ (estado s_A) o en la $(1, 2)$ (estado s_B). Él sabe dónde puso su nave, así que distingue las dos situaciones: $C_1(s_A) = \{s_A\}$ e $C_1(s_B) = \{s_B\}$. El jugador 2, sin embargo, tiene la cuadrícula rival cubierta por niebla, y para él ambos tableros son idénticos: $O_2(s_A) = O_2(s_B) = o$. La función no es inyectiva ($s_A \sim_2 s_B$) y resulta

$$C_2(s_A) = C_2(s_B) = \{s_A, s_B\} \tag{3.9}$$

La clase de observación del jugador 2 tiene dos estados ($|C_2| = 2 > 1$). No sabe si el barco enemigo está en $(1, 1)$ o en $(1, 2)$ y se ve obligado a adivinar o a estimar probabilidades.

La Figura 3.1 ilustra esquemáticamente esta distinción mediante clases de estados compatibles con observaciones inyectivas y no inyectivas. Es una ilustración de observación de estados, no un árbol de conjuntos de información sobre historias.

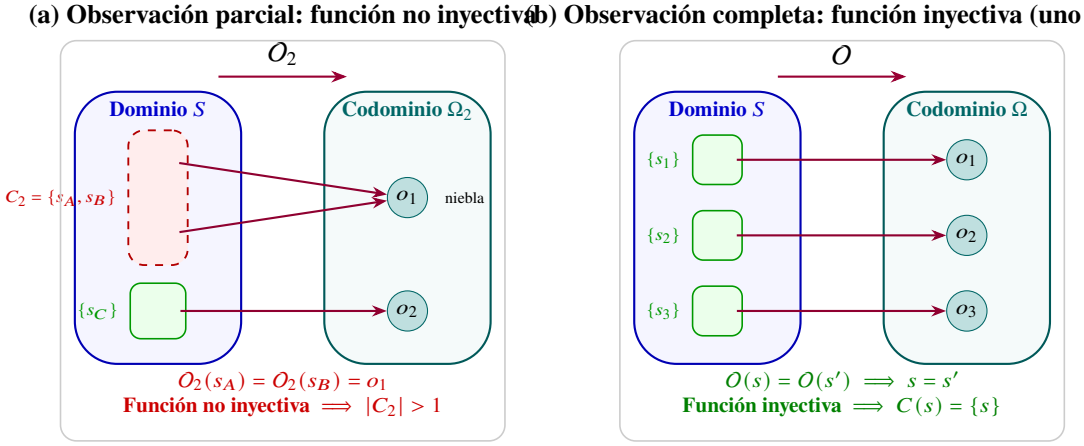


Figura 3.1: Clases de estados compatibles con una observación. En (a), dos estados producen la misma observación y pertenecen a una clase no unitaria; en (b), la observación inyectiva identifica cada estado. La figura ilustra observación de estados, no conjuntos de información sobre historias.

3.2.6. Implicación para el modelo formal Γ

La información perfecta permite razonar sin incertidumbre sobre la historia observada en el nodo de decisión. En el modelo de estados del proyecto se conservarán el turno y los antecedentes necesarios para que las consultas $L_i(s)$, $T(s, \mathbf{a})$ y $G_i(s)$ tengan argumentos suficientes. En

Ajedrez, observar únicamente la colocación de piezas no sustituye los derechos de enroque ni los antecedentes requeridos por las reglas de repetición o captura al paso.

El determinismo de T es un supuesto adicional del alcance, independiente de la Ecuación (3.3). Por ello una acción conjunta legal producirá un único sucesor. El agente seguirá enfrentando incertidumbre estratégica sobre las acciones del rival y, si se estudia MCTS, podrá emplear muestreo aleatorio dentro de un juego de dinámica determinista.

3.3. General Game Playing

Un General Game Player (GGP) es un agente que recibe la descripción formal de un juego y debe jugarlo sin ayuda humana ni conocimiento específico programado de antemano para ese dominio. Genesereth, Love y Pell definen el área como el estudio de sistemas capaces de aceptar la descripción formal de un juego y jugarlo bien sin depender de algoritmos diseñados para juegos particulares [8].

Lo que separa a un motor especializado de un General Game Player es dónde vive el conocimiento del juego. En el motor especializado, las reglas y heurísticas suelen estar dentro del código fuente. En un GGP, las reglas llegan como entrada externa, y el agente debe interpretarlas, construir su propia representación interna y aplicar un proceso de decisión general. Por eso la generalidad no consiste solo en poder ejecutar dos juegos: el mismo núcleo algorítmico tiene que funcionar cuando cambia la descripción del juego.

De aquí se sigue algo para este Trabajo Terminal: no queremos un motor de Ajedrez que rivalice con los sistemas especializados, sino una arquitectura desacoplada. Lo que hay que demostrar es que el sistema carga reglas externas, genera movimientos legales, reconoce estados terminales y escoge jugadas con un criterio general de búsqueda. Y la prueba de que es general está en cambiar el archivo de reglas sin tocar el motor.

3.4. Modelo formal de juego

Para el alcance adoptado, se utilizará un modelo de transición finito y determinista, inspirado en GGP clásico [8]-[10]. Las acciones conjuntas ofrecen una representación computacional uniforme; los turnos secuenciales se codificarán con acciones nulas de los roles inactivos. Esta representación no supone que los participantes del TT decidan simultáneamente.

Definición 3.4: Juego en General Game Playing

Un juego formal Γ es un sistema de transición de estados finito, determinista, síncrono y de información perfecta caracterizado por una tupla de ocho componentes [8]-[10]:

$$\Gamma = \langle R, S, s_0, \{A_i\}_{i=1}^n, \{L_i\}_{i=1}^n, T, F, \{G_i\}_{i=1}^n \rangle$$

donde:

- $R = \{r_1, r_2, \ldots, r_n\}$ es el conjunto finito de roles o participantes ($|R| = n \geq 1$).
- S es el conjunto finito y no vacío de estados posibles del entorno.

- $s_0 \in S$ es el estado inicial distinguido en el instante $t = 0$.
- A_i es el catálogo finito de acciones primitivas del participante r_i, conformando el espacio conjunto $\mathbf{A} = \prod_{i=1}^{n} A_i$.
- $L_i : S \to \mathcal{P}(A_i)$ es la función de legalidad que asigna a cada estado las acciones permitidas para el participante r_i.
- $T : S \times \mathbf{A} \to S$ es la función de transición dinámica que mapea el estado actual y un vector de acciones conjuntas a un único estado sucesor.
- $F \subseteq S$ es el subconjunto de estados terminales.
- $G_i : S \to [0, 100]$ es la función de utilidad o meta que asigna la recompensa numérica alcanzada por el rol r_i.

A continuación se detalla la semántica y formalización de cada uno de los componentes de la tupla.

3.4.1. Conjunto de roles (R)

El conjunto $R = \{r_1, r_2, \ldots, r_n\}$ es finito y no vacío, con cardinalidad $|R| = n \geq 1$, y sus elementos son los participantes del juego. En *General Game Playing*, esos elementos no tienen para el motor de razonamiento ningún significado previo ("humano", "blancas" o "máquina"); son identificadores abstractos. En el Gato, por ejemplo, $R = \{\text{x}, \text{o}\}$, y en el Ajedrez, $R = \{\text{blancas}, \text{negras}\}$. En GDL cada participante se declara con el predicado `(role r)`.

3.4.2. Espacio de estados (S) y semántica de fluents

El espacio de estados $S = \{s_1, s_2, \ldots, s_k\}$ es un conjunto finito y no vacío con todas las configuraciones admisibles del sistema. En lugar de tratar cada estado como un identificador opaco e indivisible, en *General Game Playing* se descompone en proposiciones estructuradas, a partir del concepto de *fluent* [9], [34].

Definición 3.5: Fluent

Un *fluent* es una proposición lógica atómica (indivisible) $f \in \mathcal{F}$ —concepto originado en el cálculo de situaciones por McCarthy y Hayes [34] y adoptado como base formal en GGP [9], [10]—, cuyo valor de verdad no es estático ni constante, sino una función dependiente del tiempo o del estado del sistema:

$$\text{val}(f, t) \in \{\text{Verdadero}, \text{Falso}\}$$

A diferencia de las relaciones estáticas del dominio, que valen igual durante toda la partida (las dimensiones del tablero o las reglas de adyacencia), un *fluent* representa un hecho contingente del entorno: puede aparecer, mantenerse o desaparecer según las transiciones que provocan las acciones ejecutadas.

Definición 3.6: Estado

En un entorno determinista y de información perfecta, el estado de un juego en el instante t se define lógicamente como la conjunción lógica ($\bigwedge$) de todos los *fluents* atómicos que son verdaderos simultáneamente en ese instante:

$$s_t \equiv \bigwedge_{i=1}^{m} f_i = f_1 \wedge f_2 \wedge \cdots \wedge f_m$$

Bajo la perspectiva conjuntista de la programación lógica declarativa y el principio de mundo cerrado (*Closed World Assumption*) formalizado por Reiter [35], el estado se modela de forma compacta como el subconjunto $s \subseteq \mathcal{F}$ de hechos atómicos verdaderos en dicho instante [9], [10]. En consecuencia, el espacio global de estados satisface:

$$S \subseteq \mathcal{P}(\mathcal{F})$$

donde cualquier *fluent* $f \notin s$ se evalúa formalmente como falso.

Ejemplo ilustrativo en el juego del Gato

Para ver esta semántica en un caso concreto, tomemos el juego del Gato (*Tic-Tac-Toe*) sobre una retícula de 3×3.

El universo de fluents $\mathcal{F}$ tiene dos clases de proposiciones atómicas. Las primeras describen la ocupación de cada casilla: $\mathcal{F}_{\text{celdas}} = \{\text{cell}(r, c, m) \mid r \in \{1, 2, 3\},\ c \in \{1, 2, 3\},\ m \in \{\text{x}, \text{o}, \text{b}\}\}$, donde b indica una casilla vacía (*blank*); son $3 \times 3 \times 3 = 27$ átomos posibles. Las segundas indican a quién le toca jugar: $\mathcal{F}_{\text{turno}} = \{\text{control}(\text{x}), \text{control}(\text{o})\}$, que son 2 átomos. Así, $\mathcal{F} = \mathcal{F}_{\text{celdas}} \cup \mathcal{F}_{\text{turno}}$, con $|\mathcal{F}| = 29$.

Al empezar la partida, las nueve casillas están vacías y el turno es de x. El estado inicial s_0 es entonces la conjunción de diez fluents verdaderos:

$$s_0 \equiv \left(\bigwedge_{r=1}^{3} \bigwedge_{c=1}^{3} \text{cell}(r, c, \text{b}) \right) \wedge \text{control}(\text{x}) \tag{3.10}$$

o, en su representación como conjunto:

$$s_0 = \{\text{cell}(1, 1, \text{b}), \text{cell}(1, 2, \text{b}), \ldots, \text{cell}(3, 3, \text{b}), \text{control}(\text{x})\} \tag{3.11}$$

Proposiciones como $\text{cell}(2, 2, \text{x})$ no pertenecen a s_0 y, por lo tanto, son falsas en $t = 0$.

Si x marca el centro con la acción $\text{mark}(2, 2)$, la función de transición quita los fluents invalidados y agrega los nuevos:

$$s_1 = \big(s_0 \setminus \{\text{cell}(2, 2, \text{b}), \text{control}(\text{x})\}\big) \cup \{\text{cell}(2, 2, \text{x}), \text{control}(\text{o})\} \tag{3.12}$$

Como conjunción, la nueva configuración s_1 queda así:

$$s_1 \equiv \text{cell}(2, 2, \text{x}) \wedge \text{control}(\text{o}) \wedge \bigwedge_{(r,c) \neq (2,2)} \text{cell}(r, c, \text{b}) \tag{3.13}$$

Las ocho celdas vacías restantes se mantienen igual por inercia.

En GDL, para preguntar si un *fluent* vale en el estado actual se usa `(true f)`. De cómo empieza el juego y de cómo cambia se ocupan `(init f)` y `(next f)`; más adelante, en los listados del Gato, se ven los tres en acción [9], [10].

3.4.3. Estado inicial (s_0)

El elemento distinguido $s_0 \in S$ es la configuración del entorno en el instante $t = 0$, antes de que los agentes tomen ninguna decisión. En una descripción declarativa, s_0 se construye con todos los hechos que se declaran mediante el predicado `(init f)`.

3.4.4. Acciones individuales (A_i) y espacio conjunto de acciones (A)

Para cada rol $r_i \in R$, el conjunto finito $A_i = \{a_{i,1}, a_{i,2}, \ldots, a_{i,m_i}\}$ reúne las acciones primitivas que ese participante puede concebir. Como el modelo es síncrono, en cada paso de tiempo todos los jugadores eligen una acción a la vez, y el espacio global de decisiones es el producto cartesiano de los conjuntos individuales:

$$\mathbf{A} = \prod_{i=1}^{n} A_i = A_1 \times A_2 \times \cdots \times A_n \tag{3.14}$$

Un elemento $\mathbf{a} = (a_1, a_2, \ldots, a_n) \in \mathbf{A}$ es una acción conjunta, un vector de decisiones concurrentes en el que cada componente $a_i \in A_i$ es la jugada del rol r_i.

3.4.5. Función de legalidad (L_i)

Que una acción sea válida depende del estado actual del sistema. La legalidad de cada jugador r_i se modela con la correspondencia:

$$L_i : S \to \mathcal{P}(A_i) \tag{3.15}$$

que asigna a cada configuración $s \in S$ el subconjunto de acciones permitidas $L_i(s) \subseteq A_i$. En su formulación relacional original [8], [9], el criterio se expresa como una relación binaria $L_i \subseteq A_i \times S$, en la que $(a, s) \in L_i$ significa que la acción a se puede ejecutar en s; corresponde directamente al predicado `(legal r a)` [10].

Una acción conjunta $\mathbf{a} = (a_1, \ldots, a_n)$ es legal en el estado s si y solo si cada componente es legal para su rol:

$$\mathbf{a} \in \mathbf{L}(s) \iff \forall i \in \{1, \ldots, n\}, \quad a_i \in L_i(s) \tag{3.16}$$

donde $\mathbf{L}(s) = \prod_{i=1}^{n} L_i(s)$ es el espacio de jugadas conjuntas válidas en ese estado.

3.4.6. Función de transición dinámica (T)

La evolución del juego en el tiempo la rige la función de transición determinista:

$$T : S \times \mathbf{A} \to S \tag{3.17}$$

que determina de forma única el estado sucesor $s' = T(s, \mathbf{a})$ que resulta de aplicar la acción conjunta $\mathbf{a} \in \mathbf{L}(s)$ sobre la configuración s. Dos propiedades la caracterizan. Es estrictamente determinista: a diferencia de los procesos estocásticos o del control continuo, evaluar las reglas produce un único estado resultante y no hay incertidumbre en la física del sistema. Y es reactiva y síncrona: el universo cambia solo como consecuencia de las decisiones conjuntas de los agentes, algo que en GDL se especifica con las reglas de deducción de hechos futuros `(next f)`.

3.4.7. Conjunto de estados terminales (F)

El subconjunto $F \subseteq S$ marca las condiciones en las que termina una partida. Cuando el sistema llega a una configuración $s \in F$, la trayectoria se detiene: ya no se admiten más movimientos y se calculan los resultados finales. En los juegos de mesa, este conjunto incluye situaciones como el jaque mate, las tablas por repetición, una línea ganadora o el tablero lleno. En el motor lógico se verifica con el predicado `(terminal)`.

3.4.8. Función de utilidad o meta (G_i)

Para cada participante $r_i \in R$, la función de utilidad o pago:

$$G_i : S \to [0, 100] \tag{3.18}$$

Aunque formalmente se puede evaluar en cualquier estado, la función tiene sentido sobre todo en los estados terminales $s \in F$. En GDL clásico los valores de meta son enteros entre 0 y 100 [10]. Para los ejemplos de Gato y Ajedrez se adoptará la convención 100 para victoria, 0 para derrota y 50 para empate; la interpretación de esos valores deberá declararse en cada juego. En este modelo se consultará la utilidad como resultado de la partida en los estados terminales; no se confundirá con una evaluación heurística de estados no terminales.

En los juegos adversariales de dos participantes y suma constante, como el Ajedrez y el Gato, las utilidades terminales cumplen:

$$G_1(s) + G_2(s) = 100, \quad \forall s \in F \tag{3.19}$$

lo cual asegura el carácter competitivo de suma constante que exigen algoritmos como Minimax. Esta función se calcula con la relación lógica `(goal r v)`.

3.4.9. Axiomas de admisibilidad y juegos bien formados

Para que un agente autónomo pueda resolver computacionalmente un juego formalizado como máquina de estados, se piden dos propiedades estructurales [8], [9]:

Definición 3.7: Juego bien formado

Un juego formal Γ se define como bien formado (*well-formed game*) [8]-[10] si satisface dos axiomas fundamentales:

1. **Jugabilidad (*Playability*):** Para todo estado no terminal, cada rol dispone de al menos

una acción legal ejecutable:

$$\forall s \in S \setminus F, \quad \forall r_i \in R : \quad L_i(s) \neq \emptyset$$

Esta condición previene bloqueos indeterminados (*deadlocks*) en la ejecución algorítmica del motor.

2. **Terminación garantizada (*Termination*):** Toda trayectoria legal inducida a partir del estado inicial s_0:

$$s_0 \xrightarrow{\mathbf{a}_0} s_1 \xrightarrow{\mathbf{a}_1} s_2 \ldots \xrightarrow{\mathbf{a}_{k-1}} s_k$$

debe culminar forzosamente en un estado terminal $s_k \in F$ en un número finito de transiciones ($k < \infty$).

En dominios donde podrían presentarse ciclos infinitos, la terminación se asegura con reglas de repetición o con cotas explícitas de turnos.

3.4.10. Distancia en el grafo de estados y estimación hacia configuraciones terminales

El modelo Γ induce un grafo dirigido: sus vértices son estados y sus aristas son transiciones producidas por acciones conjuntas legales desde estados no terminales. Una trayectoria expresa una sucesión posible de jugadas; por sí sola no representa una estrategia que garantice un resultado ante cualquier respuesta del adversario.

Definición 3.8: Distancia dirigida entre estados

Para $s_a, s_b \in S$, se define $d(s_a, s_b)$ como la longitud mínima de una trayectoria legal desde s_a hasta s_b [1], [36]:

$$d(s_a, s_b) = \min \left\{ k \in \mathbb{N}_0 \;\middle|\; \begin{array}{l} \exists \mathbf{a}_0, \ldots, \mathbf{a}_{k-1} : \\ s^{(0)} = s_a,\ s^{(k)} = s_b, \\ \forall\, 0 \leq j < k :\ s^{(j)} \notin F,\ \mathbf{a}_j \in \mathbf{L}(s^{(j)}), \\ s^{(j+1)} = T(s^{(j)}, \mathbf{a}_j) \end{array} \right\}. \tag{3.20}$$

Se adopta $\min \emptyset = \infty$. La trayectoria vacía admite $k = 0$, de modo que $d(s, s) = 0$. No es una métrica ordinaria: puede ser asimétrica y puede valer infinito. El superíndice $s^{(j)}$ indica el paso de la trayectoria, no una potencia.

Para un conjunto de destinos $B \subseteq S$, se entenderá $d(s, B) = \min_{b \in B} d(s, b)$, también con valor infinito si no existe destino alcanzable. En particular:

$$d(s, F) = \min_{s_f \in F} d(s, s_f). \tag{3.21}$$

Si el juego finito satisface el axioma de terminación para toda trayectoria legal alcanzable desde

s_0, existe un horizonte máximo finito $H_{\text{máx}}$ y:

$$\forall s \in \text{Reach}(s_0): \quad 0 \leq d(s, F) \leq H_{\text{máx}} < \infty. \tag{3.22}$$

Además, $d(s, F) = 0$ si y solo si $s \in F$. La cota depende del supuesto de terminación; la finitud del conjunto de estados no excluye por sí sola ciclos infinitos.

Con la convención de utilidad de los ejemplos y desde el rol r_i, sean $F_{\text{victoria}} = \{s \in F \mid G_i(s) = 100\}$ y $F_{\text{derrota}} = \{s \in F \mid G_i(s) = 0\}$. Las distancias son:

$$d(s, F_{\text{victoria}}) = \min_{s_f \in F_{\text{victoria}}} d(s, s_f), \tag{3.23}$$

$$d(s, F_{\text{derrota}}) = \min_{s_f \in F_{\text{derrota}}} d(s, s_f). \tag{3.24}$$

Puede existir una trayectoria corta hacia la victoria que requiera un error del oponente. Por ello, minimizar la primera distancia no equivale a asegurar una victoria. Minimax optimiza la utilidad que puede garantizar frente a respuestas adversariales, según la recurrencia por turnos de la Ecuación (3.32). Un desempate por duración requeriría definir su tratamiento sobre estrategias y no solo sobre el camino favorable más corto.

Cuando no se alcanza la frontera terminal se utiliza una heurística. La cercanía a una meta puede ser un rasgo de esa evaluación, pero no es por definición una probabilidad de ganar ni garantiza utilidad alta.

Una cota ilustrativa se obtiene para una meta conjuntiva $\mathcal{M} \subseteq \mathcal{F}$. Sea $F_{\mathcal{M}} = \{s_f \in F \mid \mathcal{M} \subseteq s_f\}$ y:

$$\text{Faltantes}(s, \mathcal{M}) = \{f \in \mathcal{M} \mid f \notin s\}. \tag{3.25}$$

Si en cualquier transición legal se pueden incorporar como máximo $c \geq 1$ hechos de esa meta que antes eran falsos, entonces:

$$d(s, F_{\mathcal{M}}) \geq \left\lceil \frac{|\,\text{Faltantes}(s, \mathcal{M})|}{c} \right\rceil. \tag{3.26}$$

La desigualdad cuenta transiciones conjuntas. En Gato, para una línea fija de X, cada transición agrega a lo sumo una marca de X ($c = 1$). Si falta una marca, se requiere al menos una transición; si es turno de O o este bloquea la línea, la cota puede ser poco informativa o el destino inalcanzable. No implica una victoria forzada ni se extiende sin justificar sus supuestos a cualquier condición terminal.

En planificación, la **relajación de efectos negativos** elimina las listas de borrado de los operadores: los hechos añadidos no se destruyen [36], [37]. Para tareas proposicionales STRIPS finitas y explícitas, comprobar alcanzabilidad en esa relajación es polinomial, pero obtener la longitud del plan relajado óptimo sigue siendo NP-hard. El óptimo relajado proporciona una cota inferior del costo original bajo esa relajación; aproximaciones eficientes como la extracción de planes de FF no garantizan en general admisibilidad [37]. Su traslado a reglas de juegos requiere justificar la representación, la relajación y el papel del adversario; no se considera una garantía general de GGP ni un algoritmo seleccionado para este proyecto.

3.4.11. Modelado síncrono de juegos por turnos alternados

Aunque $\mathbf{A} = A_1 \times A_2$ reúne las acciones de ambos participantes, los juegos típicamente secuenciales, como el Gato o el Ajedrez, encajan en este esquema mediante una acción pasiva o nula, noop (*no-operation*) [9], [10].

En un estado s donde le corresponde mover al jugador blanco, la función de legalidad asigna:

$$L_{\text{blancas}}(s) \subseteq A_{\text{blancas}} \setminus \{\text{noop}\} \tag{3.27}$$

$$L_{\text{negras}}(s) = \{\text{noop}\} \tag{3.28}$$

y cada acción conjunta admisible tiene la forma:

$$\mathbf{a} = (a_{\text{blanca}}, \text{noop}) \tag{3.29}$$

Al calcular la transición $s' = T(s, \mathbf{a})$ mediante la función de la Ecuación (3.17), las reglas del juego actualizan los fluents de control y si el sucesor no es terminal, se invierte la legalidad en la siguiente configuración:

$$L_{\text{blancas}}(s') = \{\text{noop}\} \tag{3.30}$$

$$L_{\text{negras}}(s') \subseteq A_{\text{negras}} \setminus \{\text{noop}\} \tag{3.31}$$

Así la máquina de estados conserva una arquitectura síncrona homogénea sin perder la alternancia secuencial propia del dominio.

3.4.12. Interfaz computacional para el motor de decisión

Para que esta formalización pueda usarse en la arquitectura de software del agente, el motor de juego debe ofrecer una interfaz programática con cinco consultas básicas: obtener el estado inicial (s_0); consultar las acciones legales de un jugador en un estado dado ($L_i(s)$); aplicar una acción conjunta y calcular el estado sucesor ($T(s, \mathbf{a})$); determinar si un estado cumple la condición de terminación ($s \in F$); y evaluar la utilidad que obtiene cada participante ($G_i(s)$).

Con esta interfaz, el motor de búsqueda queda completamente separado de las particularidades de cada juego. El algoritmo de decisión razona solo sobre estados abstractos, acciones legales y recompensas numéricas, sin conocimiento empírico propio del dominio.

3.5. Lenguaje de descripción de juegos (GDL)

Para que un agente pueda jugar sin que nadie le explique las reglas, estas tienen que estar escritas en un lenguaje que él mismo pueda leer, analizar y ejecutar. Eso es un lenguaje de descripción de juegos. En *General Game Playing*, el de referencia es el *Game Description Language* (GDL), desarrollado en la Universidad de Stanford [8], [10].

En un lenguaje imperativo como C++ o Python, la lógica de un juego se programa con estructuras de control, bucles y modificaciones destructivas de la memoria: se ordena qué hacer paso a paso. Un lenguaje de descripción de reglas va por otro lado, el declarativo, y en lugar de decir cómo se ejecuta el juego, dice qué es.

3.5.1. Características fundamentales de GDL

A partir de los fundamentos de Genesereth et al. [8], Love et al. [10] y Cropper et al. [38], pueden destacarse cuatro propiedades de GDL.

La primera es que se basa en lógica de primer orden y es declarativo. El autor del juego no programa cómo se ejecutan las jugadas: solo declara qué hechos son verdaderos y bajo qué condiciones cambian, y es el motor de inferencia del agente el que deduce las consecuencias de esas reglas. La segunda es su independencia del dominio. GDL no trae ideas previas sobre geometría de tableros, piezas de ajedrez o cartas; todo se representa con símbolos abstractos, y un agente general aprende la física y las restricciones del entorno mientras procesa el archivo de reglas, porque no las conoce de antemano.

La tercera es su semántica, basada en Datalog con negación estratificada. GDL es una variante estricta de Datalog que limita el uso de funciones recursivas complejas, lo que garantiza que el espacio de estados y la deducción lógica sean finitos y decidibles en cada paso de tiempo, y evita ciclos de inferencia infinitos. La cuarta es el desacoplamiento arquitectónico: la física del juego, descrita en el archivo de reglas, queda totalmente separada de los algoritmos de búsqueda y decisión, que viven en el motor del agente. Por eso el mismo núcleo puede jugar juegos muy distintos sin cambiar una línea de código fuente.

3.5.2. Concepto y estructura de un predicado lógico

Para entender la sintaxis de GDL sin caer en un formalismo matemático excesivo, conviene partir de la experiencia del desarrollo de software:

Definición 3.9: Predicado lógico

En la semántica lógica declarativa de GDL [9], [10], un **predicado** es una afirmación estructurada sobre uno o varios objetos del dominio que, al ser evaluada contra el estado actual del juego, devuelve estrictamente un valor de verdad: verdadero o falso.

Un programador que quiere saber si hay una cruz en el renglón 1 y la columna 1 escribiría algo como `tablero[1][1] == 'x'`: mirar una celda de una matriz. GDL no consulta nada, afirma un hecho. La misma pregunta se escribe

```
(cell 1 1 x)
```

y la respuesta es simplemente si ese hecho es verdadero o falso en el estado actual.

Un predicado se escribe como una lista entre paréntesis y tiene tres partes. La primera es el *functor*, el identificador inicial que dice qué propiedad o relación se expresa (por ejemplo, `cell`, `control`, `legal`, `next`). La segunda son los términos o argumentos, los elementos sobre los que recae la afirmación. Estos pueden ser constantes, símbolos atómicos que denotan objetos concretos del dominio, como coordenadas (`1`, `2`, `3`) o identificadores (`x`, `o`, y `b` para una casilla vacía o *blank*); o variables, que en GDL son cualquier símbolo que empieza con un signo de interrogación (`?r` para el renglón, `?c` para la columna, `?p` para el participante) y permiten escribir reglas generales válidas para cualquier casilla o jugador.

La tercera parte es la aridad, el número de argumentos que acepta el predicado. Con aridad 0 tenemos una proposición pura, como `(terminal)`, que no recibe argumentos y expresa un hecho

global: si la partida terminó. Con aridad 1, una propiedad de un objeto, por ejemplo `(role x)` o `(control x)`, que indican que `x` es un rol o que tiene el turno. Con aridad 3, una relación ternaria como `(cell 2 2 x)`, que une renglón, columna y ficha.

Construcción de reglas de inferencia (<=)

Los hechos no siempre están dados de forma estática; muchas veces hay que deducirlos de otros hechos. Para eso, GDL usa reglas de inferencia con el operador de implicación <=, que se lee "la cabeza es verdadera si todas las premisas del cuerpo son verdaderas":

```
(<= (cabeza_deducida)
    (premisa_o_condicion_1)
    (premisa_o_condicion_2)
    ...
    (premisa_o_condicion_k))
```

Código 3.1: Estructura canónica de una regla de inferencia en GDL.

El motor lógico solo da por cierta la cabeza si logra verificar que todas las condiciones del cuerpo se cumplen a la vez en el estado del juego.

3.5.3. Predicados reservados del sistema en GDL

Para que un archivo de reglas describa la tupla formal Γ de la Sección 3.4, GDL define un conjunto cerrado de ocho predicados del sistema [9], [10], que se resumen en la Tabla 3.1.

Tabla 3.1: Predicados reservados de GDL y componente de Γ que describen.

Predicado	Función
`(role ?r)`	Declara a los participantes del juego e instancia el conjunto R.
`(init ?f)`	Especifica los fluents atómicos que forman el estado inicial s_0.
`(true ?f)`	Consulta si un fluent dinámico f es verdadero en el estado actual s.
`(does ?r ?a)`	Representa la acción $a \in A_i$ que el rol r ejecuta en el turno corriente.
`(legal ?r ?a)`	Deduce si la acción a pertenece al conjunto de jugadas permitidas $L_i(s)$.
`(next ?f)`	Deduce qué fluents formarán parte del estado sucesor $s' = T(s, \mathbf{a})$.
`(terminal)`	Evalúa si la configuración actual pertenece al conjunto de estados terminales F.
`(goal ?r ?v)`	Determina la utilidad o recompensa numérica $G_i(s) = v \in [0, 100]$ del jugador r.

3.5.4. Ejemplificación mediante código de especificación: El juego del Gato

Para ver cómo trabajan juntos estos predicados, tomemos la especificación del juego del Gato (*Tic-Tac-Toe*), que se puede dividir en cuatro bloques lógicos.

Roles y estado inicial

El primer bloque (Código 3.2) declara los dos jugadores y describe el tablero al inicio ($t = 0$): las nueve casillas vacías (`b`) y el turno en manos de `x`.

```
;; ========================================
;; 1. Declaracion de roles (Jugadores)
;; ========================================
(role x)
(role o)

;; ========================================
;; 2. Estado inicial (t = 0):
;;    Nueve casillas vacias (b = blank)
;;    y control asignado a x
;; ========================================
(init (cell 1 1 b))
(init (cell 1 2 b))
(init (cell 1 3 b))
(init (cell 2 1 b))
(init (cell 2 2 b))
(init (cell 2 3 b))
(init (cell 3 1 b))
(init (cell 3 2 b))
(init (cell 3 3 b))
(init (control x))
```

Código 3.2: Declaración de roles y estado inicial para el juego del Gato.

Reglas de legalidad y alternancia de turnos

El segundo bloque (Código 3.3) formaliza la función de legalidad $L_i(s)$. Una jugada es legal si quien la intenta tiene el control y la casilla elegida está libre. La sincronía se modela con la jugada nula noop:

```
;; ========================================
;; 3. Acciones legales
;; ========================================
;; Un jugador puede marcar (?r, ?c) si es su turno
;; y la casilla actualmente contiene 'b'
(<= (legal ?p (mark ?r ?c))
    (true (control ?p))
    (true (cell ?r ?c b)))

;; Si NO es el turno del jugador, su unica accion legal es 'noop'
(<= (legal ?p noop)
    (role ?p)
    (not (true (control ?p))))
```

Código 3.3: Reglas de legalidad de movimientos en GDL.

Con estas reglas el motor rechaza por sí solo cualquier intento de poner una ficha en una casilla ocupada, y al rival que no tiene el turno le deja únicamente la acción nula.

Transición de estado y persistencia inercial

El tercer bloque (Código 3.4) gobierna la función de transición $T(s, \mathbf{a})$. En lógica declarativa, un hecho no se conserva en el estado siguiente a menos que una regla lo diga de forma explícita, así que hay que definir tanto los cambios directos como la inercia del sistema:

```
;; ========================================
;; 4. Funcion de transicion (Siguiente estado)
;; ========================================
;; A. Efecto directo: la casilla marcada pasa a tener la ficha del jugador
(<= (next (cell ?r ?c ?p))
    (does ?p (mark ?r ?c)))

;; B. Inercia de fichas: las celdas ya marcadas conservan su contenido
(<= (next (cell ?r ?c ?mark))
    (true (cell ?r ?c ?mark))
    (distinct ?mark b))

;; C. Inercia de celdas vacias: una casilla sigue vacia si nadie la marco
(<= (next (cell ?r ?c b))
    (true (cell ?r ?c b))
    (not (does x (mark ?r ?c)))
    (not (does o (mark ?r ?c))))

;; D. Alternancia del turno
(<= (next (control o)) (true (control x)))
(<= (next (control x)) (true (control o)))
```

Código 3.4: Reglas de transición dinámica e inercia en GDL.

Condiciones terminales y asignación de recompensas

El cuarto bloque (Código 3.5) define el conjunto de estados terminales F y la función de utilidad $G_i(s)$: premia la formación de líneas de tres fichas y reconoce el empate cuando el tablero se llena:

Los cuatro bloques incluyen los auxiliares `line`, `open` y `board_full`. Las líneas se restringen a los roles para que las casillas vacías no se interpreten como una victoria. La partida se detendrá al deducir `terminal`; las consultas de legalidad y transición se utilizarán únicamente mientras el estado actual no sea terminal. Es una especificación didáctica de Gato en GDL, no una prueba de que el intérprete del proyecto ya esté implementado ni una elección del lenguaje definitivo.

3.6. GDL, Ludii y lenguajes de dominio específico

Describir un juego con GDL clásico es una opción entre varias. Otros lenguajes intentan que la escritura sea más cómoda sin perder expresividad ni volverse difíciles de implementar. Ludii es un caso claro: en vez de predicados lógicos habla de tableros, piezas y movimientos, más cerca de cómo piensa quien diseña juegos [28].

```
;; ========================================
;; 5. Lineas, tablero lleno y utilidades
;; ========================================
;; Lineas completas: filas, columnas y las dos diagonales
(<= (line ?p)
    (role ?p)
    (true (cell ?r 1 ?p))
    (true (cell ?r 2 ?p))
    (true (cell ?r 3 ?p)))
(<= (line ?p)
    (role ?p)
    (true (cell 1 ?c ?p))
    (true (cell 2 ?c ?p))
    (true (cell 3 ?c ?p)))
(<= (line ?p)
    (role ?p)
    (true (cell 1 1 ?p))
    (true (cell 2 2 ?p))
    (true (cell 3 3 ?p)))
(<= (line ?p)
    (role ?p)
    (true (cell 1 3 ?p))
    (true (cell 2 2 ?p))
    (true (cell 3 1 ?p)))

;; La negacion se aplica a un predicado sin variables libres
(<= open (true (cell ?r ?c b)))
(<= board_full (not open))

;; Victoria absoluta (100 puntos) si el jugador forma una linea
(<= (goal ?p 100) (line ?p))

;; Derrota (0 puntos) si el adversario completo una linea
(<= (goal ?p 0)   (role ?p) (line ?other) (distinct ?p ?other))

;; Empate (50 puntos) si el tablero esta lleno y nadie gano
(<= (goal ?p 50)  (role ?p) (not (line x)) (not (line o)) board_full)

;; ========================================
;; 6. Condicion de fin de partida (terminal)
;; ========================================
(<= terminal (line x))
(<= terminal (line o))
(<= terminal board_full)
```

Código 3.5: Reglas de utilidad (goal) y condición de terminación en GDL.

Comparar GDL con Ludii importa para este Trabajo Terminal porque obliga a tomar una decisión de arquitectura: implementar GDL completo, adoptar solo un subconjunto o diseñar un lenguaje de dominio específico (DSL) inspirado en ambos. GDL tiene una base académica sólida y una semántica lógica general, pero puede resultar pesado para un prototipo de alcance acotado. Ludii, por su parte, demuestra que un lenguaje puede seguir siendo general y ser más legible para quienes diseñan y desarrollan juegos.

Un DSL propio permitiría concentrarse en lo que el proyecto necesita: jugadores, estados, acciones, reglas de legalidad, transiciones, condiciones terminales y recompensas. Esta opción tendrá que justificarse formalmente, para que el lenguaje no termine siendo una colección de casos especiales. El criterio de diseño es que permita describir los juegos dentro del alcance definido: discretos, deterministas, por turnos y de información perfecta.

3.7. Intérprete y representación interna

Tener el archivo de reglas no basta para jugar. Hace falta un intérprete que lea la descripción, valide su estructura y construya una representación interna que el motor del agente pueda consultar. El proceso suele pasar por análisis léxico, análisis sintáctico, validación semántica y construcción de un modelo interno.

El análisis léxico separa el texto en unidades reconocibles y el sintáctico comprueba que esas unidades respeten la gramática del lenguaje. La validación semántica revisa que las reglas tengan sentido dentro del modelo de juego; por ejemplo, que las acciones hagan referencia a jugadores que existen o que una condición terminal pueda evaluarse. Al final, la representación interna organiza toda esa información para responder consultas durante la partida.

La representación interna hace de puente entre el lenguaje textual y los algoritmos de decisión. Una vez construido el modelo, el motor ya no debería trabajar con cadenas de texto, sino con estructuras que permitan evaluar legalidad, transiciones y recompensas. Esto ayuda al mantenimiento: en principio se podría cambiar el lenguaje textual sin rehacer el motor de búsqueda, siempre que se conserve la interfaz conceptual del juego.

En las implementaciones avanzadas de GGP, las reglas también pueden compilarse en estructuras más eficientes. Goldwaser y Thielscher señalan que una descripción GDL puede convertirse en una red proposicional (PropNet), con nodos lógicos, de entrada, de salida y de transición [31]. Esto interesa porque interpretar las reglas desde cero en cada nodo de búsqueda resulta caro. Una PropNet completa quizá quede fuera de una primera versión, pero la idea sirve para entender por qué la eficiencia de la representación interna pesa tanto como el algoritmo de búsqueda.

3.8. Árboles de juego y explosión combinatoria

Con acciones legales y transiciones ya se puede armar un árbol de juego [1]: la raíz es el estado actual, de ella salen tantas ramas como acciones legales haya, y cada rama termina en el estado que produce esa acción. Si el proceso se repite, el agente ve hacia adelante y puede comparar a dónde lleva cada decisión.

El problema es que el número de nodos crece muy rápido. Si un estado tiene un factor de ramificación promedio de b acciones y se explora hasta la profundidad d, el número de nodos puede crecer del orden de b^d. En juegos pequeños es manejable; en juegos complejos, explorarlo todo no es práctico.

Por la explosión combinatoria hay que limitar la búsqueda: el agente debe decidir cuánto explorar, qué ramas priorizar y cómo evaluar los estados que no son terminales. Aquí entran Minimax, la poda alfa-beta y Monte Carlo Tree Search, junto con funciones heurísticas que estiman qué tan bueno es un estado sin llegar hasta el final de la partida.

3.9. Minimax

Minimax es un algoritmo clásico para juegos adversariales de dos jugadores. Se apoya en el teorema minimax de von Neumann [39] y fue formulado para decidir en árboles de juego por Shannon [12] y, más tarde, por Russell y Norvig [1]. La idea básica es suponer que un jugador trata de maximizar su utilidad mientras que el oponente trata de minimizarla, de modo que en el árbol los niveles se alternan entre decisiones de MAX y de MIN.

Definición 3.10: Función de valor Minimax y decisión óptima

En un juego formal de dos participantes ($R = \{r_1, r_2\}$) determinista, de información perfecta y suma constante [1], [12], [39], el **valor Minimax** $V(s)$ de un estado $s \in S$ desde la perspectiva de MAX (r_1) se define recursivamente como:

$$V(s) = \begin{cases} G_1(s) & \text{si } s \in F \\ \max_{a \in L_1(s)} V(T(s, (a, \text{noop}))) & \text{si el turno corresponde a } r_1 \\ \min_{a \in L_2(s)} V(T(s, (\text{noop}, a))) & \text{si el turno corresponde a } r_2 \end{cases} \tag{3.32}$$

La **decisión óptima** $a^* \in L_i(s)$ para el participante activo corresponde a la acción que optimiza (maximiza para r_1, minimiza para r_2) el valor del estado sucesor inducido.

Si el árbol puede explorarse por completo hasta los estados terminales F, Minimax garantiza elegir una acción óptima siempre que ambos jugadores actúen racionalmente. El valor de un estado terminal sale directo de la función de utilidad $G_1(s)$, y el de cualquier estado no terminal se calcula propagando hacia arriba los valores de sus sucesores: MAX toma el mayor y MIN el menor.

En la práctica, casi todos los juegos tienen factores de ramificación demasiado grandes para llegar a la frontera terminal. Por eso se fija una profundidad máxima de búsqueda y se evalúan los estados de la frontera con una función heurística. Así Minimax se puede usar en juegos más grandes, pero aparece la pregunta central para *General Game Playing*: cómo evaluar estados sin usar conocimiento del dominio programado de antemano.

3.10. Poda alfa-beta

La poda alfa-beta es una optimización básica de Minimax, analizada con detalle por Knuth y Moore [40] y descrita en la literatura clásica de inteligencia artificial [1]. Bajo las mismas condiciones de evaluación no cambia la decisión final, pero evita explorar las ramas que no pueden modificar el resultado. Para ello mantiene dos valores: alfa, la mejor alternativa que ha encontrado MAX, y beta, la mejor que ha encontrado MIN.

Cuando detecta que una rama no puede dar una decisión mejor que otra ya conocida, deja de explorarla. Así visita menos nodos y puede llegar a mayor profundidad con el mismo presupuesto de cómputo. Su efectividad depende del orden en que se exploren las acciones: si las mejores se revisan temprano, se descartan más ramas.

En este Trabajo Terminal, alfa-beta interesa porque permite comparar dos cosas: la decisión que produce Minimax y la eficiencia que se gana al podar. En la evaluación experimental, métricas como los nodos explorados, el tiempo de decisión y la profundidad alcanzada pueden mostrar si la optimización mejora el rendimiento sin afectar la legalidad ni la coherencia de las jugadas. La Figura 3.2 muestra cómo funciona el corte: en cuanto MIN encuentra una alternativa con valor menor o igual a la cota α que MAX ya tiene asegurada, se descartan de inmediato los demás sucesores de ese subárbol.

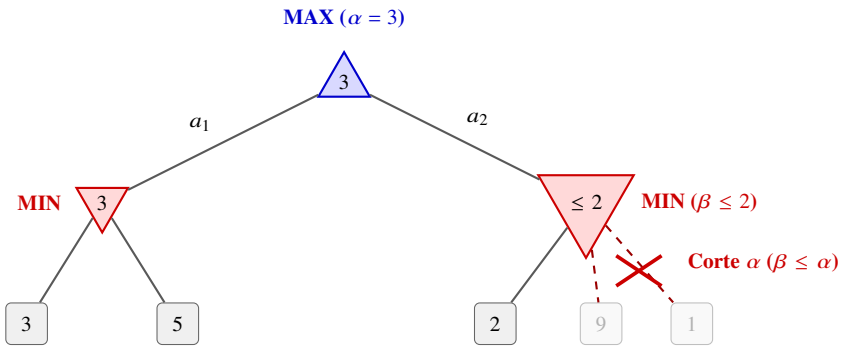


Figura 3.2: Árbol de juego adversarial con poda alfa-beta. Tras evaluarse el nodo con valor 2, el nodo MIN asegura un valor a lo sumo de 2, el cual es estrictamente inferior al valor $\alpha = 3$ ya garantizado por MAX en la rama izquierda; en consecuencia, se interrumpe la exploración del resto de los sucesores.

3.11. Funciones heurísticas generales

Una función heurística estima qué tan favorable es un estado cuando no se puede explorar el árbol hasta el final. En los motores especializados suele llevar conocimiento del dominio; en Ajedrez, por ejemplo, se puede dar un valor a cada pieza o medir el control del centro. Pero esas reglas no son generales: si el agente sabe cuánto vale una reina, ya trae conocimiento específico de Ajedrez.

En GGP el problema es evaluar sin saber de qué juego se trata. Se puede recurrir a rasgos estructurales: cuántas acciones legales hay (la movilidad), qué tan cerca se está de un estado terminal, cómo cambian las recompensas o qué resultados dan las simulaciones. Ninguno de estos

rasgos funciona bien en todos los juegos; lo valioso es que ninguno presupone un dominio y, combinados, pueden aproximar una evaluación en un agente que no conoce el juego.

Aquí está uno de los puntos más delicados del proyecto. Limitar la búsqueda obliga a decidir qué estados parecen mejores, pero ser generales impide escribir esa evaluación para un juego concreto. De ahí que las heurísticas generales y los métodos basados en simulación ocupen un lugar central en la fundamentación teórica del motor de decisión.

3.12. Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) es una familia de métodos de búsqueda que estima el valor de las acciones mediante simulaciones. En vez de depender solo de una función heurística explícita, va construyendo poco a poco un árbol de búsqueda y usa los resultados de partidas simuladas para orientar la exploración [23], [41].

MCTS funciona como un ciclo iterativo y asimétrico de cuatro fases que se repite mientras haya tiempo de cómputo, como se ve en la Figura 3.3. En la *selección*, el algoritmo baja por el árbol acumulado desde la raíz eligiendo los sucesores más prometedores con una política de balance. En la *expansión* se agregan uno o varios nodos hijos que corresponden a acciones legales aún no exploradas. En la *simulación*, o *rollout*, se juega una partida rápida con movimientos muestreados al azar hasta llegar a una configuración terminal. Por último, en la *retropropagación*, la recompensa terminal se propaga hacia la raíz y actualiza las estadísticas de visitas y el valor medio de cada estado del camino recorrido.

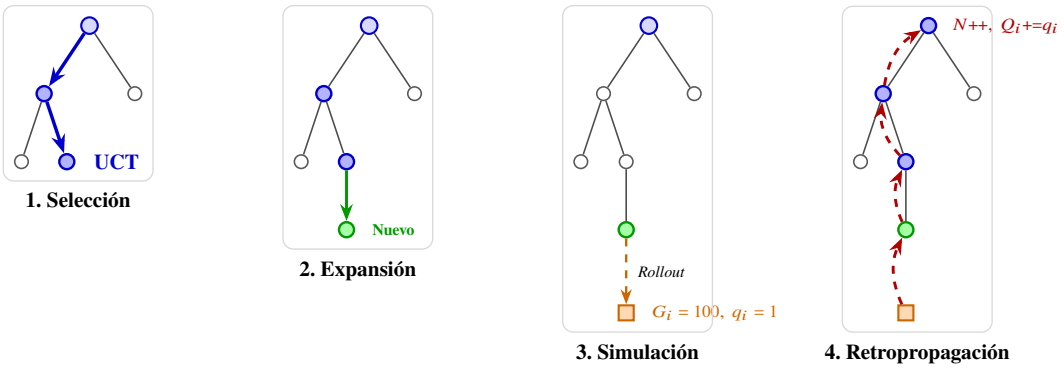


Figura 3.3: Ciclo iterativo de cuatro fases en Monte Carlo Tree Search (MCTS): Selección de nodos mediante la cota UCT, Expansión de nuevas acciones legales, Simulación estocástica (*rollout*) hasta un estado terminal y Retropropagación de recompensas hacia la raíz.

Lo que hace tentador a MCTS para GGP es que no exige heurísticas propias de un juego. Basta con un modelo capaz de dar las acciones legales, aplicar una transición y decir cuándo se terminó la partida; con eso se juegan partidas al azar y los resultados indican qué acciones convienen. El costo es que todo depende de qué tan rápido y bien esté hecho ese modelo, y de cuántas simulaciones alcancen en el tiempo disponible.

3.13. UCT

Una pregunta central en la fase de selección de MCTS es qué rama explorar. Si el agente explora solo la acción que hasta ese momento parece la mejor, corre el riesgo de ignorar alternativas poco visitadas que en realidad serían superiores. Si explora sin criterio, puede perder tiempo de cómputo en acciones de baja calidad. El algoritmo UCT (*Upper Confidence Bounds applied to Trees*) trata este dilema entre explotación y exploración de manera formal [23], [42].

Definición 3.11: Criterio de selección UCT

Para ilustrar UCT en los juegos por turnos del alcance, sea $p(v)$ el rol activo en el nodo v. Por cada simulación que termina en $s_f \in F$, se normalizará el resultado de cada rol como $q_i = G_i(s_f)/100 \in [0, 1]$. En cada nodo visitado se acumulará ese resultado en Q_i y se incrementará su contador N [23], [42].
Si existe un hijo sin visitas, se seleccionará uno de esos hijos antes de evaluar la expresión siguiente; para ilustrar el caso de empate se puede usar cualquier orden fijo. Cuando todos tienen $N(v') > 0$, y por tanto $N(v) > 0$, se aplicará:

$$v^* \in \arg \max_{v' \in \text{hijos}(v)} \left[\frac{Q_{p(v)}(v')}{N(v')} + C\sqrt{\frac{\ln N(v)}{N(v')}} \right], \quad C > 0. \tag{3.33}$$

Aquí $Q_i(v')$ suma las recompensas normalizadas del rol r_i en las simulaciones que visitaron v'. La media se toma desde la perspectiva del jugador activo en el padre, no siempre desde la raíz. Así ambos participantes buscan su propia utilidad. El parámetro C regula la exploración y no se fija como decisión de implementación.

El primer sumando estima la utilidad media y el segundo favorece acciones poco visitadas. Con esta normalización, la elección $C = \sqrt{2}$ reproduce la forma usual de la cota UCB1 [43]; no se traslada sin ajuste a acumulados en la escala $[0, 100]$. No se evalúan cocientes con cero visitas. En estados terminales no hay selección de sucesores; su utilidad se retropropaga.

UCT es un antecedente para la búsqueda por simulación. Sus resultados de consistencia dependen de las hipótesis del análisis, como el horizonte finito o descuento en el artículo original [42]; no aseguran una decisión óptima con un presupuesto finito ni determinan el algoritmo que se implementará.

3.14. Aprendizaje por refuerzo y aprendizaje profundo

El aprendizaje por refuerzo parte de una escena sencilla: un agente en un entorno mira el estado, actúa, recibe una recompensa y cae en otro estado. No hay un maestro que diga en cada paso qué era lo correcto, como ocurre en el aprendizaje supervisado; en un juego, la señal de que algo salió bien o mal puede llegar solo al terminar la partida, y el agente tiene que averiguar qué jugadas la provocaron.

En los juegos, el resultado final (ganar, perder o empatar) puede servir para ajustar la política de decisión del agente. Al combinar esto con redes neuronales profundas se habla de aprendizaje por refuerzo profundo, capaz de aproximar funciones de valor o de política muy complejas,

aunque a cambio pide muchos datos, un entrenamiento intensivo y una buena forma de representar el estado.

Con aprendizaje profundo el problema se complica en GGP. No basta con que el agente aprenda a jugar bien: tiene que hacerlo con juegos que le llegan descritos desde fuera, y eso obliga a que la representación del estado y de las acciones sirva para cualquiera de ellos. Por esa razón lo tratamos aquí como antecedente y como línea de investigación avanzada, y no como algo que este proyecto vaya a implementar.

3.15. AlphaZero como antecedente moderno

AlphaZero es un antecedente importante porque combinó autojuego, aprendizaje por refuerzo profundo y búsqueda Monte Carlo para alcanzar un alto desempeño en Ajedrez, Shogi y Go [7]. Su arquitectura usa una red neuronal que estima una política de acciones y un valor de estado, mientras que MCTS afina la elección de movimientos con búsqueda.

Sin embargo, AlphaZero no resuelve por sí mismo el problema central de este Trabajo Terminal. Aunque usa un algoritmo general para más de un juego, necesita simuladores y representaciones adecuadas para cada dominio. En un GGP el reto es otro: interpretar reglas externas, construir el modelo del juego y lograr que el motor de decisión funcione sin conocimiento específico codificado para cada caso.

Goldwaser y Thielscher llevan ideas de aprendizaje por refuerzo profundo al contexto de GGP y muestran que existe una línea de investigación que intenta unir descripciones formales de juegos con métodos inspirados en AlphaZero [31]. Para el alcance de este trabajo, esas ideas son una referencia del estado del arte y ayudan a acotar el proyecto: se busca una arquitectura general e interpretable, no replicar un sistema de entrenamiento profundo a gran escala.

3.16. Síntesis del marco teórico

Desarrollar un General Game Player exige integrar varias áreas de la inteligencia artificial y de las ciencias computacionales. Primero, una representación formal del juego que separe estados, acciones, transiciones, terminalidad y recompensas. Segundo, un lenguaje de descripción o DSL que exprese las reglas sin codificarlas directamente en el motor. Tercero, un intérprete que convierta esa descripción en una representación interna consultable. Y por último, un agente que elija acciones con algoritmos generales de búsqueda o simulación.

Esa integración se organiza en un flujo desacoplado, que muestra la Figura 3.4. El analizador léxico y sintáctico procesa el archivo de reglas externo y crea en memoria el modelo formal Γ. Sobre este modelo, el generador de estados resuelve consultas abstractas de legalidad, transiciones y condiciones terminales, y con eso el motor de decisión (Minimax, poda alfa-beta o MCTS) elige la mejor acción disponible $\mathbf{a}^*$ sin guardar conocimiento particular del dominio.

Lo que aporta este trabajo no es el mejor jugador de un juego en particular, sino una arquitectura desacoplada: el motor debe seguir igual cuando cambia la descripción del juego. Por eso el Gato sirve como caso funcional completo para validar el flujo de reglas, estados y terminalidad, y el Ajedrez como caso de mayor complejidad, cuyo diseño experimental (posiciones, oponente y métricas) está pendiente de definirse, como se detalla en la Sección 1.6. Con esta delimitación se

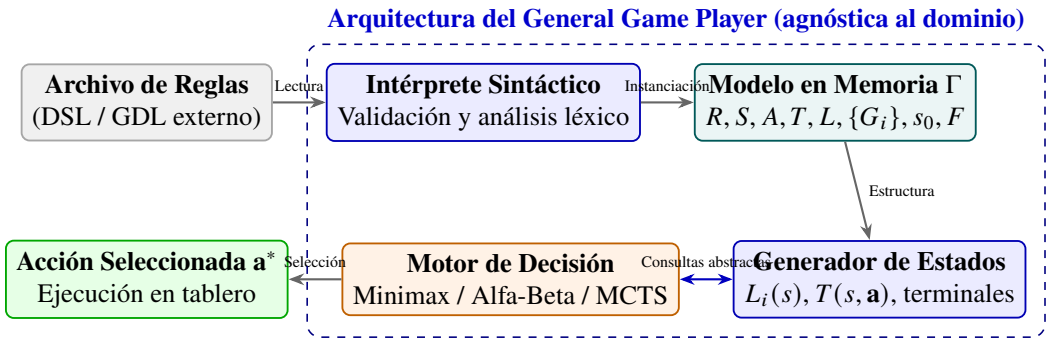


Figura 3.4: Flujo conceptual de ejecución y arquitectura desacoplada del General Game Player. El flujo detallado se presenta en la Figura 5.1.

puede estudiar la generalidad operativa sin confundir el prototipo con un motor especializado de competencia.

Capítulo 4

Análisis

El análisis de este Trabajo Terminal empieza por el lenguaje de descripción de juegos, porque es la pieza de la que dependen todas las demás. El intérprete, el generador de sucesores y el motor de decisión que se proponen en la Sección 5.1 trabajan sobre el modelo formal Γ de la Sección 3.4, pero ese modelo no llega al agente como un objeto matemático: llega como un archivo de texto. Este capítulo formaliza la relación entre ese archivo y el juego: la definición de lenguaje de descripción de juegos, los conjuntos en que viven sus descripciones, sus palabras reservadas, su semántica, las condiciones de validez de un archivo y el juego que este denota.

El capítulo procede de lo general a lo particular. Primero se aclara qué objeto se pretende describir y por qué no basta con escribirlo de forma explícita (Sección 4.1). Después se propone una definición de lenguaje de descripción de juegos como tupla, independiente de cualquier lenguaje concreto (Sección 4.2). Con ese marco se analiza GDL capa por capa: su nivel léxico, su vocabulario y sus palabras reservadas, su gramática (Sección 4.3), las restricciones que separan una descripción válida de una que solo es sintácticamente correcta (Sección 4.4) y la semántica que asocia a cada descripción válida un juego Γ (Sección 4.5). Las secciones siguientes desarrollan ejemplos completos, un catálogo de descripciones defectuosas, la expresividad y el costo computacional del lenguaje, la comparación con otros lenguajes de la literatura y, por último, los requisitos que de todo ello se desprenden para el intérprete.

4.1. Representación formal de juegos y niveles de descripción

4.1.1. Delimitación del objeto descrito

Un lenguaje de descripción de juegos no describe partidas, ni estrategias, ni la manera de dibujar un tablero en pantalla. Describe la *física* del juego: quiénes participan, cómo empieza, qué se puede hacer en cada situación, qué ocurre cuando se hace, cuándo termina y cuánto obtiene cada participante al terminar. En el vocabulario de la Definición 3.4, lo que se describe es la tupla

$$\Gamma = \langle R, S, s_0, \{A_i\}_{i=1}^{n}, \{L_i\}_{i=1}^{n}, T, F, \{G_i\}_{i=1}^{n} \rangle. \tag{4.1}$$

Cada componente conserva el significado que se le dio en la Sección 3.4: $R = \{r_1, \ldots, r_n\}$ es el conjunto finito de roles y $n = |R|$ su número; S es el espacio de estados; $s_0 \in S$ es el estado inicial; A_i es el catálogo de acciones del rol r_i; $L_i : S \to \mathcal{P}(A_i)$ es su función de legalidad

(Ecuación (3.15)); $T : S \times \mathbf{A} \to S$ es la función de transición (Ecuación (3.17)); $F \subseteq S$ es el conjunto de estados terminales; y G_i es la función de utilidad del rol r_i (Ecuación (3.18)). La notación $\{A_i\}_{i=1}^{n}$ designa una *familia indexada*: una componente $A_1, A_2, \ldots, A_n$ por cada rol, y lo mismo vale para $\{L_i\}_{i=1}^{n}$ y $\{G_i\}_{i=1}^{n}$. Describir un juego significa, entonces, especificar sin ambigüedad las ocho componentes de esta tupla.

Todo lo demás queda fuera de la descripción por diseño. El conocimiento estratégico (qué jugada conviene, qué posición es buena) pertenece al agente, no a las reglas; si una descripción incluyera esa información, el agente dejaría de ser general en el sentido de la Sección 3.4. Tampoco forman parte del lenguaje los relojes de juego ni el protocolo de comunicación entre el administrador de la partida y los jugadores, que en la competencia de GGP se manejan por separado [8]; ni la presentación gráfica, que en este proyecto corresponde a la interfaz descrita en la Sección 5.2.

Conviene fijar desde ahora una distinción que recorre todo el capítulo. Un *juego* es un objeto matemático abstracto: un conjunto de estados, unas funciones, un conjunto de terminales. Una *descripción* es un objeto sintáctico finito: una secuencia de caracteres en un archivo. La relación entre ambos es la misma que hay entre un número y los numerales que lo escriben: “6”, “VI” y “110_2” son tres numerales distintos para un único número. De igual manera, habrá muchas descripciones distintas para un mismo juego, y una parte del análisis consiste en entender cuándo dos descripciones denotan el mismo juego.

4.1.2. Representación extensional y representación factorizada

La manera más ingenua de describir Γ sería enumerarlo: listar uno por uno sus estados, escribir para cada estado sus acciones legales y para cada par estado–acción su sucesor. Esta forma de descripción se denomina representación explícita.

Definición 4.1: Representación explícita de un juego

Una **representación explícita** de un juego finito Γ es una codificación que enumera extensionalmente cada componente de la tupla (4.1): la lista de estados de S, la tabla de $L_i(s)$ para cada $s \in S$ y cada rol, la tabla de $T(s, \mathbf{a})$ para cada estado no terminal y cada acción conjunta legal, el conjunto F y la tabla de $G_i(s)$ para cada $s \in F$. Su tamaño, medido en número de entradas, es como mínimo

$$\mathrm{tam}_{\mathrm{exp}}(\Gamma) \;=\; |S| \;+\; \sum_{s \in S \setminus F} |\mathbf{L}(s)| \;+\; n\,|F|. \tag{4.2}$$

La Ecuación (4.2) cuenta cuántos datos hay que escribir, como mínimo, para enumerar el juego completo, y cada uno de sus tres sumandos corresponde a una parte de la tabla. La notación $|X|$ denota la cardinalidad del conjunto X, es decir, su número de elementos.

El primer sumando, $|S|$, es el número de estados: la lista de estados necesita una entrada por cada uno.

El segundo sumando cuenta las transiciones. Para un estado s, el conjunto $\mathbf{L}(s) = \prod_{i=1}^{n} L_i(s)$ es el espacio de acciones conjuntas legales de la Ecuación (3.16): contiene todos los vectores $\mathbf{a} = (a_1, \ldots, a_n)$ en los que cada rol elige una acción legal. Cada uno de esos vectores produce

exactamente una entrada $T(s, \mathbf{a})$ en la tabla de transiciones, de modo que el estado s aporta $|\mathbf{L}(s)|$ entradas. La suma recorre $S \setminus F$, la diferencia de conjuntos formada por los estados de S que no pertenecen a F, porque en un estado terminal la partida se detiene y no se definen sucesores (Sección 3.4). Cada una de estas entradas es, además, una arista del grafo dirigido que Γ induce sobre sus estados, introducido en la Definición 3.8; por eso el segundo sumando es igual al número de aristas del grafo de juego.

El tercer sumando, $n\,|F|$, cuenta las utilidades: en cada estado terminal $s \in F$ hay que anotar los n valores $G_1(s), \ldots, G_n(s)$, uno por rol.

La tabla de legalidad no aparece como sumando independiente porque su tamaño está acotado por el de la tabla de transiciones. En un estado no terminal de un juego bien formado, cada $L_i(s)$ es no vacío (Definición 3.7), y cada acción legal de un rol aparece en al menos una acción conjunta; por lo tanto, $\sum_{i=1}^{n} |L_i(s)| \leq n \cdot |\mathbf{L}(s)|$, y la tabla de legalidad agrega a lo sumo un factor n al segundo sumando. Por esa razón la Ecuación (4.2) es una cota inferior: cualquier representación explícita debe tener al menos ese número de entradas, y una representación concreta puede necesitar algunas más.

La representación explícita es perfectamente rigurosa, pero inviable en cuanto el juego deja de ser diminuto. En el Gato, las reglas del Capítulo 3 generan $|S| = 5\,478$ estados alcanzables, de los cuales $|F| = 958$ son terminales, y 16 167 transiciones (Sección 4.7); con $n = 2$, la Ecuación (4.2) da $5\,478 + 16\,167 + 2 \cdot 958 = 23\,561$ entradas para un juego elemental. En el Ajedrez, Shannon estimó del orden de 10^{43} posiciones y del orden de 10^{120} partidas posibles [12]. Ningún archivo puede enumerar eso. Estas cifras se demuestran, en el caso del Gato, y se justifican, en el del Ajedrez, en la Sección 4.1.3, donde además se distingue con precisión entre el número de estados y el número de partidas.

La alternativa es una representación factorizada, que es precisamente la idea de *fluent* de la Definición 3.5: en lugar de nombrar cada estado como un todo indivisible, se describe cada estado como un conjunto de hechos atómicos, y en lugar de tabular las transiciones, se escriben reglas generales que dicen cómo cambia cada hecho.

Definición 4.2: Representación factorizada o declarativa

Una **representación factorizada** de un juego Γ es una codificación en la que (i) los estados son subconjuntos de un universo de proposiciones atómicas $\mathcal{F}$, de modo que $S \subseteq \mathcal{P}(\mathcal{F})$; y (ii) las componentes L_i, T, F y G_i no se enumeran, sino que se definen *intensionalmente*, mediante reglas cuyas condiciones se evalúan sobre los hechos verdaderos de cada estado.

En la condición (i), $\mathcal{F}$ es el universo de fluents de la Definición 3.5 y $\mathcal{P}(\mathcal{F})$ es su conjunto potencia, el conjunto de todos sus subconjuntos. La inclusión $S \subseteq \mathcal{P}(\mathcal{F})$ repite lo establecido en la Definición 3.6: cada estado es el conjunto de fluents verdaderos en un instante, y por la hipótesis de mundo cerrado todo fluent ausente es falso. En la condición (ii), "intensional" se opone a "extensional": en lugar de listar los pares que forman una función, se da una regla que permite calcular su valor en cualquier argumento.

La ganancia de la factorización puede ser enorme. Si el universo tiene $|\mathcal{F}| = m$ fluents, cada estado queda determinado por la decisión, para cada uno de los m fluents, de si pertenece o no al

estado. Son m decisiones binarias independientes, y por el principio multiplicativo hay

$$|\mathcal{P}(\mathcal{F})| = \underbrace{2 \cdot 2 \cdots 2}_{m \text{ veces}} = 2^m \tag{4.3}$$

estados representables. Basta, pues, nombrar m fluents para poder hablar de 2^m configuraciones: una cantidad lineal de símbolos describe una cantidad exponencial de estados. En el Gato, con $m = 29$, son $2^{29} = 536\,870\,912$ configuraciones representables, aunque solo 5 478 de ellas sean estados alcanzables. El precio es que la información ya no está a la vista: para saber si $\mathrm{mark}(2, 2)$ es legal en un estado hay que *deducirlo* de las reglas. Un lenguaje de descripción de juegos es, en esencia, un lenguaje para escribir representaciones factorizadas junto con un mecanismo preciso para hacer esas deducciones.

Observación

En la Sección 4.9 se mostrará que para el Gato la diferencia es de 31 sentencias en la descripción factorizada de los Códigos 3.2 a 3.5, frente a 39 731 sentencias en una descripción explícita escrita en el mismo lenguaje. Ambas describen el mismo juego.

4.1.3. Espacio de estados y espacio de partidas

Al hablar del "tamaño" de un juego se confunden con frecuencia dos magnitudes distintas. La primera es el número de *estados*: cuántas configuraciones diferentes pueden presentarse. La segunda es el número de *partidas*: cuántas formas diferentes hay de jugar el juego de principio a fin. Un mismo estado puede aparecer en muchas partidas, porque varias secuencias de jugadas pueden conducir a él, y por eso la segunda magnitud suele ser mucho mayor que la primera. Esta subsección define ambas con precisión, las calcula de manera exacta para el Gato y presenta la estimación clásica para el Ajedrez.

Definición de partida

La noción de partida formaliza la idea de trayectoria legal que la Sección 3.4 usó al definir la distancia entre estados (Definición 3.8), con dos exigencias adicionales: que empiece en el estado inicial y que termine en un estado terminal.

Definición 4.3: Partida

Sea Γ un juego. Una **partida** de Γ es una secuencia finita de estados

$$\mathbf{w} = \left(s^{(0)}, s^{(1)}, \ldots, s^{(k)}\right), \qquad k \geq 0, \tag{4.4}$$

que satisface las cuatro condiciones siguientes:

1. **Inicio:** $s^{(0)} = s_0$.
2. **Fin:** $s^{(k)} \in F$.
3. **Continuidad:** $s^{(j)} \notin F$ para todo $0 \leq j < k$.

4. **Legalidad:** para todo $0 \leq j < k$ existe una acción conjunta $\mathbf{a}_j \in \mathbf{L}(s^{(j)})$ tal que $s^{(j+1)} = T(s^{(j)}, \mathbf{a}_j)$.

El número k es la **longitud** de la partida y $\mathrm{Part}(\Gamma)$ denota el conjunto de todas las partidas de Γ.

Cada condición tiene un propósito. La de inicio exige que toda partida empiece en la configuración de $t = 0$. La de fin exige que llegue a un estado terminal, donde se calculan las utilidades G_i. La de continuidad impide que una secuencia "siga jugando" después de alcanzar un estado terminal, puesto que en F la partida se detiene. La de legalidad exige que cada paso sea una transición permitida: el estado $s^{(j+1)}$ debe ser el sucesor que produce la función T de la Ecuación (3.17) al aplicar a $s^{(j)}$ alguna acción conjunta legal en el sentido de la Ecuación (3.16). Las condiciones de continuidad y legalidad son las mismas que impone la Ecuación (3.20) a las trayectorias que definen la distancia d; una partida es, por lo tanto, una trayectoria legal de s_0 a F.

Una partida se define como secuencia de estados y no de acciones. En el Gato ambas descripciones son equivalentes, porque dos jugadas distintas desde un mismo estado marcan casillas distintas y producen sucesores distintos: la secuencia de estados determina la secuencia de jugadas y viceversa. En un juego en el que dos acciones distintas pudieran producir el mismo sucesor habría que distinguir entre ambas nociones.

Conjuntos en que viven los estados y las partidas

La diferencia entre estados y partidas se comprende de forma muy intuitiva con una metáfora visual: un estado es una fotografía instantánea del tablero en un instante determinado, mientras que una partida es una película completa formada por la sucesión ordenada de esas fotografías desde el inicio hasta el desenlace. Esta distinción se refleja formalmente en el tipo de conjunto matemático que contiene a cada uno. Un estado es simplemente un conjunto de fluents, sin orden ni repeticiones, y por eso vive en el conjunto potencia del universo de fluents: $S \subseteq \mathcal{P}(\mathcal{F})$, de modo que $|\mathcal{P}(\mathcal{F})| = 2^{|\mathcal{F}|}$ es una cota superior del número de estados posibles (Ecuación (4.3)). Una partida, en cambio, es una secuencia estrictamente *ordenada* de estados, y aquí el orden de las escenas sí importa: (s_0, s_1, s_2) y (s_0, s_1', s_2) son partidas distintas aunque terminen en el mismo tablero final. Por esta razón, el conjunto potencia no sirve para contar partidas. El conjunto adecuado es el de las secuencias finitas de estados,

$$\mathrm{Part}(\Gamma) \;\subseteq\; S^{*} = \bigcup_{k \geq 0} S^{k+1}, \tag{4.5}$$

donde $S^{k+1} = S \times \cdots \times S$ es el producto cartesiano de $k+1$ copias de S, es decir, el conjunto de secuencias de $k+1$ estados, y S^{*} es la cerradura de Kleene de S, construida igual que Σ^{*} pero con estados en lugar de caracteres. Si toda partida tiene longitud a lo sumo $H_{\text{máx}}$, como garantiza la Ecuación (3.22) en un juego que termina, la unión se detiene en $k = H_{\text{máx}}$ y se obtiene la cota $|\,\mathrm{Part}(\Gamma)| \leq \sum_{k=0}^{H_{\text{máx}}} |S|^{k+1}$. Esta cota es muy holgada, porque cuenta todas las secuencias de estados y no solo las que respetan las transiciones; en la Proposición 4.3 se obtiene el valor exacto mediante una recurrencia sobre el grafo de juego.

Número de estados del Gato

En lo que sigue, un estado del Gato se identifica con su tablero: la asignación de una marca `x`, `o` o `b` a cada una de las nueve casillas. Se escribe $\#_{\texttt{x}}(s)$ y $\#_{\texttt{o}}(s)$ para el número de casillas marcadas con `x` y con `o` en s, y $k = \#_{\texttt{x}}(s) + \#_{\texttt{o}}(s)$ para el número total de marcas, que es también la profundidad del estado en el árbol de juego. Una *línea* es cualquiera de los ocho conjuntos de tres casillas alineadas (tres filas, tres columnas y dos diagonales) que definen las reglas de `line` del Código 3.5. Dos hechos geométricos elementales se usarán varias veces: dos líneas distintas comparten a lo sumo una casilla, y por lo tanto contener dos líneas exige al menos $3 + 3 - 1 = 5$ casillas.

Proposición 4.1: Invariante de conteo

Todo estado alcanzable s del Gato con k marcas cumple

$$\#_{\texttt{x}}(s) = \lceil k/2 \rceil\,, \qquad \#_{\texttt{o}}(s) = \lfloor k/2 \rfloor\,, \tag{4.6}$$

y el turno corresponde a `x` si k es par y a `o` si k es impar. En consecuencia, el tablero determina por completo el estado.

Demostración. Por inducción sobre k. Las funciones techo $\lceil\cdot\rceil$ y piso $\lfloor\cdot\rfloor$ redondean hacia arriba y hacia abajo. Para $k = 0$, el estado s_0 tiene cero marcas de cada tipo y el turno de `x`. Supóngase la afirmación cierta para los estados con k marcas y sea s' un sucesor de uno de ellos, s. Según el Código 3.3, solo el rol con el turno puede marcar, y según el Código 3.4, la transición agrega exactamente una marca de ese rol, conserva las demás e intercambia el turno. Si k es par, marca `x`, de modo que $\#_{\texttt{x}}$ pasa de $k/2$ a $k/2 + 1 = \lceil (k+1)/2 \rceil$, $\#_{\texttt{o}}$ se mantiene en $k/2 = \lfloor (k+1)/2 \rfloor$ y el turno pasa a `o`, como corresponde a $k + 1$ impar. El caso k impar es simétrico. Como el turno se deduce de la paridad de k, dos estados alcanzables con el mismo tablero tienen también el mismo turno y, por lo tanto, son el mismo estado. ∎

Sea B_k el conjunto de tableros con exactamente $\lceil k/2 \rceil$ marcas `x` y $\lfloor k/2 \rfloor$ marcas `o`. Para construir uno se eligen primero las casillas de `x` entre las nueve y después las de `o` entre las restantes; con $x = \lceil k/2 \rceil$ y $o = \lfloor k/2 \rfloor$, el principio multiplicativo da el coeficiente multinomial

$$|B_k| = \binom{9}{x}\binom{9-x}{o} = \frac{9!}{x!\;o!\;(9-k)!}, \tag{4.7}$$

donde $\binom{a}{b} = \frac{a!}{b!\,(a-b)!}$ es el número de maneras de elegir b elementos de un conjunto de a. Por la Proposición 4.1, el conjunto R_k de estados alcanzables con k marcas cumple $R_k \subseteq B_k$. Que un tablero de B_k sea alcanzable depende, además, de cómo pudo haberse llegado a él. La Definición 4.6 da directamente el criterio: para $k \geq 1$, un tablero $b \in B_k$ pertenece a R_k si y solo si existe una casilla con la marca del último jugador (`x` si k es impar, `o` si k es par) cuya eliminación produce un tablero $b' \in R_{k-1}$ que *no es terminal*. En efecto, b' es el estado anterior a la última jugada, debe ser alcanzable y, para que la partida haya continuado, no puede pertenecer a F.

Proposición 4.2: Número de estados alcanzables del Gato

El juego denotado por la descripción del Gato tiene exactamente 5 478 estados alcanzables, distribuidos según el número de marcas como indica la Tabla 4.1.

Demostración. Se calcula $|R_k|$ para cada $k = 0, \ldots, 9$ con el criterio anterior.

Caso $k \leq 4$. Ningún jugador tiene más de dos marcas, de modo que nadie puede tener una línea y, como quedan casillas vacías, ningún estado es terminal. Por inducción, todo tablero de B_k es alcanzable: al quitarle una marca del último jugador se obtiene un tablero de B_{k-1}, alcanzable por hipótesis y no terminal. Así $|R_k| = |B_k|$, y la Ecuación (4.7) da 1, 9, 72, 252 y 756 para $k = 0, 1, 2, 3, 4$.

Caso $k = 5$ (tres `x`, dos `o`). Al quitar cualquier `x` se obtiene un tablero de $B_4 = R_4$, que no es terminal. Todo tablero es alcanzable y $|R_5| = |B_5| = 9!/(3!\,2!\,4!) = 1\,260$.

Caso $k = 6$ (tres `x`, tres `o`; acaba de jugar `o`). El predecesor se obtiene quitando una `o`; pertenece a $R_5 = B_5$ y no es terminal si y solo si `x` no tiene línea, porque `o` solo tiene dos marcas. Quitar una `o` no altera las marcas de `x`, así que b es alcanzable si y solo si `x` no tiene línea en b. Los tableros de B_6 en los que las tres `x` forman una línea son $8 \cdot \binom{6}{3} = 8 \cdot 20 = 160$: se elige una de las 8 líneas para `x` y tres de las seis casillas restantes para `o`. Luego $|R_6| = 1\,680 - 160 = 1\,520$.

Caso $k = 7$ (cuatro `x`, tres `o`; acaba de jugar `x`). El predecesor b', obtenido al quitar una `x`, debe estar en R_6 (`x` sin línea en b') y no ser terminal (ni `x` ni `o` con línea en b'). Las marcas de `o` no cambian, así que se necesita que `o` no tenga línea en b. Si esto ocurre, basta quitar una `x` adecuada: cuatro casillas contienen a lo sumo una línea, y quitando una casilla de ella (o cualquiera, si no la hay) `x` queda sin línea. Por lo tanto, b es alcanzable si y solo si `o` no tiene línea. Los tableros de B_7 con las tres `o` en línea son $8 \cdot \binom{6}{4} = 8 \cdot 15 = 120$, y $|R_7| = 1\,260 - 120 = 1\,140$.

Caso $k = 8$ (cuatro `x`, cuatro `o`; acaba de jugar `o`). Por el argumento simétrico, b es alcanzable si y solo si `x` no tiene línea. Un conjunto de cuatro casillas que contiene una línea se forma con una de las 8 líneas y una de las 6 casillas restantes, 48 conjuntos sin repeticiones, puesto que cuatro casillas no contienen dos líneas; las `o` ocupan cuatro de las cinco casillas que quedan, 5 maneras. Hay $48 \cdot 5 = 240$ tableros excluidos y $|R_8| = 630 - 240 = 390$.

Caso $k = 9$ (cinco `x`, cuatro `o`; tablero lleno). Se necesita que `o` no tenga línea y que exista una `x` cuya eliminación deje a `x` sin línea. Con cinco casillas, `x` contiene a lo sumo dos líneas, y si contiene dos, estas comparten exactamente una casilla; quitar esa casilla común, o cualquier casilla de la única línea, deja a `x` sin línea. La condición se reduce, pues, a que `o` no tenga línea. Los conjuntos de cuatro casillas de `o` que contienen una línea son $8 \cdot 6 = 48$, y $|R_9| = 126 - 48 = 78$.

Sumando, $|\,\mathrm{Reach}\,| = 1 + 9 + 72 + 252 + 756 + 1\,260 + 1\,520 + 1\,140 + 390 + 78 = 5\,478$. ■

El mismo razonamiento cuenta los estados terminales. Para $k = 5$, son los $8 \cdot \binom{6}{2} = 120$ tableros en que `x` completa una línea. Para $k = 6$, son los tableros alcanzables en que `o` tiene línea: $8 \cdot 20 = 160$ elecciones, menos aquellas en que las tres `x` forman también una línea, necesariamente paralela a la de `o`; cada una de las 6 filas y columnas tiene 2 líneas paralelas, lo que da $160 - 12 = 148$. Para $k = 7$, son los 444 tableros alcanzables en que `x` tiene línea; para $k = 8$, los 168 en que la tiene `o`; y para $k = 9$, los 78 tableros llenos, de los cuales 62 son victorias de `x` y 16 empates. En total, $|F \cap \mathrm{Reach}\,| = 120 + 148 + 444 + 168 + 78 = 958$, con 626 victorias de `x`, 316 de `o` y 16 empates.

La demostración muestra también por qué el conjunto potencia es una cota muy pobre en este

caso. Cada restricción del juego reduce el número de configuraciones posibles:

$$\underbrace{2^{29}}_{536\,870\,912} \;>\; \underbrace{2\cdot 3^{9}}_{39\,366} \;>\; \underbrace{3^{9}}_{19\,683} \;>\; \underbrace{\textstyle\sum_{k=0}^{9} |B_k|}_{6\,046} \;>\; \underbrace{|\,\mathrm{Reach}\,|}_{5\,478}. \tag{4.8}$$

El primer término cuenta todos los subconjuntos de los 29 fluents, incluidos los que asignan dos marcas a una misma casilla o ninguna. El segundo exige que cada casilla tenga exactamente una de sus tres marcas posibles, 3^9 tableros, y que exactamente uno de los dos roles tenga el turno. El tercero aprovecha que el turno queda determinado por el tablero (Proposición 4.1). El cuarto impone el invariante de conteo (4.6). El último impone, además, que no se haya seguido jugando después de una línea, que es la condición de continuidad de la Definición 4.3.

Árbol de juego, transposiciones y número de partidas

El *árbol de juego* es la representación de todas las partidas posibles: su raíz es s_0, los hijos de un nodo no terminal son sus sucesores legales y sus hojas son estados terminales. A diferencia del grafo de estados, en el árbol un mismo estado aparece tantas veces como formas haya de llegar a él; cada nodo del árbol representa una *historia*, un prefijo de partida. Cuando dos historias distintas conducen al mismo estado se habla de una *transposición*. La Figura 4.1 muestra los primeros niveles del árbol del Gato y una transposición.

El número de partidas puede calcularse sin enumerarlas, mediante una recurrencia sobre el grafo de juego. Sea $\mathrm{Suc}(s) = \{T(s,\mathbf{a}) \mid \mathbf{a} \in \mathbf{L}(s)\}$ el conjunto de sucesores legales de un estado no terminal s, y defínase el número de partidas que empiezan en s como

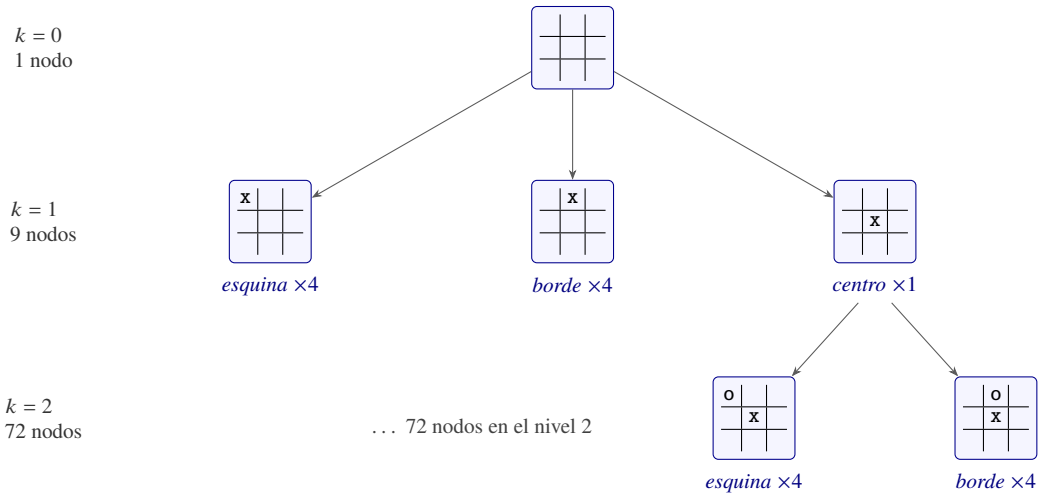
$$N(s) = \begin{cases} 1 & \text{si } s \in F, \\ \displaystyle\sum_{s' \in \mathrm{Suc}(s)} N(s') & \text{si } s \notin F. \end{cases} \tag{4.9}$$

Proposición 4.3: Número de partidas

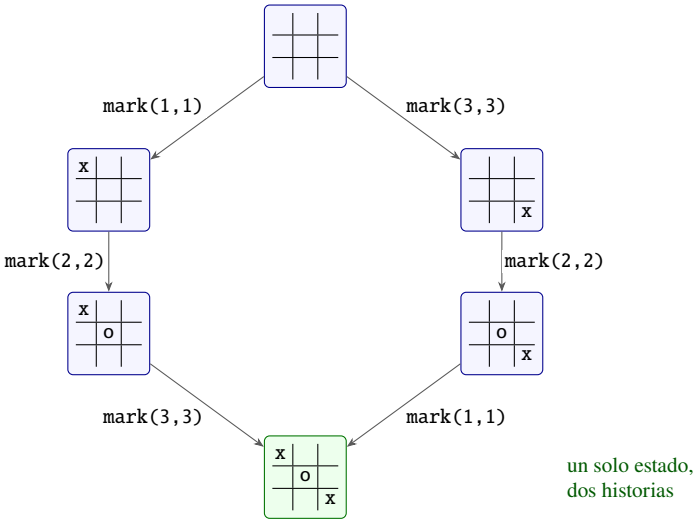
En todo juego que satisface la terminación, $|\,\mathrm{Part}(\Gamma)| = N(s_0)$. Para el Gato, $N(s_0) = 255\,168$.

Demostración. Llámese partida desde s a una secuencia que cumple la Definición 4.3 con s en lugar de s_0; se prueba que hay exactamente $N(s)$ de ellas, por inducción sobre la longitud máxima de las trayectorias legales que parten de s, finita por la condición de terminación y acotada por $H_{\text{máx}}$ (Ecuación (3.22)). Si $s \in F$, la condición de continuidad impide avanzar y la única partida desde s es la secuencia (s) de longitud cero, de modo que hay $1 = N(s)$. Si $s \notin F$, toda partida desde s tiene la forma $(s, s', \dots)$, donde $s' \in \mathrm{Suc}(s)$ por la condición de legalidad y el resto $(s', \dots)$ es una partida desde s'. Recíprocamente, anteponer s a cualquier partida desde un sucesor s' produce una partida desde s. Esta correspondencia es biyectiva y agrupa las partidas desde s según su segundo estado; por la hipótesis de inducción, el grupo de s' tiene $N(s')$ elementos, y el total es la suma de (4.9). Para el Gato, al evaluar la recurrencia sobre los 5 478 estados de la Proposición 4.2 se obtiene $N(s_0) = 255\,168$. ∎

La recurrencia (4.9) no recorre las 255 168 partidas: basta calcular $N(s)$ una vez por estado, empezando por los terminales, lo que requiere una suma por cada una de las 16 167 transiciones.



(a) Primeros niveles del árbol de juego, agrupando por simetría los tableros equivalentes.



(b) Transposición: dos historias distintas conducen al mismo estado.

Figura 4.1: Árbol de juego del Gato. Cada nodo del árbol es una historia; un mismo estado puede corresponder a varios nodos, como muestra la transposición del panel (b).

Es la misma idea que permite a los motores de búsqueda reutilizar el valor de un estado alcanzado por caminos distintos mediante tablas de transposición. Dos comprobaciones manuales confirman algunas cifras. Una partida que termina con la quinta marca llega a uno de los 120 estados terminales de $k = 5$, y las tres `x` y dos `o` pudieron colocarse en cualquier orden, $3! \cdot 2! = 12$ órdenes, porque antes de la quinta marca nadie puede tener línea; hay $120 \cdot 12 = 1\,440$ partidas de longitud 5. De manera análoga, cada uno de los 148 estados terminales de $k = 6$ admite $3! \cdot 3! = 36$ órdenes, porque `x` no tiene línea y la línea de `o` solo se completa con su tercera marca, de donde salen $148 \cdot 36 = 5\,328$ partidas de longitud 6. Por otra parte, una partida queda determinada por el orden en que se marcan las casillas, que es un prefijo de una permutación de las nueve casillas, y por lo tanto $|\,\mathrm{Part}\,| \leq 9! = 362\,880$.

La Tabla 4.1 reúne por nivel el número de tableros compatibles con el invariante, de estados alcanzables, de estados terminales, de nodos del árbol (historias) y de partidas. Las 255 168 partidas se reparten en 131 184 victorias de `x`, 77 904 de `o` y 46 080 empates.

Tabla 4.1: Estados, historias y partidas del Gato según el número k de marcas en el tablero. B_k: tableros compatibles con el invariante (4.6); R_k: estados alcanzables; F_k: estados terminales alcanzables; H_k: nodos del árbol de juego; P_k: partidas de longitud k.

k	0	1	2	3	4	5	6	7	8	9	Total
$\lvert B_k \rvert$	1	9	72	252	756	1 260	1 680	1 260	630	126	6 046
$\lvert R_k \rvert$	1	9	72	252	756	1 260	1 520	1 140	390	78	5 478
$\lvert F_k \rvert$	0	0	0	0	0	120	148	444	168	78	958
H_k	1	9	72	504	3 024	15 120	54 720	148 176	200 448	127 872	549 946
P_k	0	0	0	0	0	1 440	5 328	47 952	72 576	127 872	255 168

La comparación entre las filas R_k y H_k mide el efecto de las transposiciones: el árbol tiene 549 946 nodos, unas cien veces más que estados distintos. Un algoritmo que explore el árbol sin reconocer estados repetidos, como un Minimax ingenuo, visita cada estado tantas veces como historias conducen a él; uno que trabaje sobre el grafo, con memoria de los estados ya evaluados, visita cada estado una sola vez.

Relación con la distancia entre estados: conteo de trayectorias

Para comprender esta sección de manera muy simple, imagínese un mapa de caminos entre distintas posiciones del tablero: si queremos avanzar desde un estado inicial s hasta alcanzar un estado meta s', podemos hacernos dos preguntas fundamentales. La primera es: *¿cuántas jugadas tiene el camino más corto para llegar?* A esa medida la llamamos **distancia** (el mínimo de pasos). La segunda pregunta es: *¿cuántas rutas distintas existen para llegar a ese mismo desenlace?* A eso lo llamamos **conteo de trayectorias** (la cantidad de partidas diferentes que conducen a esa posición). Ambos conceptos estudian exactamente el mismo objeto matemático: el conjunto de secuencias de jugadas legales que unen dos configuraciones del juego. La distancia toma la menor longitud dentro de ese conjunto de caminos, mientras que el conteo de partidas mide su tamaño o cardinalidad.

Definición 4.4: Conjunto de trayectorias entre dos estados

Para $s, s' \in S$ y $k \in \mathbb{N}_0$, sea $\mathrm{Tray}_k(s, s')$ el conjunto de secuencias $(s^{(0)}, \ldots, s^{(k)})$ con $s^{(0)} = s$, $s^{(k)} = s'$, $s^{(j)} \notin F$ y $s^{(j+1)} = T(s^{(j)}, \mathbf{a}_j)$ para algún $\mathbf{a}_j \in \mathbf{L}(s^{(j)})$, para todo $0 \le j < k$. Son exactamente las trayectorias que intervienen en la Ecuación (3.20). Se define

$$\mathrm{Tray}(s, s') = \bigcup_{k \ge 0} \mathrm{Tray}_k(s, s'). \tag{4.10}$$

Con esta notación, la distancia del Capítulo 3 se reescribe como

$$d(s, s') = \min\{\, k \in \mathbb{N}_0 \mid \mathrm{Tray}_k(s, s') \neq \emptyset \,\}, \tag{4.11}$$

con $\min \emptyset = \infty$, y el conjunto de partidas de la Definición 4.3 es la unión de las trayectorias que salen de s_0 y llegan a algún estado terminal:

$$\mathrm{Part}(\Gamma) = \biguplus_{f \in F} \mathrm{Tray}(s_0, f), \qquad |\mathrm{Part}(\Gamma)| = \sum_{f \in F} |\mathrm{Tray}(s_0, f)|. \tag{4.12}$$

La unión es disjunta, y por eso las cardinalidades se suman, porque una trayectoria tiene un único estado final: dos trayectorias que terminan en estados distintos $f \neq f'$ son necesariamente distintas. Las cuatro condiciones de la Definición 4.3 quedan absorbidas: el inicio, porque las trayectorias parten de s_0; el fin, porque $f \in F$; la continuidad y la legalidad, porque son parte de la definición de Tray. En palabras sencillas: $d(s_0, f)$ es la longitud de la partida *más corta* que termina en f, mientras que $|\mathrm{Tray}(s_0, f)|$ es el número de partidas *distintas* que terminan en f.

Trayectorias desde todos los estados. Una partida completa siempre arranca en el estado inicial s_0 y concluye en un estado terminal de victoria o empate. No obstante, durante el desarrollo del juego un agente suele encontrarse a la mitad del camino. Una **continuación** es precisamente eso: imagínese cargar una partida guardada en un turno intermedio y jugar desde esa posición hasta el desenlace. Formalmente, si en la Ecuación (4.12) se reemplaza s_0 por un estado arbitrario y se reúnen las trayectorias que salen de todos los estados posibles, se obtiene el conjunto

$$\mathrm{Cont}(\Gamma) = \biguplus_{s \in S} \; \biguplus_{f \in F} \mathrm{Tray}(s, f), \tag{4.13}$$

cuyos elementos representan todas las formas legales de terminar el juego a partir de alguna configuración intermedia. Este conjunto *no* coincide con $\mathrm{Part}(\Gamma)$, sino que lo contiene como un caso particular: las partidas completas son simplemente aquellas continuaciones que tuvieron la particularidad de comenzar desde el principio absoluto (s_0). Si $S = \mathrm{Reach}(\Gamma)$, las continuaciones son exactamente las *colas* o sufijos de las partidas. En efecto, si $(s^{(j)}, \ldots, s^{(k)})$ es la cola de una partida, es una trayectoria legal de $s^{(j)}$ a $s^{(k)} \in F$. Recíprocamente, si s es alcanzable, existe una trayectoria de s_0 a s que no pasa por F antes de llegar a s, y al concatenarla con cualquier continuación desde s se obtiene una partida de la que esa continuación es cola. Si, en cambio, se tomara $S = \mathcal{P}(\mathcal{F})$, la unión (4.13) incluiría además trayectorias que parten de configuraciones inalcanzables y que no forman parte de ninguna partida. El conjunto de todas las partidas es, por lo tanto, el término $s = s_0$ de la unión (4.13), y el resto de los términos cuenta sus continuaciones.

La recurrencia (4.9) se relaciona directamente con estos conjuntos: $N(s) = \sum_{f \in F} |\operatorname{Tray}(s, f)|$ es el número de continuaciones desde s. En consecuencia,

$$|\operatorname{Part}(\Gamma)| = N(s_0), \qquad |\operatorname{Cont}(\Gamma)| = \sum_{s \in S} N(s). \tag{4.14}$$

Formulación matricial. Las dos magnitudes, la distancia y el número de trayectorias, se obtienen de una misma matriz. Sea A la *matriz de adyacencia* del grafo de juego: una matriz cuadrada con una fila y una columna por estado, definida por

$$A_{s,s'} = \begin{cases} 1 & \text{si } s \notin F \text{ y } s' \in \operatorname{Suc}(s), \\ 0 & \text{en otro caso.} \end{cases} \tag{4.15}$$

Las filas de los estados terminales son nulas: así se impone la condición de continuidad, porque ninguna trayectoria puede seguir después de pasar por F. Con $A^0 = I$, la matriz identidad, la entrada (s, s') de la potencia A^k es el número de trayectorias de longitud exactamente k:

$$(A^k)_{s,s'} = |\operatorname{Tray}_k(s, s')|. \tag{4.16}$$

La prueba es por inducción sobre k. Para $k = 0$, la única trayectoria de longitud cero de s a s' existe si y solo si $s = s'$, que es lo que dice I. Para el paso inductivo, el producto de matrices da $(A^{k+1})_{s,s'} = \sum_{u \in S} (A^k)_{s,u}\, A_{u,s'}$, y esta suma agrupa las trayectorias de longitud $k + 1$ según su penúltimo estado u: hay $(A^k)_{s,u}$ maneras de llegar de s a u en k pasos y $A_{u,s'} \in \{0, 1\}$ indica si de u se puede dar el último paso hacia s'.

De la Ecuación (4.16) se siguen las dos magnitudes. La distancia es la primera potencia en la que la entrada se vuelve positiva, y el número de trayectorias es la suma de todas las potencias:

$$d(s, s') = \min\{\, k \in \mathbb{N}_0 \mid (A^k)_{s,s'} > 0 \,\}, \qquad |\operatorname{Tray}(s, s')| = \sum_{k \geq 0} (A^k)_{s,s'} = C_{s,s'}, \tag{4.17}$$

donde $C = \sum_{k \geq 0} A^k$. Si el juego satisface la terminación, el grafo de juego no tiene ciclos y ninguna trayectoria es más larga que $H_{\text{máx}}$; por lo tanto, $A^k = 0$ para todo $k > H_{\text{máx}}$ (se dice que A es *nilpotente*), la suma es finita y coincide con la inversa de $I - A$:

$$C = \sum_{k=0}^{H_{\text{máx}}} A^k = (I - A)^{-1}. \tag{4.18}$$

La igualdad se comprueba multiplicando: $(I - A) \sum_{k=0}^{H_{\text{máx}}} A^k = I - A^{H_{\text{máx}}+1} = I$. Sean $\mathbf{1}_F$ el vector con un 1 en las posiciones de los estados terminales y 0 en las demás, $\mathbf{1}$ el vector de unos y $\mathbf{e}_{s_0}$ el vector con un único 1 en la posición de s_0. Las Ecuaciones (4.12) y (4.14) toman entonces la forma

$$N = C\,\mathbf{1}_F, \qquad |\operatorname{Part}(\Gamma)| = \mathbf{e}_{s_0}^{\top} C\,\mathbf{1}_F, \qquad |\operatorname{Cont}(\Gamma)| = \mathbf{1}^{\top} C\,\mathbf{1}_F, \tag{4.19}$$

donde $\top$ indica transposición. Multiplicar C por $\mathbf{1}_F$ suma, en cada fila s, las trayectorias hacia todos los estados terminales, que es $N(s)$; $\mathbf{e}_{s_0}^{\top}$ selecciona la fila de s_0, y $\mathbf{1}^{\top}$ suma todas las filas. La recurrencia (4.9) es la manera eficiente de calcular el vector $N = C\,\mathbf{1}_F$ sin construir C: como $N = \mathbf{1}_F + AN$, cada $N(s)$ se obtiene de los de sus sucesores.

Aplicación al Gato. En el Gato, A es una matriz de $5\,478 \times 5\,478$ con 16 167 entradas iguales a 1, una por transición, y es nilpotente con $A^{10} = 0$, porque ninguna partida tiene más de nueve jugadas. El juego tiene, además, una propiedad que simplifica la Ecuación (4.17): como cada transición agrega exactamente una marca, toda trayectoria de s a s' tiene la misma longitud, igual a la diferencia de marcas $k(s') - k(s)$. Por lo tanto, la única potencia que contribuye es esa,

$$d(s, s') = k(s') - k(s) \qquad \text{y} \qquad |\operatorname{Tray}(s, s')| = \left(A^{\,k(s')-k(s)}\right)_{s,s'} \tag{4.20}$$

siempre que s' sea alcanzable desde s. Para un estado terminal f, $d(s_0, f) = k(f)$ es su número de marcas, y $|\operatorname{Tray}(s_0, f)|$ es el número de órdenes en que pudieron colocarse sus marcas sin que la partida terminara antes. Los valores extremos lo ilustran. Para un estado terminal con cinco marcas, cualquier orden de las tres `x` y las dos `o` es válido, y $|\operatorname{Tray}(s_0, f)| = 3! \cdot 2! = 12$, el mínimo. Para un empate, en el que nadie formó línea en ningún momento, cualquier orden de las cinco `x` y las cuatro `o` es válido, y $|\operatorname{Tray}(s_0, f)| = 5! \cdot 4! = 2\,880$, el máximo. Al sumar sobre los 958 estados terminales se obtiene, de acuerdo con la Ecuación (4.12), $|\operatorname{Part}| = 255\,168$. Al sumar en cambio $N(s)$ sobre los 5 478 estados alcanzables, según la Ecuación (4.14), se obtiene $|\operatorname{Cont}| = 987\,496$ continuaciones: casi cuatro veces el número de partidas, porque cada partida de longitud k tiene $k + 1$ colas y muchas colas son compartidas por partidas distintas. Estas cifras se obtuvieron con el mismo procedimiento de enumeración empleado para la Tabla 4.1.

Es fundamental destacar que esta elegante formulación matricial tiene un carácter estrictamente analítico y teórico, útil para unificar la distancia y el conteo de trayectorias en un marco matemático riguroso. Sin embargo, a nivel computacional y en la implementación real del software, construir, almacenar y operar matrices de adyacencia de miles de dimensiones resulta una tarea con un gasto computacional inasumible en memoria y tiempo de procesamiento. Por este motivo, el intérprete y el motor de decisión desarrollados en este proyecto no operan construyendo matrices completas ni invirtiendo sistemas lineales; en su lugar, exploran el juego de manera dinámica y hacia adelante conforme avanza la partida, valiéndose de tablas de transposición para reutilizar el cómputo de estados ya visitados sin sufrir la penalización de una representación global exhaustiva.

Comparación entre el número de partidas y el número de estados

Las cifras anteriores llevan a comparar el número de partidas distintas, $|\operatorname{Part}(\Gamma)|$, con el número de estados, $|S|$. En el Gato hay 255 168 partidas y 5 478 estados, y en el Ajedrez las estimaciones de Shannon dan 10^{120} frente a 10^{43}; parecería que las partidas siempre superan a los estados. No es así. La respuesta general es que ninguna de las dos magnitudes domina a la otra en todos los juegos, pero cada una acota a la otra, y la diferencia entre ambas está gobernada por las transposiciones. En esta subsección se toma $S = \operatorname{Reach}(\Gamma)$, se escribe F en lugar de $F \cap \operatorname{Reach}(\Gamma)$ y se definen dos parámetros del juego: la **longitud máxima de una partida**, $\ell = \max\{\, k \mid (s^{(0)}, \ldots, s^{(k)}) \in \operatorname{Part}(\Gamma) \,\}$, y el **factor de ramificación máximo**, $b = \max_{s \in S \setminus F} |\operatorname{Suc}(s)|$. Ambos son finitos si el juego satisface la terminación, porque entonces el árbol de juego es finito (Proposición 4.8).

El primer paso es comprobar que ningún estado alcanzable queda fuera de todas las partidas.

Proposición 4.4: Todo estado alcanzable pertenece a alguna partida

Sea Γ un juego que satisface la terminación y la jugabilidad. Para todo $s \in \text{Reach}(\Gamma)$ existe una partida $\mathbf{w} \in \text{Part}(\Gamma)$ que contiene a s.

Demostración. Como s es alcanzable, por la Ecuación (4.29) existe una trayectoria legal $(s_0, \ldots, s)$, es decir, $\text{Tray}(s_0, s) \neq \emptyset$. Si $s \in F$, esa trayectoria ya es una partida. Si $s \notin F$, se prolonga del siguiente modo: por la jugabilidad, cada $L_i(s)$ es no vacío, de modo que $\mathbf{L}(s) = \prod_i L_i(s) \neq \emptyset$ y existe al menos un sucesor $T(s, \mathbf{a})$, que también es alcanzable. Se elige uno y se repite el razonamiento mientras el estado obtenido no sea terminal. Este proceso no puede continuar indefinidamente, porque produciría una sucesión infinita de estados alcanzables no terminales unidos por transiciones legales, lo que contradice la terminación (Definición 4.22). Por lo tanto se alcanza un estado terminal tras un número finito de pasos, y la concatenación de la trayectoria inicial con la prolongación es una partida que contiene a s. ■

Teorema 4.1: Relación entre el número de partidas y el número de estados

Sea Γ un juego que satisface la terminación y la jugabilidad, con $S = \text{Reach}(\Gamma)$, longitud máxima de partida ℓ y factor de ramificación máximo b. Entonces:

1. **Cotas para las partidas:**

$$|F| \;\leq\; |\,\text{Part}(\Gamma)| \;\leq\; b^{\ell}. \tag{4.21}$$

2. **Cota de los estados por las partidas:**

$$|S| \;\leq\; (\ell + 1)\,|\,\text{Part}(\Gamma)|. \tag{4.22}$$

3. **Juegos sin transposiciones:** si todo estado alcanzable tiene una única historia, es decir, $|\,\text{Tray}(s_0, s)| = 1$ para todo $s \in S$, y $s_0 \notin F$, entonces

$$|\,\text{Part}(\Gamma)| = |F| < |S|. \tag{4.23}$$

   En consecuencia, $|\,\text{Part}(\Gamma)| \geq |S|$ solo es posible si el juego tiene transposiciones.

4. **Ausencia de un orden general:** para cada $\ell \geq 1$ existen juegos bien formados con $|\,\text{Part}(\Gamma)| = 1$ y $|S| = \ell + 1$, y juegos bien formados con $|\,\text{Part}(\Gamma)| = 2^{\ell}$ y $|S| = 2\ell + 1$. Por lo tanto, ni $|\,\text{Part}(\Gamma)| \leq |S|$ ni $|\,\text{Part}(\Gamma)| \geq |S|$ se cumple para todos los juegos.

Demostración.

Parte 1, cota inferior. Cada estado terminal $f \in F$ es alcanzable, de modo que $\text{Tray}(s_0, f) \neq \emptyset$ y $|\,\text{Tray}(s_0, f)| \geq 1$. Por la Ecuación (4.12),

$$|\,\text{Part}(\Gamma)| = \sum_{f \in F} |\,\text{Tray}(s_0, f)| \;\geq\; \sum_{f \in F} 1 = |F|.$$

Cada estado terminal es el final de al menos una partida, y partidas con finales distintos son distintas.

Parte 1, cota superior. Para $s \in S$, sea $h(s)$ la longitud máxima de una trayectoria legal de s a F, de modo que $h(s) = 0$ si $s \in F$ y $h(s') \le h(s) - 1$ para todo $s' \in \mathrm{Suc}(s)$. Se prueba por inducción sobre $h(s)$ que $N(s) \le b^{h(s)}$, con N la función de la Ecuación (4.9). Si $h(s) = 0$, entonces $s \in F$ y $N(s) = 1 = b^0$. Si $h(s) > 0$, la recurrencia y la hipótesis de inducción dan

$$N(s) = \sum_{s' \in \mathrm{Suc}(s)} N(s') \;\le\; |\,\mathrm{Suc}(s)| \cdot b^{h(s)-1} \;\le\; b \cdot b^{h(s)-1} = b^{h(s)}.$$

Como $h(s_0) = \ell$ y $|\,\mathrm{Part}(\Gamma)| = N(s_0)$ por la Proposición 4.3, se obtiene $|\,\mathrm{Part}(\Gamma)| \le b^{\ell}$. La jugabilidad garantiza $b \ge 1$. En palabras: una partida se forma eligiendo, en cada uno de a lo sumo ℓ pasos, uno de a lo sumo b sucesores.

Parte 2. Por la Proposición 4.4, todo estado de S aparece en alguna partida, de modo que S está contenido en la unión de los conjuntos de estados de todas las partidas. Una partida de longitud $k \le \ell$ contiene a lo sumo $k + 1 \le \ell + 1$ estados distintos. El tamaño de una unión no excede la suma de los tamaños, y por lo tanto

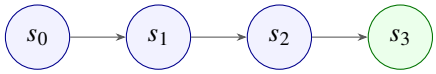
$$|S| \;\le\; \sum_{\mathbf{w} \in \mathrm{Part}(\Gamma)} \big|\{\text{estados de } \mathbf{w}\}\big| \;\le\; (\ell + 1)\,|\,\mathrm{Part}(\Gamma)|.$$

Parte 3. Si $|\,\mathrm{Tray}(s_0, s)| = 1$ para todo s, la Ecuación (4.12) da $|\,\mathrm{Part}(\Gamma)| = \sum_{f \in F} 1 = |F|$. Como $s_0 \in S$ y $s_0 \notin F$, el conjunto F está estrictamente contenido en S y $|F| < |S|$. Por contraposición, si $|\,\mathrm{Part}(\Gamma)| \ge |S|$, algún estado tiene al menos dos historias, es decir, hay una transposición. Si $s_0 \in F$, el juego termina antes de empezar, $S = \{s_0\}$ y ambas magnitudes valen 1.

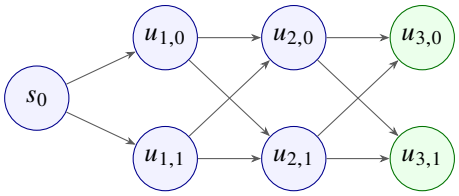
Parte 4. Se exhiben dos familias de juegos con un único rol, ilustradas en la Figura 4.2. En la *cadena*, los estados son $s_0, s_1, \ldots, s_\ell$, el único rol tiene una sola acción legal a en cada estado no terminal, $T(s_j, a) = s_{j+1}$, $F = \{s_\ell\}$ y la utilidad en s_ℓ es 100. Es bien formado: termina tras ℓ pasos, siempre hay una acción legal y el único rol gana. Tiene $|S| = \ell + 1$ estados y una sola partida, de modo que $|\,\mathrm{Part}\,| = 1 < |S|$; además alcanza con igualdad la cota (4.22). En la *escalera*, los estados son s_0 y $u_{j,c}$ para $j = 1, \ldots, \ell$ y $c \in \{0, 1\}$; el rol tiene dos acciones, a_0 y a_1, y la acción a_c lleva de s_0 a $u_{1,c}$ y de cualquier $u_{j,c'}$ con $j < \ell$ a $u_{j+1,c}$; los terminales son $u_{\ell,0}$ y $u_{\ell,1}$, con utilidad 100. También es bien formado. Tiene $|S| = 2\ell + 1$ estados, y cada partida corresponde a una elección de $c \in \{0, 1\}$ en cada uno de los ℓ pasos; elecciones distintas producen secuencias de estados distintas, de modo que $|\,\mathrm{Part}\,| = 2^{\ell}$, que alcanza con igualdad la cota superior de (4.21) con $b = 2$. Para $\ell \ge 3$ se cumple $2^{\ell} > 2\ell + 1$, y las partidas superan a los estados. ∎

La Tabla 4.2 contrasta los juegos analizados en este capítulo. El juego de la resta de la Sección 4.6 muestra que la condición de la parte 3 es solo necesaria: tiene una transposición, porque el estado $\langle 0, \mathtt{p1}\rangle$ se alcanza por $\langle 3, \mathtt{p1}\rangle \to \langle 2, \mathtt{p2}\rangle \to \langle 0, \mathtt{p1}\rangle$ y por $\langle 3, \mathtt{p1}\rangle \to \langle 1, \mathtt{p2}\rangle \to \langle 0, \mathtt{p1}\rangle$, y aun así sus 3 partidas son menos que sus 6 estados. En el Gato las cotas del teorema se verifican con holgura: $958 \le 255\,168 \le 9^9$ y $5\,478 \le 10 \cdot 255\,168$.

El teorema explica también por qué, en los juegos grandes, las partidas terminan dominando. Por la Ecuación (4.21), el número de partidas puede crecer como b^{ℓ}, exponencialmente con la longitud, porque las elecciones de cada paso se *multiplican*. Los estados, en cambio, se *acumulan*: cada estado se cuenta una sola vez, sin importar cuántas historias lleven a él, y su número está acotado por el número de configuraciones posibles, como en la cadena de cotas (4.8). Cuando las partidas son largas y muchas historias convergen en los mismos estados, como en el Gato y



(a) Cadena, $\ell = 3$
$|S| = 4$, $|\,\mathrm{Part}\,| = 1$



(b) Escalera, $\ell = 3$
$|S| = 7$, $|\,\mathrm{Part}\,| = 2^3 = 8$

Figura 4.2: Dos familias de juegos que muestran que no hay un orden general entre el número de partidas y el número de estados. En la cadena no hay transposiciones y cada estado tiene una única historia; en la escalera, cada estado del nivel j tiene 2^{j-1} historias, y las partidas crecen exponencialmente mientras los estados crecen linealmente.

Tabla 4.2: Número de estados y de partidas en los juegos de este capítulo.

Juego	ℓ	$\lvert S\rvert$	$\lvert\,\mathrm{Part}\,\rvert$	**Relación**
Cadena	ℓ	$\ell + 1$	1	Menos partidas que estados; sin transposiciones.
Escalera	ℓ	$2\ell + 1$	2^ℓ	Más partidas que estados para $\ell \geq 3$.
Resta (pila de 3)	3	6	3	Menos partidas que estados, con una transposición.
Gato	9	5 478	255 168	Unas 47 veces más partidas que estados.
Ajedrez (estimación)	–	$\sim 10^{43}$	$\sim 10^{120}$	Las partidas dominan por 77 órdenes de magnitud.

sobre todo en el Ajedrez, el producto supera con mucho a la suma. Cuando las partidas son cortas o casi no hay transposiciones, como en la cadena o en el juego de la resta, ocurre lo contrario. Para el motor de decisión la lección es la misma que se obtuvo de la Tabla 4.1: explorar el grafo de estados con memoria de los estados visitados puede ser exponencialmente más barato que explorar el árbol de partidas.

Estimación para el Ajedrez

En el Ajedrez no es posible un cálculo exacto como el realizado para el Gato. La imposibilidad no se debe a que falte una fórmula ingeniosa, sino a cuatro obstáculos de naturaleza distinta que se refuerzan entre sí: el costo de recorrer el espacio, la ausencia de una caracterización sencilla de los estados alcanzables, la dependencia del estado respecto de la historia y la presencia de ciclos en el grafo de juego.

El método exacto exige recorrer todo el espacio. Las cifras del Gato se obtuvieron de dos maneras: por la demostración de la Proposición 4.2, que aprovecha la estructura particular de ese juego, y por enumeración, construyendo Reach con la iteración $X_0, X_1, X_2, \ldots$ de la Definición 4.6 y evaluando la recurrencia (4.9). La enumeración visita cada estado alcanzable y cada transición al menos una vez, y debe recordar los estados ya visitados para no contarlos dos veces; su tiempo es, por lo tanto, al menos proporcional a $|\,\mathrm{Reach}\,|$ más el número de transiciones, y su memoria al menos proporcional a $|\,\mathrm{Reach}\,|$. Con los 5 478 estados del Gato eso es trivial. Con los del orden de 10^{43} del Ajedrez no lo es: aun suponiendo una máquina capaz de procesar 10^{18} estados

por segundo, el recorrido tardaría unos $10^{43}/10^{18} = 10^{25}$ segundos, mientras que la edad del universo, unos 13 800 millones de años, equivale a unos 4.4×10^{17} segundos. El recorrido duraría decenas de millones de veces la edad del universo, y la memoria necesaria para recordar los estados visitados, aunque fuera un solo bit por estado, supera con mucho cualquier capacidad de almacenamiento existente. El argumento es el de la explosión combinatoria de la Sección 3.8, aplicado ahora no a la búsqueda de una jugada sino al simple conteo.

No existe una caracterización sencilla de los estados alcanzables. La demostración de la Proposición 4.2 evitó la enumeración gracias a tres propiedades del Gato. Primero, el juego es *monótono*: cada jugada agrega una marca y ninguna la quita, de modo que el número de marcas k coincide con la profundidad y el grafo de juego se organiza en niveles. Segundo, el contenido de un tablero queda determinado por cuántas marcas tiene cada jugador y dónde están, sin importar el orden en que se colocaron, salvo por la condición de que nadie haya formado línea antes del final. Tercero, esa condición se expresa con conteos simples sobre las ocho líneas. Gracias a ello, la alcanzabilidad se redujo a contar tableros con fórmulas cerradas como (4.7). El Ajedrez no tiene ninguna de estas propiedades. Las piezas se desplazan sin que se agregue nada al tablero, las capturas eliminan piezas, las promociones cambian el tipo de una pieza y una misma posición puede alcanzarse con partidas de longitudes muy distintas. Decidir si una colocación concreta de piezas es alcanzable desde la posición inicial exige razonar hacia atrás sobre la historia que pudo producirla, lo que en ajedrez se conoce como análisis retrógrado. Algunas restricciones son evidentes, como que un peón no puede estar en la primera fila de su propio bando. Otras dependen de varias piezas a la vez: dos peones del mismo bando en una misma columna implican que hubo al menos una captura, y por lo tanto que falta alguna pieza del rival; dos alfiles del mismo bando sobre casillas del mismo color implican una promoción. Estas condiciones interactúan entre sí y no se conoce una fórmula cerrada que las combine. La estimación de Shannon que se presenta más adelante cuenta colocaciones de piezas sin verificar si son alcanzables, y por eso es solo una aproximación.

El estado depende de la historia de la partida. En el Gato, el tablero determina el estado (Proposición 4.1). En el Ajedrez no: como se advirtió en la Sección 3.2.6, la colocación de las piezas no basta para aplicar las reglas. Para saber si un enroque es legal hay que saber si el rey o la torre se movieron antes; para saber si una captura al paso es legal hay que saber cuál fue la jugada inmediatamente anterior; y las reglas de tablas por repetición de posiciones o por un número de jugadas sin capturas ni movimientos de peón dependen de partes más largas de la historia. Para que la función de transición T dependa solo del estado, como exige la Definición 3.4, el estado debe incluir toda esa información: derechos de enroque, casilla de captura al paso, contador de jugadas y, en rigor, las posiciones previas que intervienen en la regla de repetición. La consecuencia es que el número de estados del Ajedrez no tiene un valor único: depende de qué información se incluya en el estado. Contar solo colocaciones de piezas, contar colocaciones junto con turno, derechos de enroque y captura al paso, o contar estados completos con contadores e historia produce cifras distintas, y cada inclusión multiplica el total.

El grafo de posiciones tiene ciclos. La recurrencia (4.9) y la fórmula matricial (4.18) exigen que el grafo de juego no tenga ciclos, condición que en la Sección 4.1.3 se expresó diciendo

que la matriz de adyacencia A es nilpotente. Si los estados del Ajedrez se identifican con las posiciones del tablero, esa condición falla: basta que un caballo salga y regrese a su casilla, y que el rival haga lo mismo, para volver a la posición de partida. Entre dos posiciones unidas por un ciclo hay infinitas trayectorias, $A^k \neq 0$ para todo k, la suma $\sum_k A^k$ diverge y el número de partidas no es finito. Solo al incorporar al estado los contadores de la regla de repetición y de la regla de jugadas sin capturas, o un contador de pasos como el del Ejemplo 4.9, el grafo se vuelve acíclico y el número de partidas pasa a ser finito. Pero entonces ese número depende de la regla de terminación elegida, y la longitud máxima de una partida se cuenta en miles de jugadas, de modo que el número de partidas resulta todavía mucho mayor que el de estados.

Lo que sí puede calcularse con exactitud. El conteo exacto es viable a profundidades pequeñas. Desde la posición inicial, las blancas disponen de 20 jugadas: cada uno de los 8 peones puede avanzar una o dos casillas, 16 jugadas, y cada uno de los 2 caballos tiene 2 casillas de destino, 4 jugadas. Ninguna de esas jugadas altera las 20 respuestas iniciales de las negras, de modo que tras una jugada de cada bando hay exactamente $20 \cdot 20 = 400$ historias. A partir de ahí las jugadas empiezan a interactuar, y el número de historias crece aproximadamente en un factor de 30 por jugada, el factor de ramificación b que en la Sección 3.8 llevó a la estimación b^d del tamaño de un árbol de profundidad d. Los conteos exactos para unas cuantas jugadas se obtienen por enumeración exhaustiva, pero el costo crece de forma exponencial con la profundidad y se vuelve inalcanzable mucho antes de llegar a la longitud de una partida completa.

Por estas razones, las cifras disponibles para el Ajedrez completo son estimaciones de orden de magnitud. Shannon propuso las dos que se citan con más frecuencia [12].

Para el número de posiciones, Shannon contó las maneras de colocar las 32 piezas iniciales sobre las 64 casillas. Si las piezas fueran todas distinguibles, habría $64 \cdot 63 \cdots 33 = 64!/32!$ colocaciones: la primera pieza tiene 64 casillas disponibles, la segunda 63, y así hasta la trigésima segunda. Pero las piezas del mismo tipo y color son intercambiables, y permutarlas entre sí no produce una posición nueva: cada bando tiene 8 peones idénticos, que se pueden permutar de 8! maneras, y pares idénticos de torres, caballos y alfiles, que se pueden permutar de 2! maneras cada uno, seis pares en total. Al dividir por esas permutaciones,

$$\frac{64!}{32!\,(8!)^2\,(2!)^6} \approx 4.6 \times 10^{42} \approx 10^{43}. \tag{4.24}$$

La estimación es aproximada en ambos sentidos: no cuenta las posiciones con piezas capturadas o promovidas, que aumentan el total, ni descarta las posiciones ilegales, como las que tienen peones en la primera fila o ambos reyes en jaque, que lo disminuyen. En una descripción GDL del Ajedrez, además, el estado incluye fluents que no se ven en el tablero, como el turno, los derechos de enroque, la casilla de captura al paso y el contador de pasos que garantiza la terminación (Ejemplo 4.9); cada uno multiplica el número de estados distintos.

Para el número de partidas, Shannon razonó sobre el árbol de juego. Una partida típica dura unos 40 movimientos por bando, y en cada par de movimientos, uno de cada bando, hay del orden de 10^3 posibilidades, porque cada bando dispone en promedio de unas 30 jugadas legales y $30 \cdot 30 \approx 10^3$. El número de partidas de esa longitud es entonces del orden de

$$\left(10^3\right)^{40} = 10^{120}. \tag{4.25}$$

Esta cifra es una estimación del tamaño del árbol de juego, es decir, del número de historias, y no del número de estados. La relación entre ambas, 10^{120} frente a 10^{43}, es la misma que se observó en el Gato, 255 168 partidas frente a 5 478 estados, pero amplificada por muchos órdenes de magnitud. Para el motor de decisión las dos cifras tienen consecuencias distintas: el número de estados acota la memoria que haría falta para recordar todo el grafo de juego, y el número de partidas acota el trabajo de una búsqueda que recorriera el árbol completo. En el Ajedrez ambas son inalcanzables, y por eso la búsqueda debe limitarse en profundidad y apoyarse en evaluaciones heurísticas, como se discutió en el Capítulo 3.

4.1.4. Niveles de abstracción de una descripción

Una misma descripción atraviesa varios niveles de abstracción desde que se escribe hasta que el agente la usa para jugar, y en cada nivel vive en un conjunto distinto. Identificarlos con nombre propio es útil porque cada fase del intérprete se mueve exactamente entre dos de ellos, y cada error de un archivo de reglas puede ubicarse en uno de ellos.

En el nivel textual la descripción es una cadena $d \in \Sigma^*$ sobre un alfabeto de caracteres Σ: es el archivo tal como se guarda en disco. Aquí Σ^*, llamado *cerradura de Kleene* de Σ, es el conjunto de todas las secuencias finitas de caracteres de Σ, de cualquier longitud, incluida la secuencia vacía [44]; todo archivo de texto, tenga o no sentido, es un elemento de Σ^*. En el nivel léxico es una secuencia de *tokens* $\tau(d) \in \mathit{Tok}^*$, ya sin espacios ni comentarios, en la que cada unidad sabe si es un paréntesis, una variable o un identificador. En el nivel sintáctico abstracto es un programa lógico $D = \pi(\tau(d))$, un conjunto finito de reglas en el que ya no importa el orden de escritura ni la tipografía. En el nivel semántico es un juego $[\![D]\!] = \Gamma_D$, una tupla como la de la Ecuación (4.1). Las funciones que conectan los niveles son parciales: una cadena puede no admitir una segmentación en tokens válida, una secuencia de tokens puede no respetar la gramática y un programa puede no satisfacer las restricciones que garantizan que tiene significado. La Figura 4.3 resume este recorrido.

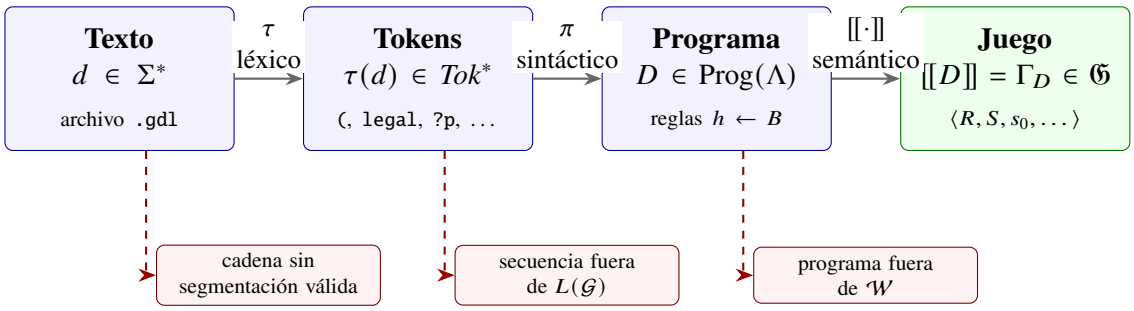


Figura 4.3: Niveles de abstracción de una descripción de juego. Cada función es parcial; los dominios de definición se van estrechando de izquierda a derecha hasta quedar únicamente las descripciones con significado.

Una descripción vive, por lo tanto, en varios conjuntos a la vez: en Σ^* como archivo, en Tok^* como secuencia léxica, en $\mathrm{Prog}(\Lambda)$ como programa lógico y, cuando todo sale bien, denota un elemento de la clase de juegos $\mathfrak{G}$. La Tabla 4.3 fija la notación.

Tabla 4.3: Conjuntos en que vive una descripción en cada nivel de abstracción.

Nivel	Conjunto	Qué contiene y qué se ignora
Textual	Σ^*	Todas las cadenas finitas de caracteres. Incluye espacios, saltos de línea, comentarios y cualquier texto sin sentido.
Léxico	Tok^*	Secuencias de unidades léxicas clasificadas. Se descartan espacios y comentarios.
Sintáctico	$L(\mathcal{G}) \subseteq Tok^*$	Secuencias que respetan la gramática. Garantiza paréntesis balanceados y forma de regla, no significado.
Abstracto	$\mathrm{Prog}(\Lambda)$	Conjuntos finitos de reglas sobre una signatura Λ. Se olvida el orden y la notación concreta.
Válido	$\mathcal{W} \subseteq \mathrm{Prog}(\Lambda)$	Programas que cumplen las restricciones estáticas que aseguran un significado único y finito.
Semántico	$\mathfrak{G}$	Juegos formales Γ. Es el único nivel que el motor de decisión necesita conocer.

4.1.5. Arquitectura del intérprete y flujo de procesamiento

La correspondencia entre los niveles de abstracción anteriores se materializa en el software mediante una arquitectura de interpretación por etapas. Cuando el sistema carga un juego, el archivo con extensión `.gdl` atraviesa un flujo de procesamiento secuencial que transforma una cadena de texto sin procesar en conocimiento formal operativo para el motor de búsqueda.

En la primera etapa, el analizador léxico o *lexer* examina el flujo de caracteres del archivo, eliminando de forma transparente los espacios en blanco redundantes, las tabulaciones, los saltos de línea y los comentarios, que en GDL inician con el carácter de punto y coma `;`. Su labor consiste en segmentar el texto continuo en unidades atómicas clasificadas denominadas fichas o *tokens*, tales como paréntesis de apertura y cierre, palabras reservadas del sistema, identificadores de constantes y predicados definidos por el autor, y variables que inician con el signo de interrogación. Si el documento contiene caracteres fuera del alfabeto admitido o una construcción léxica anómala, el análisis se interrumpe de inmediato emitiendo un diagnóstico preciso con la línea y columna del fallo.

A continuación, la secuencia lineal de fichas es recibida por el analizador sintáctico o *parser*. Su propósito fundamental es validar que la estructura del documento respete rigurosamente la gramática libre de contexto del lenguaje, asegurando que los paréntesis se encuentren perfectamente balanceados y que las listas anidadas sigan el formato de expresiones simbólicas prefijas característico de KIF y LISP. A partir de esta verificación, el parser construye una representación interna en memoria denominada árbol de sintaxis abstracta (*Abstract Syntax Tree* o AST). En este punto se descartan los detalles superficiales de la tipografía y el archivo queda formalizado como un conjunto de sentencias lógicas bien estructuradas, separando con claridad la cabeza de cada regla y los literales que integran su cuerpo.

La tercera etapa corresponde al validador de reglas, un módulo de análisis estático que examina el AST para garantizar que el programa denote un juego matemáticamente consistente y con un significado único antes de intentar simular cualquier partida. Este componente verifica que todas las reglas sean seguras —descartando variables libres o sueltas en las conclusiones que derivarían universos infinitos de hechos—, construye el grafo de dependencias entre relaciones para asegurar

que la negación esté estratificada y libre de ciclos paradójicos, y comprueba que las relaciones reservadas se empleen exclusivamente en las posiciones y con los tipos permitidos. Cualquier infracción en este paso genera un mensaje de error descriptivo que detiene la carga, protegiendo al agente de descripciones defectuosas o incompletas.

Finalmente, el programa validado queda disponible para el motor de inferencia y unificación, que actúa como el evaluador dinámico del sistema. Durante la partida, este componente resuelve en tiempo real las consultas que el motor de decisión formula sobre el estado de la partida. Al recibir la configuración del tablero actual codificada en hechos de `true` y las jugadas ejecutadas por los jugadores con `does`, el evaluador efectúa un proceso de unificación lógica y derivación por encadenamiento hacia adelante (*forward chaining*), contrastando los hechos conocidos contra los patrones de las reglas. De esta forma determina con absoluta precisión qué movimientos son legales en cada instante mediante `legal`, cómo evoluciona el estado del juego hacia el siguiente turno con `next`, en qué momento se alcanza el desenlace de la partida a través de `terminal` y cuál es la utilidad o puntaje obtenido por cada participante con `goal`.

4.2. Definición conceptual de un lenguaje de descripción de juegos

Un lenguaje de descripción de juegos puede entenderse conceptualmente como el puente formal que transforma un documento de texto plano en un juego interactivo ejecutable por una computadora. Esta perspectiva proporciona una plantilla general que permite comparar GDL con otros enfoques como Ludii o RBG, integrando tres tradiciones complementarias: la teoría de lenguajes formales para el análisis léxico y sintáctico [44], la semántica de la programación lógica para asignar un significado preciso e inequívoco a las reglas [45], [46], y los fundamentos de GGP para caracterizar las reglas del juego [10], [47].

Definición 4.5: Lenguaje de descripción de juegos

Conceptualmente, un **lenguaje de descripción de juegos** se formaliza mediante la tupla

$$\mathfrak{L} = \langle \Sigma, K, \tau, \mathcal{G}, \mathcal{W}, \mathfrak{G}, [\![\cdot]\!] \rangle \tag{4.26}$$

donde Σ es el alfabeto de caracteres permitidos; K es el conjunto finito de palabras reservadas cuyo significado es fijo y no modificable; τ es el analizador léxico que segmenta el texto en fichas; $\mathcal{G}$ es la gramática libre de contexto que establece la estructura de las sentencias; $\mathcal{W}$ es el conjunto de programas válidos que satisfacen las restricciones lógicas y semánticas; $\mathfrak{G}$ es la clase de juegos formales que se pretende modelar; y $[\![\cdot]\!]$ es la función semántica que asocia a cada descripción válida su correspondiente modelo formal Γ.

En términos prácticos, esta tupla describe el recorrido que realiza cualquier archivo de reglas: el texto sobre el alfabeto Σ se convierte en fichas mediante τ, se valida con la gramática $\mathcal{G}$ para formar un programa, se depura con las restricciones de $\mathcal{W}$ para descartar anomalías y, finalmente, la semántica $[\![\cdot]\!]$ produce el modelo formal Γ sobre el cual puede competir el motor de decisión.

La composición de estas etapas define la interpretación global del archivo:

$$\mathcal{I}_{\mathfrak{L}} = [\![\cdot]\!] \circ \pi \circ \tau \;:\; \Sigma^* \rightharpoonup \mathfrak{G}, \tag{4.27}$$

la cual tiene éxito únicamente cuando el documento es léxicamente admisible, sintácticamente correcto y semánticamente válido.

4.2.1. Propiedades fundamentales de un lenguaje de descripción

Para que un lenguaje de descripción resulte idóneo en el ámbito de la inteligencia artificial general, debe satisfacer cuatro propiedades conceptuales esenciales que garantizan su utilidad práctica. En primer lugar, la **corrección** (*soundness*) asegura que toda descripción válida admitida por el lenguaje denote sin excepción un juego formal legítimo y bien estructurado dentro de la clase objetivo. En segundo lugar, la **universalidad o completitud** exige que el lenguaje sea capaz de modelar cualquier juego perteneciente a dicha clase, admitiendo al menos una descripción válida para cada estructura de juego posible, módulo isomorfismo. En tercer lugar, la **efectividad computacional** garantiza que las consultas requeridas por el motor de decisión —tales como conocer el estado inicial, verificar la legalidad de una acción o calcular el siguiente estado— sean efectivamente computables en tiempo finito mediante algoritmos deterministas. Por último, la **concisión** (*succinctness*) permite especificar juegos con espacios de estados astronómicos mediante un conjunto compacto de reglas declarativas, evitando la necesidad de tabular configuraciones de manera explícita.

La efectividad es un requisito innegociable para este proyecto, pues un agente autónomo no puede competir en un entorno cuyas reglas no pueda evaluar de manera computable. Asimismo, la universalidad cobra sentido cuando se aclara bajo qué criterio dos juegos aparentemente distintos representan la misma experiencia estratégica. Cuando dos descripciones únicamente difieren en los nombres asignados a las casillas, a las piezas o a los roles, para efectos del razonamiento se trata del mismo juego; esta equivalencia se formaliza con precisión mediante la noción de isomorfismo restringido a la parte alcanzable.

Definición 4.6: Estados alcanzables

Dado un juego Γ, el conjunto de **estados alcanzables** $\mathrm{Reach}(\Gamma)$ es el menor conjunto $X \subseteq S$ que satisface

$$s_0 \in X \qquad \text{y} \qquad \big(s \in X \setminus F \;\wedge\; \mathbf{a} \in \mathbf{L}(s)\big) \implies T(s, \mathbf{a}) \in X. \tag{4.28}$$

Equivalentemente, $\mathrm{Reach}(\Gamma) = \bigcup_{k \geq 0} X_k$ con $X_0 = \{s_0\}$ y $X_{k+1} = X_k \cup \{T(s, \mathbf{a}) \mid s \in X_k \setminus F,\ \mathbf{a} \in \mathbf{L}(s)\}$.

La primera condición de (4.28) dice que el estado inicial es alcanzable. La segunda es una condición de cerradura: si un estado s ya es alcanzable, no es terminal ($s \notin F$) y $\mathbf{a}$ es una acción conjunta legal en él en el sentido de la Ecuación (3.16), entonces su sucesor $T(s, \mathbf{a})$, dado por la función de transición de la Ecuación (3.17), también es alcanzable. Hay muchos conjuntos que cumplen ambas condiciones (el propio S, por ejemplo); "el menor" es la intersección de todos ellos, y contiene exactamente los estados a los que se llega desde s_0 con un número finito de

jugadas legales. La formulación iterativa lo hace explícito: X_k es el conjunto de estados alcanzables con k transiciones o menos, y la unión $\bigcup_{k\geq 0} X_k$ reúne los de todas las longitudes. En términos de la distancia dirigida de la Definición 3.8,

$$\mathrm{Reach}(\Gamma) = \{\, s \in S \mid d(s_0, s) < \infty \,\}, \tag{4.29}$$

es decir, un estado es alcanzable si y solo si existe una trayectoria legal de longitud finita desde s_0 hasta él. Este es el conjunto que en la Ecuación (3.22) se denotó $\mathrm{Reach}(s_0)$.

Definición 4.7: Isomorfismo de juegos

Sean Γ y Γ' dos juegos con el mismo número n de roles. Un **isomorfismo** $\Gamma \cong \Gamma'$ es una terna $(\rho, h, \{\alpha_i\}_{i=1}^{n})$ formada por una biyección de roles $\rho : R \to R'$, una biyección de estados alcanzables $h : \mathrm{Reach}(\Gamma) \to \mathrm{Reach}(\Gamma')$ y, para cada rol, una biyección de acciones $\alpha_i : A_i \to A'_{\rho(i)}$, tales que para todo $s \in \mathrm{Reach}(\Gamma)$:

$$h(s_0) = s'_0, \tag{4.30}$$

$$\alpha_i\big(L_i(s)\big) = L'_{\rho(i)}\big(h(s)\big), \tag{4.31}$$

$$h\big(T(s, \mathbf{a})\big) = T'\big(h(s), \boldsymbol{\alpha}(\mathbf{a})\big) \quad \text{para } s \notin F,\ \mathbf{a} \in \mathbf{L}(s), \tag{4.32}$$

$$s \in F \iff h(s) \in F', \tag{4.33}$$

$$G_i(s) = G'_{\rho(i)}\big(h(s)\big) \quad \text{para } s \in F, \tag{4.34}$$

donde $\boldsymbol{\alpha}(\mathbf{a})$ es la acción conjunta de Γ' que se obtiene al traducir cada componente de $\mathbf{a} = (a_1, \ldots, a_n)$ y colocarla en la posición del rol correspondiente:

$$\boldsymbol{\alpha}(\mathbf{a}) = (a'_1, \ldots, a'_n), \qquad a'_{\rho(i)} = \alpha_i(a_i) \quad \text{para } i = 1, \ldots, n. \tag{4.35}$$

Una *biyección* es una función que asocia a cada elemento de su dominio un elemento distinto de su codominio y que cubre todo el codominio; por eso tiene inversa, y puede leerse como un "diccionario" que traduce nombres de un juego a otro sin perder ni confundir nada. Las tres biyecciones traducen, respectivamente, roles, estados y acciones. Las cinco condiciones exigen que la traducción respete cada componente de la tupla (4.1). La condición (4.30) dice que el estado inicial de Γ se traduce en el estado inicial de Γ'. En (4.31), la expresión $\alpha_i(L_i(s))$ es la *imagen* del conjunto $L_i(s)$ bajo α_i, es decir, $\{\alpha_i(a) \mid a \in L_i(s)\}$; la condición exige que, al traducir las acciones legales del rol r_i en s, se obtengan exactamente las acciones legales del rol correspondiente en el estado traducido. La condición (4.32) afirma que da igual jugar primero y traducir después, o traducir primero y jugar después: el sucesor traducido coincide con el sucesor del estado y la acción traducidos. La condición (4.33) exige que un estado sea terminal si y solo si lo es su traducción, y (4.34) que cada rol obtenga en los estados terminales la misma utilidad que su contraparte.

Cada condición preserva, así, una de las componentes de Γ: (4.30) el estado inicial, (4.31) la legalidad, (4.32) la dinámica, (4.33) la terminación y (4.34) las utilidades. Como un algoritmo de búsqueda general como Minimax solo accede al juego por medio de esas cinco consultas (Sección 3.4), dos juegos isomorfos producen árboles de búsqueda idénticos salvo por el nombre de sus nodos y aristas, y por lo tanto las mismas decisiones. Esa es la justificación de que la universalidad se exija solo módulo isomorfismo.

4.3. Sintaxis del *Game Description Language*

Con la Definición 4.5 a la mano, analizar GDL equivale a identificar sus siete componentes, que se denotan

$$\mathfrak{L}_{\text{GDL}} = \langle \Sigma_{\text{GDL}}, K_{\text{GDL}}, \tau_{\text{GDL}}, \mathcal{G}_{\text{GDL}}, \mathcal{W}_{\text{GDL}}, \mathfrak{G}_{\text{GDL}}, [\![\cdot]\!]_{\text{GDL}} \rangle \tag{4.36}$$

y cuyo subíndice se omite cuando no hay ambigüedad. Esta sección cubre las cuatro primeras componentes; la Sección 4.4 se dedica a $\mathcal{W}$ y la Sección 4.5 a $\mathfrak{G}$ y $[\![\cdot]\!]$. La fuente normativa es la especificación de Love et al. [10], complementada con el libro de Genesereth y Thielscher [9] y con la semántica formal de Schiffel y Thielscher [47].

4.3.1. Fundamentos sintácticos en KIF y en las expresiones simbólicas

La sintaxis de GDL no se inventó desde cero. Es un fragmento del *Knowledge Interchange Format* (KIF), un lenguaje desarrollado en Stanford para intercambiar conocimiento lógico entre sistemas [48], y KIF a su vez adopta la notación de *expresiones simbólicas* o S-expresiones de LISP [49]. Una S-expresión es o bien un símbolo atómico o bien una lista de S-expresiones entre paréntesis, con el primer elemento en posición de operador. Por eso en GDL no se escribe legal(x, noop) sino `(legal x noop)`: notación prefija, completamente parentizada y sin comas. La ventaja práctica es que el análisis léxico y sintáctico se vuelve trivialmente sencillo; la desventaja es la legibilidad para quien no está acostumbrado. La literatura posterior de GGP emplea también una notación infija al estilo de Prolog para las mismas reglas [9], lo que confirma que la notación concreta es accidental: lo esencial vive en el nivel abstracto de la Tabla 4.3.

4.3.2. Alfabeto y especificación léxica

El alfabeto de GDL es un conjunto de caracteres imprimibles con espacios en blanco; en este trabajo se fija Σ_{GDL} como los caracteres imprimibles de ASCII junto con el espacio, el tabulador y el salto de línea. Dentro de Σ distinguimos cuatro subconjuntos: el conjunto W de caracteres de espacio en blanco, los delimitadores $\{$`(`, `)`$\}$, el carácter de comentario `;` y el prefijo de variable `?`. Con ellos se define el conjunto de caracteres de símbolo

$$\Delta = \Sigma \setminus \big(W \cup \{\texttt{(}, \texttt{)}, \texttt{;}, \texttt{?}\}\big), \tag{4.37}$$

que contiene letras, dígitos y signos como <, =, - o _. La Ecuación (4.37) se lee así: de todos los caracteres del alfabeto Σ se retiran, mediante la diferencia de conjuntos $\setminus$, los que tienen una función especial (los espacios en blanco, que separan palabras; los paréntesis, que delimitan listas; el punto y coma, que inicia comentarios; y el signo de interrogación, que marca variables). Lo que queda, Δ, son los caracteres que pueden formar parte de un nombre. Las categorías léxicas se definen entonces mediante expresiones regulares [44], como resume la Tabla 4.4.

Las expresiones regulares de la tabla se leen de la siguiente manera. $Id = \Delta^{+}$ es el conjunto de cadenas formadas por uno o más caracteres de Δ: `cell` pertenece a Id porque sus cuatro caracteres están en Δ, mientras que `ce ll` no, porque contiene un espacio. $Var = \texttt{?} \cdot \Delta^{+}$ es el conjunto de cadenas que empiezan con `?` seguido de un identificador: el punto $\cdot$ indica que las dos partes se escriben una a continuación de la otra. Un comentario es un `;` seguido de cualquier cantidad de

Tabla 4.4: Categorías léxicas de GDL. La notación X^+ indica una o más repeticiones de elementos de X; $\cdot$ denota concatenación.

Categoría	Expresión regular	Ejemplos y comentario
Apertura	`(`	Inicia una lista.
Cierre	`)`	Cierra la lista abierta más reciente.
Identificador	$Id = \Delta^+$	`cell`, `x`, `3`, `board_full`, `<=`. Los números son identificadores como cualquier otro.
Variable	$Var = \texttt{?} \cdot \Delta^+$	`?p`, `?r`, `?mark`. Su alcance es la regla en que aparecen.
Comentario	$\texttt{;} \cdot (\Sigma \setminus \{\backslash n\})^*$	Se descarta; no produce token.
Espacio	W^+	Se descarta; solo separa tokens.

caracteres distintos del salto de línea $\backslash n$, es decir, todo lo que resta de la línea. Un espacio es una secuencia no vacía de caracteres de W.

El conjunto de tokens es $Tok = \{(,)\} \cup Id \cup Var$: un token es un paréntesis, un identificador o una variable. El analizador léxico τ recorre la cadena aplicando la regla de la coincidencia más larga (*maximal munch*): en cada posición consume el prefijo más largo que pertenezca a alguna categoría, descarta los espacios y comentarios y emite el resto como tokens. Por ejemplo,

$$\tau\big(\texttt{(<= (legal ?p noop) ; turno ajeno}\backslash n \quad \texttt{(role ?p))}\big)$$
$$= \texttt{(<= (legal ?p noop) (role ?p))},$$

una secuencia de doce tokens. El comentario y los espacios han desaparecido. La función τ es parcial porque hay cadenas que no admiten segmentación: un `?` aislado seguido de espacio no pertenece a ninguna categoría.

Un detalle léxico tiene consecuencias semánticas importantes: GDL no tiene números. Los símbolos `1`, `2`, `100` son identificadores sin más significado que el de ser distintos entre sí. No existe una operación de suma incorporada; si un juego necesita contar, debe definir la aritmética que use, típicamente mediante hechos de sucesor como `(succ 0 1)`. La única excepción parcial es el segundo argumento de `goal`, que la especificación interpreta como un número entre 0 y 100 [10].

4.3.3. Signatura y sintaxis de términos, átomos y reglas

Sobre los tokens se construye el vocabulario lógico. En lógica de primer orden, el vocabulario de un lenguaje se llama *signatura* [45].

Definición 4.8: Signatura de una descripción GDL

Una **signatura** es una tupla $\Lambda = \langle C, \Phi, \Pi, \mathrm{ar}\rangle$ donde $C \subseteq Id$ es un conjunto finito de **constantes de objeto**, $\Phi \subseteq Id$ es un conjunto finito de **símbolos de función**, $\Pi \subseteq Id$ es un conjunto finito de **símbolos de relación**, los tres disjuntos dos a dos, y $\mathrm{ar} : \Phi \cup \Pi \to \mathbb{N}$ asigna a cada símbolo su aridad, con $\mathrm{ar}(\varphi) \geq 1$ para todo $\varphi \in \Phi$. El conjunto de variables $\mathcal{V} = Var$ es numerable y disjunto de los anteriores.

La signatura formaliza el vocabulario que en la Sección 3.5.2 se describió informalmente. Allí se llamó *functor* al identificador inicial de una expresión y *aridad* a su número de argumentos;

aquí los functores se reparten en tres clases según su papel. Las constantes de C nombran objetos concretos (`x`, `3`); los símbolos de Φ construyen objetos compuestos a partir de otros (`cell` construye una casilla a partir de dos coordenadas y una marca); y los símbolos de Π expresan afirmaciones que pueden ser verdaderas o falsas (`legal`, `line`). Que los tres conjuntos sean "disjuntos dos a dos" significa $C \cap \Phi = C \cap \Pi = \Phi \cap \Pi = \emptyset$: ningún símbolo pertenece a dos clases. La función ar asocia a cada símbolo de función o de relación su aridad, un número natural; las constantes no necesitan aridad porque no reciben argumentos. Se exige $\mathrm{ar}(\varphi) \geq 1$ a las funciones porque una función sin argumentos sería simplemente una constante. Por último, que $\mathcal{V}$ sea numerable significa que sus elementos pueden enumerarse como $v_1, v_2, v_3, \ldots$, de modo que nunca se agotan los nombres de variable disponibles.

Definición 4.9: Términos, átomos y literales

Los **términos** sobre Λ forman el menor conjunto $\mathrm{Term}(\Lambda, \mathcal{V})$ tal que (i) toda variable $v \in \mathcal{V}$ es un término; (ii) toda constante $c \in C$ es un término; y (iii) si $\varphi \in \Phi$ con $\mathrm{ar}(\varphi) = k$ y $t_1, \ldots, t_k$ son términos, entonces $\varphi(t_1, \ldots, t_k)$ es un término. Un **átomo** es una expresión $p(t_1, \ldots, t_k)$ con $p \in \Pi$, $\mathrm{ar}(p) = k$ y t_j términos; si $k = 0$ se escribe simplemente p. Un **literal** es un átomo α (literal positivo), su negación $\neg\alpha$ (literal negativo) o una desigualdad $t \neq u$ entre términos. Un término, átomo o literal es **básico** (*ground*) si no contiene variables.

La definición de término es *inductiva*: las cláusulas (i) y (ii) dan los términos más simples, y la cláusula (iii) explica cómo construir términos nuevos a partir de otros ya construidos. "El menor conjunto" significa que no hay más términos que los que se obtienen aplicando esas tres cláusulas un número finito de veces. Por ejemplo, `2` y `x` son términos por (ii); `?c` es término por (i); y como $\mathrm{ar}($`cell`$) = 3$, la cláusula (iii) produce el término `cell(2, ?c, x)`. Este término no es básico, porque contiene la variable `?c`; al reemplazarla por `2` se obtiene el término básico `cell(2, 2, x)`, que es exactamente uno de los fluents de la Sección 3.4.2. Un átomo, en cambio, aplica un símbolo de relación a términos y produce una afirmación: `true(cell(2, 2, x))` afirma que ese fluent vale en el estado actual. Un literal es un átomo afirmado, negado o una comparación de desigualdad; son las piezas con las que se escriben las condiciones de las reglas.

Definición 4.10: Regla y descripción

Una **regla** es una expresión

$$h \leftarrow \ell_1 \wedge \ell_2 \wedge \cdots \wedge \ell_k, \qquad k \geq 0, \tag{4.38}$$

donde la **cabeza** h es un átomo y el **cuerpo** $\ell_1, \ldots, \ell_k$ es una secuencia de literales. Si $k = 0$ y h es básico, la regla es un **hecho**. Se denotan por $B^+(\rho)$, $B^-(\rho)$ y $B^{\neq}(\rho)$ los átomos de los literales positivos, los átomos de los literales negativos y las desigualdades del cuerpo de una regla ρ. Una **descripción** o programa sobre Λ es un conjunto finito D de reglas; $\mathrm{Prog}(\Lambda)$ es el conjunto de todos ellos.

La flecha $\leftarrow$ de la Ecuación (4.38) se lee "si" y el símbolo $\wedge$ es la conjunción lógica "y": la regla afirma que la cabeza h es verdadera si los literales $\ell_1, \ldots, \ell_k$ son todos verdaderos a la vez. Es la forma matemática de la regla de inferencia de la Sección 3.5.2 y del Código 3.1. Cuando

$k = 0$ el cuerpo es vacío y no impone ninguna condición, de modo que la cabeza es verdadera sin más: eso es un hecho, como `(role x)`. La notación B^+, B^- y $B^{\neq}$ separa el cuerpo según el tipo de literal. Por ejemplo, para la regla C del Código 3.4,

$$\texttt{next}(\texttt{cell}(\texttt{?r},\texttt{?c},\texttt{b})) \leftarrow \texttt{true}(\texttt{cell}(\texttt{?r},\texttt{?c},\texttt{b})) \\ \wedge \neg\texttt{does}(\texttt{x},\texttt{mark}(\texttt{?r},\texttt{?c})) \wedge \neg\texttt{does}(\texttt{o},\texttt{mark}(\texttt{?r},\texttt{?c})),$$

se tiene $B^+ = \{\texttt{true}(\texttt{cell}(\texttt{?r},\texttt{?c},\texttt{b}))\}$, $B^- = \{\texttt{does}(\texttt{x},\texttt{mark}(\texttt{?r},\texttt{?c})),\ \texttt{does}(\texttt{o},\texttt{mark}(\texttt{?r},\texttt{?c}))\}$ y $B^{\neq} = \emptyset$. Esta separación será necesaria para definir la seguridad de las reglas (Definición 4.14) y el cálculo del modelo estándar (Definición 4.19).

En la sintaxis concreta, la regla (4.38) se escribe `(<= h` ℓ_1 `...` ℓ_k`)`, un literal negativo como `(not` α`)` y una desigualdad como `(distinct t u)`. Las reglas son exactamente las cláusulas de un programa lógico normal [45]; de ahí que GDL herede la teoría de modelos de la programación lógica y de Datalog [50], [51].

Reificación de fluents y acciones como términos

La Definición 4.9 aclara un punto que en la presentación informal del Capítulo 3 quedaba difuso. En la regla `(<= (legal ?p (mark ?r ?c)) (true (control ?p)) (true (cell ?r ?c b)))`, el símbolo `true` es una relación de aridad 1, pero `cell` *no* es una relación: es un símbolo de función de aridad 3, y `(cell ?r ?c b)` es un término, el argumento de `true`. Lo mismo ocurre con `mark`, que es una función de aridad 2 usada como argumento de `legal` y de `does`, y con `control`. Los fluents y las acciones de GDL son, por lo tanto, *términos*, objetos del dominio sobre los que se predica, y no proposiciones. Esta técnica se llama *reificación*: se convierte una proposición en un objeto para poder hablar de ella. Gracias a la reificación, un estado puede definirse como un conjunto de términos básicos y no como una interpretación lógica completa.

Ejemplo 4.1: Signatura de la descripción del Gato

Las sentencias de los Códigos 3.2 a 3.5 inducen la signatura $\Lambda_{\text{Gato}} = \langle C, \Phi, \Pi, \text{ar}\rangle$ con

$$C = \{\texttt{x},\texttt{o},\texttt{b},\texttt{1},\texttt{2},\texttt{3},\texttt{noop},\texttt{0},\texttt{50},\texttt{100}\},$$
$$\Phi = \{\texttt{cell}/3,\ \texttt{control}/1,\ \texttt{mark}/2\},$$
$$\Pi = \underbrace{\{\texttt{role}/1,\texttt{init}/1,\texttt{true}/1,\texttt{does}/2,\texttt{legal}/2,\texttt{next}/1,\texttt{terminal}/0,\texttt{goal}/2\}}_{\text{reservadas}} \\ \cup\ \underbrace{\{\texttt{line}/1,\texttt{open}/0,\texttt{board_full}/0\}}_{\text{definidas por el autor}},$$

donde s/k abrevia "símbolo s de aridad k". Las constantes `0`, `50` y `100` aparecen como valores de meta y, para el lenguaje, son constantes de objeto como `x` u `o`. La relación `distinct` no figura en Π porque no es una relación del usuario: es la desigualdad $\neq$, una construcción lógica con significado fijo.

4.3.4. Universo y base de Herbrand

Para comprender el universo y la base de Herbrand de forma intuitiva, conviene imaginarlos como el diccionario de sustantivos y el catálogo de oraciones del juego. En la semántica de la programación lógica, una descripción habla de sus propios términos sin suponer ningún dominio externo [45]. El **universo de Herbrand** reúne todos los sustantivos y objetos con los que el juego puede operar (los jugadores, las coordenadas, los símbolos de las piezas), mientras que la **base de Herbrand** agrupa todas las afirmaciones o hechos atómicos concebibles que pueden construirse combinando esos objetos con las relaciones del juego (por ejemplo, afirmar que «la casilla $(1, 1)$ contiene una marca `x`»). Una **interpretación** es simplemente la decisión sobre cuáles de esas afirmaciones se consideran verdaderas en un momento dado de la partida y cuáles falsas.

Definición 4.11: Universo y base de Herbrand

El **universo de Herbrand** U_Λ es el conjunto de todos los términos básicos construibles con C y Φ. La **base de Herbrand** B_Λ es el conjunto de todos los átomos básicos $p(t_1, \ldots, t_k)$ con $p \in \Pi$ y $t_j \in U_\Lambda$. Una **interpretación de Herbrand** es cualquier subconjunto $I \subseteq B_\Lambda$: los átomos de I son verdaderos y todos los demás son falsos.

El universo U_Λ reúne todos los objetos de los que la descripción puede hablar, y la base B_Λ reúne todas las afirmaciones básicas que pueden formularse sobre ellos; cada elemento de B_Λ es una afirmación que, en principio, puede ser verdadera o falsa. Una interpretación es una elección de cuáles de esas afirmaciones se consideran verdaderas. La última cláusula es la hipótesis de mundo cerrado de Reiter [35], que en la Definición 3.6 permitió representar un estado como el conjunto de sus fluents verdaderos, vista ahora desde la teoría de modelos: lo que no está en la interpretación es falso. Con estas definiciones se pueden ubicar con exactitud los objetos de Γ. Un fluent es un elemento de U_Λ; un estado es un subconjunto finito de U_Λ; una acción es también un elemento de U_Λ; un rol es una constante de C. En símbolos,

$$R \subseteq C, \qquad \mathcal{F} \subseteq U_\Lambda, \qquad S \subseteq \mathcal{P}_{\text{fin}}(U_\Lambda), \qquad A_i \subseteq U_\Lambda, \tag{4.39}$$

donde $\mathcal{P}_{\text{fin}}(X)$ es el conjunto de subconjuntos finitos de X. Las inclusiones (4.39) ubican cada componente del juego cuando este se describe en GDL. La primera dice que los roles son constantes, como `x` y `o`. La segunda, que los fluents de la Definición 3.5 son términos básicos, como `cell(1,1,b)`. La tercera precisa la inclusión $S \subseteq \mathcal{P}(\mathcal{F})$ de la Definición 3.6: un estado es un conjunto *finito* de términos básicos, condición necesaria para que pueda almacenarse y compararse en un programa. La cuarta dice que las acciones de A_i también son términos básicos, como `mark(2,2)` o `noop`.

Ejemplo 4.2: El universo de Herbrand del Gato es infinito

Como Φ_{Gato} contiene símbolos de función, los términos pueden anidarse sin límite. Además de los términos con sentido, como `cell(2,2,x)` o `mark(1,3)`, el universo contiene términos como `control(control(x))` o `cell(mark(1,1),noop,b)`. Por eso $|U_{\Lambda_{\text{Gato}}}| = \aleph_0$: el símbolo $\aleph_0$ (álef cero) es el cardinal de los números naturales, y la igualdad indica que el universo es infinito pero sus elementos pueden enumerarse uno tras otro, ordenándolos, por ejemplo, por su longitud. De ese universo infinito, solo 29 términos funcionan como fluents en alguna

partida y solo 10 como acciones de cada rol. Separar los términos con sentido de los demás es justamente lo que hacen las palabras reservadas `base` e `input` de la Sección 4.3.5, y lo que garantizan, de forma implícita, las restricciones de la Sección 4.4.

4.3.5. Conjunto de palabras reservadas

Las palabras reservadas son el puente entre la sintaxis y el modelo Γ. Un autor puede inventar los símbolos `cell`, `line` o `peon` con total libertad, porque el motor no les atribuye ningún significado; pero los símbolos reservados tienen una interpretación fija, la misma en todas las descripciones, que los conecta con componentes específicas de la tupla. Formalmente, K_{GDL} es un subconjunto finito de los identificadores, y se descompone en tres partes disjuntas según su función.

Definición 4.12: Palabras reservadas de GDL

El conjunto de palabras reservadas de GDL es

$$K_{\text{GDL}} = K_{\Gamma} \uplus K_{\text{dom}} \uplus K_{\text{log}} \subset Id, \tag{4.40}$$

donde $\uplus$ denota unión disjunta y:

$$K_{\Gamma} = \{\texttt{role}/1,\ \texttt{init}/1,\ \texttt{true}/1,\ \texttt{does}/2,\ \texttt{legal}/2,\ \texttt{next}/1,\ \texttt{terminal}/0,\ \texttt{goal}/2\}, \tag{4.41}$$

$$K_{\text{dom}} = \{\texttt{base}/1,\ \texttt{input}/2\}, \tag{4.42}$$

$$K_{\text{log}} = \{\texttt{<=},\ \texttt{not},\ \texttt{distinct}/2,\ \texttt{or}\}. \tag{4.43}$$

K_{Γ} son las **relaciones del juego**, cuyo significado es una componente de Γ [10]; K_{dom} son las **relaciones de dominio**, que declaran el universo de fluents y de acciones [9]; y K_{log} son las **conectivas lógicas**: implicación, negación por fallo, desigualdad sintáctica y disyunción. Toda descripción sobre una signatura Λ debe respetar la condición de no redefinición

$$(C \cup \Phi) \cap K_{\text{GDL}} = \emptyset \qquad \text{y} \qquad \Pi \cap K_{\text{log}} = \emptyset, \tag{4.44}$$

es decir, una palabra reservada nunca puede usarse como constante ni como función, y las conectivas no pueden usarse como relaciones.

En la Ecuación (4.40), el símbolo $\uplus$ denota una unión de conjuntos que, además, son disjuntos: ninguna palabra pertenece a dos de las tres partes. La notación s/k en (4.41) y (4.42) indica que el símbolo s tiene aridad k; así, `legal`$/2$ es una relación de dos argumentos, el rol y la acción, y `terminal`$/0$ una proposición sin argumentos. Las conectivas de (4.43) no llevan aridad fija, salvo `distinct`, porque no son relaciones sino operadores que construyen reglas y literales. El conjunto K_{Γ} coincide con los ocho predicados reservados de la Tabla 3.1. En la Ecuación (4.44), la intersección vacía $\cap \ldots = \emptyset$ expresa que ningún elemento es común a los dos conjuntos: no hay ninguna constante ni función que sea a la vez una palabra reservada, ni ninguna relación que sea una conectiva.

Las palabras reservadas viven, así, en $Id = \Delta^+$, el mismo conjunto de cadenas que los identificadores del usuario, y lo que las distingue no es su forma léxica sino el lugar que la definición del lenguaje les asigna. El analizador léxico no las separa; es el análisis semántico el que, al encontrar `legal` en posición de relación, sabe que esa regla contribuye a L_i. La Tabla 4.5 recoge el significado de cada una.

Tabla 4.5: Palabras reservadas de GDL, su aridad, su significado formal y la componente de Γ que determinan. La notación M se refiere al modelo estándar de la Sección 4.5.

Palabra	**Ar.**	**Lectura de (`p` t_1 `...`)**	**Componente de Γ**
`role`	1	t_1 es un participante.	$R = \{r \mid \texttt{role}(r) \in M(D)\}$
`init`	1	El fluent t_1 es verdadero al inicio.	$s_0 = \{f \mid \texttt{init}(f) \in M(D)\}$
`true`	1	El fluent t_1 es verdadero en el estado actual.	Entrada: el estado s consultado.
`does`	2	El rol t_1 ejecuta la acción t_2 en este paso.	Entrada: la acción conjunta $\mathbf{a}$.
`legal`	2	El rol t_1 puede ejecutar t_2 en el estado actual.	$L_i(s)$
`next`	1	El fluent t_1 será verdadero en el estado siguiente.	$T(s, \mathbf{a})$
`terminal`	0	El estado actual es terminal.	F
`goal`	2	El rol t_1 obtiene utilidad t_2 en el estado actual.	$G_i(s)$
`base`	1	t_1 es un fluent posible.	Universo $\mathcal{F}$
`input`	2	t_2 es una acción posible del rol t_1.	Catálogo A_i
`<=`	–	Implicación de cuerpo a cabeza.	Estructura de regla
`not`	1	Negación por fallo bajo mundo cerrado.	Literal negativo
`distinct`	2	Los términos t_1 y t_2 son sintácticamente distintos.	Desigualdad $t_1 \neq t_2$
`or`	≥ 1	Al menos uno de los literales se cumple.	Disyunción en el cuerpo

Relaciones reservadas de entrada, de salida y estáticas

Una manera directa y sumamente intuitiva de comprender las ocho relaciones reservadas del juego consiste en observar el ciclo de vida operativo de una partida o *game loop*. Al arrancar el sistema, es indispensable conocer quiénes son los participantes (`role`) y cómo se encuentra configurado el tablero en el instante inicial (`init`). Posteriormente, en el transcurso de cada turno, el motor de juego le informa al evaluador cuáles son los hechos verdaderos en ese momento mediante `true`, preguntándole de inmediato qué movimientos tiene permitidos cada participante a través de `legal`. Una vez que los jugadores eligen sus decisiones, el entorno comunica esas elecciones usando `does`, lo que habilita a las reglas de `next` para deducir con exactitud cómo se transformará el escenario hacia el turno siguiente. Finalmente, tras cada jugada se comprueba si la partida ha alcanzado su desenlace con `terminal` y, de ser así, qué puntuación o utilidad le corresponde a cada quien mediante `goal`.

Esta visión secuencial aclara la asimetría fundamental que refleja la Tabla 4.5. Las relaciones

`true` y `does` nunca se *definen* en la descripción: ninguna regla puede colocarlas en la cabeza. Son relaciones de **entrada**, cuyos hechos los suministra el motor desde fuera para formular una consulta: "supón que el estado es s y que los jugadores hicieron **a**". Las relaciones `legal`, `next`, `terminal` y `goal`, en cambio, son relaciones de **salida**: la descripción las define y el motor lee su valor. Las relaciones `role`, `init`, `base` e `input` son **estáticas**: se calculan una sola vez, sin estado ni acciones, al cargar el juego. Visto así, una descripción GDL funciona como una función que recibe un estado (y, en su caso, una acción conjunta) codificado en hechos de `true` y `does` y devuelve hechos de `legal`, `next`, `terminal` y `goal`. La Definición 4.21 de la Sección 4.5 convierte esta intuición en una definición.

Declaración de dominios mediante `base` e `input`

La especificación original de 2008 no incluía forma de declarar el universo de fluents ni el catálogo de acciones; ambos quedaban implícitos en lo que las reglas permitieran derivar. Las relaciones `base` e `input`, incorporadas en la versión del lenguaje que presentan Genesereth y Thielscher [9], permiten declararlos de manera explícita, lo que facilita al intérprete construir estructuras de datos de tamaño fijo, como las redes proposicionales mencionadas en el Capítulo 3. El Código 4.1 muestra cómo se agregan a la descripción del Gato.

```
;; Dominios auxiliares (relaciones definidas por el autor)
(index 1) (index 2) (index 3)
(filler x) (filler o) (filler b)

;; Universo de fluents: 27 celdas + 2 de control = 29
(<= (base (cell ?r ?c ?m)) (index ?r) (index ?c) (filler ?m))
(<= (base (control ?p)) (role ?p))

;; Catalogo de acciones: 9 marcas + noop = 10 por rol
(<= (input ?p (mark ?r ?c)) (role ?p) (index ?r) (index ?c))
(<= (input ?p noop) (role ?p))
```

Código 4.1: Declaración del universo de fluents y del catálogo de acciones del Gato mediante `base` e `input`.

Con estas reglas el modelo estático contiene exactamente 29 hechos de `base` y 20 hechos de `input`. El primer número resulta de la regla de `base` para `cell`: la variable `?r` toma 3 valores (los hechos de `index`), `?c` otros 3 y `?m` otros 3 (los hechos de `filler`), lo que da $3 \cdot 3 \cdot 3 = 27$ casillas posibles, a las que se suman los 2 fluents de control, uno por rol; en total, $27 + 2 = 29$. El segundo resulta de las reglas de `input`: para cada uno de los 2 roles hay $3 \cdot 3 = 9$ acciones de marcado más la acción `noop`, es decir, $2 \cdot (9 + 1) = 20$. Ambos valores coinciden con el universo $\mathcal{F}$ de la Sección 3.4.2 y con $|A_{\mathtt{x}}| = |A_{\mathtt{o}}| = 10$.

Extensiones del conjunto reservado en los dialectos de GDL

El conjunto K es lo que más cambia entre las variantes de la familia GDL, y cada ampliación corresponde a una ampliación de la clase de juegos $\mathfrak{G}$. GDL-II agrega la relación `sees`, que define lo que cada rol percibe tras cada paso, y la constante reservada `random`, un rol especial que representa al azar y elige sus acciones con distribución uniforme [26], [52]; con ello la clase objetivo pasa a incluir juegos con información imperfecta y elementos aleatorios. GDL-III agrega

la relación `knows`, que permite que las reglas dependan del conocimiento de los jugadores, y se dirige a juegos epistémicos [53]. La Tabla 4.6 resume la relación entre palabras reservadas y clase de juegos.

Tabla 4.6: Palabras reservadas añadidas por cada dialecto de la familia GDL y clase de juegos que habilitan.

Dialecto	**Conjunto reservado**	**Clase objetivo $\mathfrak{G}$**
GDL [10]	$K_\Gamma \uplus K_{\log}$	Finitos, deterministas, síncronos, de información perfecta.
GDL con dominio [9]	$K_\Gamma \uplus K_{\text{dom}} \uplus K_{\log}$	La misma clase, con $\mathcal{F}$ y A_i declarados.
GDL-II [26]	$K_{\text{GDL}} \cup \{\texttt{sees}/2, \texttt{random}\}$	Agrega información imperfecta y azar.
GDL-III [53]	$K_{\text{GDL-II}} \cup \{\texttt{knows}\}$	Agrega reglas que dependen del conocimiento de los jugadores.

Para el alcance de este Trabajo Terminal (juegos finitos, deterministas, por turnos y de información perfecta) basta con $K_\Gamma \uplus K_{\text{dom}} \uplus K_{\log}$. Las palabras `sees`, `random` y `knows` quedan fuera, y su aparición en un archivo deberá producir un diagnóstico explícito en lugar de interpretarse como un símbolo cualquiera del usuario.

4.3.6. Gramática libre de contexto de GDL

Las reglas de formación de GDL se expresan con una gramática libre de contexto en notación de Backus–Naur extendida [44]. La Tabla 4.7 presenta $\mathcal{G}_{\text{GDL}}$ en forma normalizada, basada en la gramática de la especificación [10]. Los símbolos entre ángulos son no terminales; lo escrito en tipografía monoespaciada son tokens; el símbolo ::= se lee “se define como”; la barra | separa alternativas; la estrella * indica cero o más repeticiones y la cruz $^+$, una o más.

Cada producción de la Tabla 4.7 tiene una lectura directa. Una descripción es una sucesión de sentencias, posiblemente vacía. Una sentencia es un átomo suelto (un hecho) o una lista que empieza con <=, sigue con un átomo (la cabeza) y termina con cero o más literales (el cuerpo). Un literal es un átomo, un átomo negado con `not`, una desigualdad `distinct` entre dos términos o una disyunción `or` de uno o más literales. Un átomo es un identificador solo, para las relaciones de aridad cero como `terminal`, o una lista que empieza con un identificador y sigue con uno o más términos. Un término, por último, es una variable, un identificador o una lista con la misma forma que un átomo. El conjunto $Id^\circ = Id \setminus K_{\log}$ excluye las conectivas de las posiciones de nombre, para que `not` o <= no puedan confundirse con una relación del usuario.

Ejemplo 4.3: Derivación de una regla del Gato

La regla de legalidad de la acción nula se obtiene de $\langle \textit{sentencia} \rangle$ mediante la derivación por

Tabla 4.7: Gramática $\mathcal{G}_{\text{GDL}}$ en notación EBNF. Las categorías *Id* y *Var* son las de la Tabla 4.4; $Id^\circ = Id \setminus K_{\log}$.

⟨*desc*⟩	::= ⟨*sentencia*⟩*	descripción
⟨*sentencia*⟩	::= ⟨*átomo*⟩ | `(` `<=` ⟨*átomo*⟩ ⟨*literal*⟩* `)`	hecho o regla
⟨*literal*⟩	::= ⟨*átomo*⟩ | `(` `not` ⟨*átomo*⟩ `)`	positivo o negativo
	| `(` `distinct` ⟨*término*⟩ ⟨*término*⟩ `)`	desigualdad
	| `(` `or` ⟨*literal*⟩$^+$ `)`	disyunción
⟨*átomo*⟩	::= Id° | `(` Id° ⟨*término*⟩$^+$ `)`	relación y argumentos
⟨*término*⟩	::= *Var* | Id° | `(` Id° ⟨*término*⟩$^+$ `)`	variable, constante o función

la izquierda

⟨*sentencia*⟩ ⇒ `(<=` ⟨*átomo*⟩ ⟨*literal*⟩ ⟨*literal*⟩ `)`
⇒ `(<=` `(legal` ⟨*término*⟩ ⟨*término*⟩`)` ⟨*literal*⟩ ⟨*literal*⟩ `)`
⇒* `(<=` `(legal ?p noop)` ⟨*literal*⟩ ⟨*literal*⟩ `)`
⇒* `(<=` `(legal ?p noop)` `(role ?p)` ⟨*literal*⟩ `)`
⇒ `(<=` `(legal ?p noop)` `(role ?p)` `(not` ⟨*átomo*⟩`)` `)`
⇒* `(<= (legal ?p noop) (role ?p) (not (true (control ?p))))`,

donde cada paso ⇒ reemplaza un no terminal por el lado derecho de una de sus producciones, y ⇒* abrevia varios pasos seguidos. La derivación es "por la izquierda" porque en cada paso se reemplaza el no terminal situado más a la izquierda. Que la cadena final se haya obtenido de ⟨*sentencia*⟩ prueba que pertenece al lenguaje de la gramática. La misma gramática deriva también la cadena `(<= (legal ?p noop) (not (true (control ?p))))`, que omite `(role ?p)`. Ambas son sintácticamente correctas; en la Sección 4.4 se verá que solo la primera es válida.

Restricciones contextuales y sintaxis abstracta

La gramática de la Tabla 4.7 tiene una limitación deliberada: la producción de ⟨*átomo*⟩ y la de ⟨*término*⟩ compuesto son idénticas en forma. La cadena `(cell 1 1 b)` es un átomo si aparece directamente en el cuerpo de una regla y un término si aparece como argumento de `true`. La gramática no sabe, por lo tanto, si `cell` es una relación o una función; tampoco puede comprobar que un símbolo se use siempre con la misma aridad ni que una variable de la cabeza aparezca en el cuerpo. Estas son propiedades *contextuales*: dependen de otras partes de la descripción y no pueden expresarse con una gramática libre de contexto. Por eso la Definición 4.5 separa $L(\mathcal{G})$ de $\mathcal{W}$.

La función π que lleva una secuencia gramatical a un programa abstracto resuelve la primera de estas cuestiones por posición: todo identificador que encabeza un átomo se registra en Π, todo identificador que encabeza un término compuesto se registra en Φ y todo identificador en posición

de término simple se registra en C. Además, π normaliza las disyunciones: una regla con un literal `(or` ℓ_1 `...` ℓ_m`)` en su cuerpo se reemplaza por m reglas, una por cada alternativa, lo cual preserva el significado porque una cabeza se deriva si y solo si alguna de las alternativas permite derivarla. Tras esta normalización, todo programa de $\mathrm{Prog}(\Lambda)$ tiene la forma de la Definición 4.10.

Definición 4.13: Consistencia de la signatura

Un programa D tiene **signatura consistente** si la signatura inferida por π cumple las condiciones de la Definición 4.8: los conjuntos C, Φ y Π son disjuntos, cada símbolo de $\Phi \cup \Pi$ se usa siempre con la misma aridad, toda palabra reservada de $K_\Gamma \cup K_{\mathrm{dom}}$ se usa con la aridad de la Ecuación (4.41) o (4.42) y se cumple la condición de no redefinición (4.44).

4.4. Condiciones de validez de una descripción GDL

Una descripción gramatical puede carecer de significado por tres razones distintas: porque alguna regla deriva un número infinito de hechos, porque las negaciones se contradicen circularmente o porque una palabra reservada se usa de un modo incompatible con su papel en Γ. La especificación de GDL impone una restricción para cada problema [9], [10]. En esta sección se definen con precisión y se explica qué falla cuando no se cumplen.

4.4.1. Seguridad de las reglas

Definición 4.14: Regla segura

Sea $\mathrm{vars}(X)$ el conjunto de variables que aparecen en una expresión X. Una regla $\rho = (h \leftarrow \ell_1 \wedge \cdots \wedge \ell_k)$ es **segura** si toda variable que aparece en la cabeza, en un literal negativo o en una desigualdad aparece también en algún literal positivo del cuerpo:

$$\mathrm{vars}(h) \;\cup \bigcup_{\alpha \in B^-(\rho)} \mathrm{vars}(\alpha) \;\cup \bigcup_{(t \neq u) \in B^{\neq}(\rho)} \mathrm{vars}(t,u) \;\subseteq \bigcup_{\beta \in B^+(\rho)} \mathrm{vars}(\beta). \tag{4.45}$$

La Ecuación (4.45) compara dos conjuntos de variables. Del lado izquierdo, la unión $\cup$ reúne las variables de la cabeza, las de todos los átomos negados (la gran unión $\bigcup_{\alpha \in B^-(\rho)}$ recorre cada átomo α del cuerpo negativo y junta sus variables) y las de todas las desigualdades. Del lado derecho se reúnen las variables de los átomos positivos del cuerpo. La inclusión $\subseteq$ exige que todo elemento del lado izquierdo esté también en el lado derecho. Aplicada a la regla de `noop` del Código 3.3, `(<= (legal ?p noop) (role ?p) (not (true (control ?p))))`, el lado izquierdo es $\{$`?p`$\} \cup \{$`?p`$\} = \{$`?p`$\}$ y el derecho es $\{$`?p`$\}$, aportado por `(role ?p)`; la inclusión se cumple y la regla es segura.

La intuición es que los literales positivos son los únicos que *generan* valores para las variables: para satisfacer `(true (control ?p))`, la variable `?p` debe tomar el valor de algún hecho existente, y los hechos existentes son finitos. Un literal negativo, en cambio, solo *filtra*: `(not (true (control ?p)))` es verdadero para infinitos valores de `?p`, todos los términos del universo de Herbrand que no tienen el control. Si una variable solo aparece en literales negativos

o en la cabeza, nada la acota. En términos sencillos para el desarrollador, la regla de seguridad impide que el motor de inferencia intente iterar sobre un universo infinito: anclar cada variable en al menos un predicado positivo garantiza que el software solo procese entidades reales, concretas y finitas.

Ejemplo 4.4: Por qué la regla de `noop` incluye `(role ?p)`

Considérese la variante insegura

```
(<= (legal ?p noop) (not (true (control ?p)))).
```

La variable `?p` aparece en la cabeza y en un literal negativo, pero en ningún literal positivo, de modo que viola (4.45). En el estado s_0 del Gato, la única instancia de `(true (control ?p))` es la de `x`; por lo tanto, la regla deriva `(legal t noop)` para *todo* término $t \in U_\Lambda \setminus \{\mathtt{x}\}$: para `o`, pero también para `b`, para `3`, para `cell(1,1,b)` y para cada uno de los infinitos términos restantes. El modelo resultante es infinito y la relación `legal` deja de corresponder a los roles. Al agregar `(role ?p)`, que es positivo, `?p` queda acotada a $R = \{\mathtt{x}, \mathtt{o}\}$ y el problema desaparece. Lo mismo explica el literal `(role ?p)` en las reglas de `line` y de `goal` del Código 3.5.

4.4.2. Grafo de dependencias y estratificación

La negación por fallo introduce un problema que no tiene la lógica clásica: una regla puede depender de que otra *no* se cumpla, y si esa dependencia es circular, el programa puede no tener un significado único. El concepto de estratificación resuelve de forma transparente el dilema del huevo y la gallina en la lógica computacional. Cuando una regla recurre a la negación, necesita que el hecho negado ya esté plenamente decidido. Si dos reglas dependieran negativamente una de la otra en un ciclo —por ejemplo, postulando que es el turno del jugador uno si no es el del jugador dos, y que es el turno del jugador dos si no es el del jugador uno—, el sistema caería en una ambigüedad insoluble, incapaz de decidir quién debe mover primero. La estratificación organiza las relaciones en niveles jerárquicos o pisos, asegurando que para evaluar una negación en el piso actual, todos los hechos relevantes del piso inferior ya hayan sido calculados exhaustivamente. La herramienta formal para detectarlo y garantizar este orden es el grafo de dependencias [46].

Definición 4.15: Grafo de dependencias

El **grafo de dependencias** de un programa D es el grafo dirigido con aristas etiquetadas $\mathrm{DG}(D) = \langle \Pi, E^+ \cup E^- \rangle$ cuyos vértices son los símbolos de relación de D y en el que hay una arista positiva $q \xrightarrow{+} p$ si q es la relación de algún átomo de $B^+(\rho)$, y una arista negativa $q \xrightarrow{-} p$ si q es la relación de algún átomo de $B^-(\rho)$, para alguna regla $\rho \in D$ cuya cabeza tiene relación p. Se dice que p **depende** de q si existe un camino dirigido de q a p.

Un grafo dirigido $\langle V, E \rangle$ consta de un conjunto V de vértices y un conjunto E de aristas, cada una con un origen y un destino. En $\mathrm{DG}(D)$ los vértices son los símbolos de relación de Π, y una arista $q \to p$ registra que, para establecer p, alguna regla necesita consultar q. Las aristas

se dividen en E^+, las positivas, y E^-, las negativas, según si q aparece afirmada o negada en el cuerpo. Por ejemplo, la regla `(<= board_full (not open))` del Código 3.5 aporta la arista negativa $\texttt{open} \xrightarrow{-} \texttt{board_full}$: para saber si el tablero está lleno hay que saber si *no* hay casillas abiertas.

Definición 4.16: Programa estratificado

Un programa D es **estratificado** si ningún ciclo de $\mathrm{DG}(D)$ contiene una arista negativa. Equivalentemente, si existe una función $\sigma : \Pi \to \mathbb{N}$, llamada **estratificación**, tal que para toda arista

$$q \xrightarrow{+} p \implies \sigma(q) \leq \sigma(p) \qquad \text{y} \qquad q \xrightarrow{-} p \implies \sigma(q) < \sigma(p). \tag{4.46}$$

El conjunto $D_k = \{\rho \in D \mid \sigma(\mathrm{rel}(\mathrm{cabeza}(\rho))) = k\}$ es el **estrato** k de D.

La función σ asigna a cada relación un nivel, un número natural, y las dos implicaciones de (4.46) imponen el orden en que deben calcularse. Si p usa a q en forma positiva, q debe estar en un nivel igual o inferior al de p; si la usa negada, en un nivel estrictamente inferior. En el estrato D_k se agrupan las reglas cuya cabeza, $\mathrm{cabeza}(\rho)$, tiene una relación, $\mathrm{rel}(\cdot)$, de nivel k. La estratificación ordena así el cálculo: una relación que aparece negada debe estar completamente calculada antes de usarse. Las desigualdades `distinct` no generan aristas, porque su valor depende únicamente de la identidad sintáctica de los términos y no de ninguna otra relación.

Ejemplo 4.5: Estratificación de la descripción del Gato

La Figura 4.4 muestra el grafo de dependencias de los Códigos 3.2 a 3.5. Las aristas negativas son cuatro: $\texttt{true} \xrightarrow{-} \texttt{legal}$ (por `(not (true (control ?p)))`), $\texttt{does} \xrightarrow{-} \texttt{next}$ (por la inercia de casillas vacías), $\texttt{open} \xrightarrow{-} \texttt{board_full}$ y $\texttt{line} \xrightarrow{-} \texttt{goal}$. Ninguna está en un ciclo; de hecho, el grafo es acíclico. La estratificación mínima es

$$\sigma = 0 \quad \text{para } \texttt{role}, \texttt{init}, \texttt{true}, \texttt{does}, \texttt{line}, \texttt{open},$$
$$\sigma = 1 \quad \text{para } \texttt{legal}, \texttt{next}, \texttt{board_full}, \texttt{goal}, \texttt{terminal}.$$

Obsérvese que `terminal` queda en el estrato 1 aunque no tenga aristas negativas de entrada: depende positivamente de `board_full`, que está en el estrato 1, y la condición (4.46) exige $\sigma(\texttt{board_full}) \leq \sigma(\texttt{terminal})$.

Ejemplo 4.6: Un programa no estratificado

Supóngase que alguien intenta expresar la alternancia de turnos con dos relaciones auxiliares mutuamente excluyentes:

```
(<= turno_x (not turno_o))    (<= turno_o (not turno_x)).
```

El grafo contiene el ciclo $\texttt{turno_x} \xrightarrow{-} \texttt{turno_o} \xrightarrow{-} \texttt{turno_x}$, con dos aristas negativas, y ninguna función σ satisface $\sigma(\texttt{turno_x}) < \sigma(\texttt{turno_o}) < \sigma(\texttt{turno_x})$. El programa

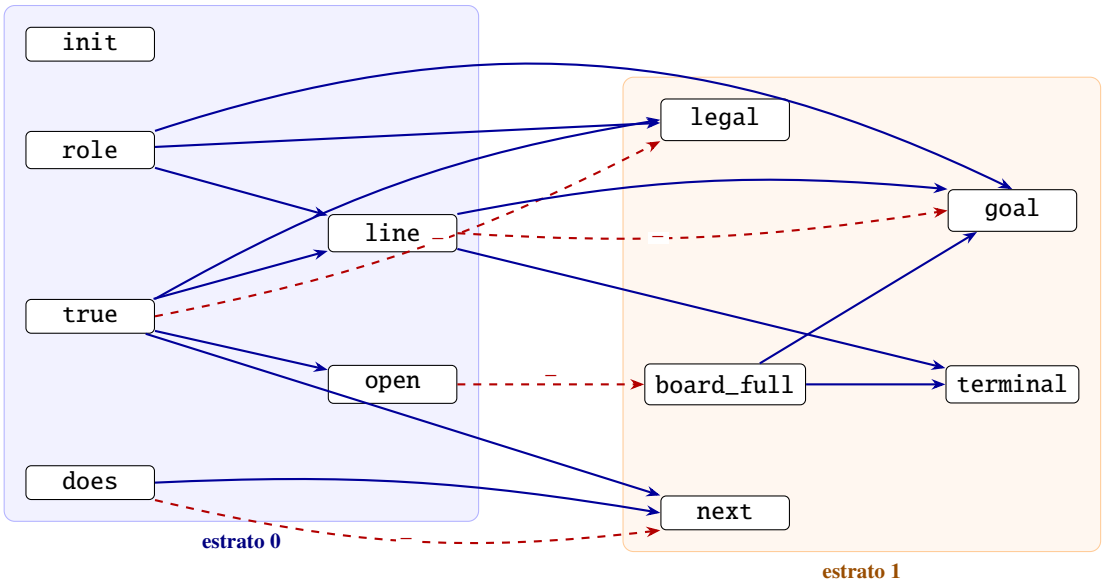


Figura 4.4: Grafo de dependencias de la descripción del Gato. Las aristas continuas son positivas y las discontinuas, negativas. Ninguna arista negativa forma parte de un ciclo, por lo que el programa es estratificado con dos estratos.

tiene dos interpretaciones igualmente defendibles, {`turno_x`} y {`turno_o`}: ambas son modelos estables en el sentido de Gelfond y Lifschitz [54], y nada en las reglas permite elegir entre ellas. Un motor no sabría a quién le toca. Peor aún es la regla `(<= p (not p))`, que no tiene ningún modelo estable: si p fuera falso, la regla obligaría a que fuera verdadero, y si fuera verdadero, no habría razón para derivarlo. La estratificación excluye ambos casos.

4.4.3. Restricción de la recursión

GDL admite recursión, es decir, ciclos positivos en el grafo de dependencias, porque muchos juegos la necesitan: por ejemplo, para definir que una casilla es alcanzable por una torre si es adyacente a otra alcanzable y vacía. En términos prácticos para el desarrollador de software, la **restricción de la recursión** tiene un objetivo muy concreto: evitar desbordamientos de memoria causados por la creación de términos lógicos infinitos. En lenguajes como Prolog es habitual modelar secuencias infinitas anidando funciones (como definir que el sucesor de X es $s(X)$); no obstante, en GDL, si una regla recursiva envolviera términos dentro de funciones sin freno, el compilador generaría objetos cada vez más largos en cada ciclo y colapsaría la memoria del agente. La especificación impone por ello una condición formal que asegura que los argumentos dentro de una llamada recursiva nunca crezcan en tamaño [10].

Definición 4.17: Restricción de la recursión

Un programa D satisface la **restricción de la recursión** si, para toda regla

$$p(t_1, \ldots, t_m) \leftarrow \cdots \wedge q(v_1, \ldots, v_k) \wedge \ldots$$

en la que q es una relación de un literal positivo y p y q pertenecen al mismo componente fuertemente conexo de $\mathrm{DG}(D)$, se cumple para cada $j \in \{1, \ldots, k\}$ al menos una de las condiciones siguientes: (i) v_j es básico; (ii) v_j es uno de los argumentos $t_1, \ldots, t_m$ de la cabeza; o (iii) v_j es una variable que aparece en algún literal positivo del cuerpo cuya relación no pertenece a ese componente.

Un *componente fuertemente conexo* de un grafo dirigido es un conjunto maximal de vértices en el que cada uno puede alcanzar a cada otro siguiendo aristas; en $\mathrm{DG}(D)$, dos relaciones están en el mismo componente si y solo si cada una depende de la otra, es decir, si son mutuamente recursivas. En la regla de la definición, $q(v_1, \ldots, v_k)$ es una llamada recursiva y $v_1, \ldots, v_k$ son sus argumentos. Las tres alternativas dicen de dónde puede provenir cada argumento: de una constante fija (i), de la propia cabeza sin modificar (ii) o de una relación externa al ciclo (iii), cuyo valor ya está calculado. La condición asegura que, a lo largo de un ciclo recursivo, los argumentos no crecen: o se copian de la cabeza, o son constantes, o se toman de relaciones ya calculadas fuera del ciclo, que son finitas.

Ejemplo 4.7: Recursión que genera infinitos términos

El programa

```
(num 0)    (<= (num (s ?x)) (num ?x))
```

define los números naturales en notación de Peano: su modelo es $\{\texttt{num(0)}, \texttt{num(s(0))}, \texttt{num(s(s(0)))}, \ldots\}$, que es infinito. La regla es recursiva ($\texttt{num} \xrightarrow{+} \texttt{num}$) y el argumento $v_1 = \texttt{?x}$ del subobjetivo no es básico, no es igual al argumento $t_1 = \texttt{(s ?x)}$ de la cabeza y no aparece en ninguna otra relación del cuerpo; viola, por lo tanto, la Definición 4.17. La forma correcta de contar en GDL es enumerar un sucesor finito mediante hechos, `(succ 0 1)`, `(succ 1 2)`, etcétera, como se hace en el ejemplo de la Sección 4.6.

4.4.4. Restricciones de uso de las relaciones reservadas

Las tres restricciones anteriores son propias de la programación lógica y tendrían sentido en cualquier programa Datalog. Las que siguen son específicas de GDL: dicen en qué posiciones puede aparecer cada palabra reservada y de qué otras relaciones puede depender [9], [10]. La Tabla 4.8 las resume.

Estas restricciones no son caprichos de estilo. Cada una garantiza que una componente de Γ tenga el *tipo* correcto. Tómese, por ejemplo, la prohibición de que `legal` dependa de `does`. En la Definición 3.4, la legalidad es una función $L_i : S \to \mathcal{P}(A_i)$, que depende solo del estado. Si una regla como `(<= (legal o noop) (does x (mark 2 2)))` fuera admisible, la legalidad de `o` dependería de lo que `x` elige *en el mismo paso*, y L_i tendría que tener tipo $S \times \mathbf{A} \to \mathcal{P}(A_i)$: no podría calcularse antes de que los jugadores decidan, que es precisamente cuando el agente la necesita. Del mismo modo, si `init` dependiera de `true`, el estado inicial se definiría en función de un estado actual que, en $t = 0$, todavía no existe. La Proposición 4.6 de la sección siguiente hace precisa esta correspondencia.

Tabla 4.8: Restricciones de uso de las relaciones reservadas y razón formal de cada una. "Depende" se entiende en el sentido de la Definición 4.15.

Relación	**Restricción**	**Qué garantiza en Γ**
`role`	Solo en hechos básicos.	R es finito y fijo, independiente de estados y acciones.
`init`	Solo en cabezas; no depende de `true`, `does`, `legal`, `next`, `terminal` ni `goal`.	s_0 está definido sin referencia a un estado previo.
`true`	Solo en cuerpos.	El estado es una entrada; ninguna regla lo modifica dentro del mismo instante.
`does`	Solo en cuerpos.	La acción conjunta es una entrada que suministran los jugadores.
`next`	Solo en cabezas.	El estado futuro se calcula pero no se consulta en el mismo paso.
`legal`, `terminal`, `goal`	No dependen de `does`.	L_i, F y G_i son funciones del estado, no de la acción.
`base`, `input`	No dependen de `true` ni de `does`.	$\mathcal{F}$ y A_i son estáticos.
`distinct`	Solo en cuerpos, con variables acotadas.	La desigualdad se evalúa sobre términos ya básicos.

4.4.5. Caracterización del conjunto de descripciones válidas

Con las cuatro familias de restricciones puede definirse el componente $\mathcal{W}$ de la tupla.

Definición 4.18: Descripción GDL válida

Un programa $D \in \mathrm{Prog}(\Lambda)$ es una **descripción GDL válida**, $D \in \mathcal{W}_{\mathrm{GDL}}$, si y solo si

1. tiene signatura consistente (Definición 4.13);
2. todas sus reglas son seguras (Definición 4.14);
3. es estratificado (Definición 4.16);
4. satisface la restricción de la recursión (Definición 4.17); y
5. respeta las restricciones de uso de la Tabla 4.8.

Todas estas condiciones son *estáticas*: se comprueban inspeccionando el texto del programa, sin simular ninguna partida. La consistencia de la signatura, la seguridad y las restricciones de uso se verifican regla por regla; la estratificación y la restricción de la recursión requieren construir $\mathrm{DG}(D)$ y calcular sus componentes fuertemente conexos, lo que puede hacerse en tiempo lineal en el tamaño del grafo. En consecuencia, $\mathcal{W}_{\mathrm{GDL}}$ es decidible, como exige la Definición 4.5.

4.5. Semántica formal de GDL

Falta la componente central: la función $[\![\cdot]\!]$ que dice qué juego denota cada descripción válida. Se construye en dos pasos. El primero asigna a cada programa válido, completado con hechos de entrada, un único modelo finito. El segundo lee las componentes de Γ en esos modelos.

4.5.1. Modelo estándar de un programa estratificado

Aunque la formalización teórica recurre a conceptos como operadores de consecuencia inmediata y puntos fijos sobre retículos, en la práctica de la ingeniería de software el modelo estándar representa un procedimiento de razonamiento hacia adelante (*forward chaining*) sumamente natural. El intérprete arranca con un conjunto inicial de hechos conocidos —el estado actual alimentado mediante `true` y las jugadas ejecutadas con `does`— y procede a evaluar las reglas del programa. Cada vez que todas las condiciones del cuerpo de una regla se cumplen con hechos ya confirmados, la afirmación de la cabeza se deduce y se incorpora a la base de conocimiento como un nuevo hecho verdadero. Este ciclo de derivación se repite paso a paso y estrato por estrato hasta que ya no es posible derivar ningún hecho novedoso; en ese instante el sistema alcanza su estado estacionario o punto fijo, obteniendo la respuesta inequívoca a las consultas del juego.

En términos precisos, sea $\mathrm{ground}(D)$ el conjunto de todas las instancias básicas de las reglas de D, obtenidas al sustituir sus variables por términos de U_Λ de todas las maneras posibles. Una **sustitución** θ es una función finita de variables a términos; $X\theta$ denota el resultado de reemplazar en X cada variable v por $\theta(v)$. Por ejemplo, si $\theta = \{\texttt{?r} \mapsto \texttt{2}, \texttt{?c} \mapsto \texttt{3}\}$, donde $v \mapsto t$ se lee "v se reemplaza por t", entonces $\texttt{cell}(\texttt{?r}, \texttt{?c}, \texttt{b})\theta = \texttt{cell}(\texttt{2}, \texttt{3}, \texttt{b})$. Una sustitución es *básica* si todos los términos que asigna son básicos.

Definición 4.19: Operador de consecuencia inmediata

Para un conjunto de reglas P, un conjunto de átomos básicos N (los que se consideran fijados para la negación) y una interpretación $I \subseteq B_\Lambda$, el **operador de consecuencia inmediata** es

$$\mathbf{T}_{P,N}(I) = \left\{ h\theta \;\middle|\; \begin{array}{l} (h \leftarrow B) \in P,\ \theta \text{ sustitución básica}, \\ B^+\theta \subseteq I, \quad B^-\theta \cap N = \emptyset, \\ t\theta \neq u\theta \text{ para toda } (t \neq u) \in B^{\neq} \end{array} \right\}. \tag{4.47}$$

Desglosado en términos prácticos, el operador $\mathbf{T}_{P,N}(I)$ funciona exactamente igual que el motor de reglas de un videojuego o una batería de instrucciones condicionales (`if-then`). Imagínese que disponemos de una libreta I donde están apuntados todos los hechos confirmados en este momento. Cada regla lógica tiene la forma «si se cumple el cuerpo B, entonces concluye la cabeza h». Para que una regla se dispare y añada una conclusión concreta $h\theta$ a la libreta, el evaluador realiza tres comprobaciones elementales: en primer lugar, revisa las premisas positivas ($B^+\theta \subseteq I$), exigiendo que todo lo que la regla pide que sea verdad ya esté registrado en la libreta I; en segundo lugar, evalúa las premisas negativas ($B^-\theta \cap N = \emptyset$), asegurándose de que nada de lo que la regla prohíbe esté presente en el conjunto de hechos descartados N; y en tercer lugar, verifica las desigualdades sintácticas ($t\theta \neq u\theta$), confirmando que los términos que deben ser distintos no coincidan. Si se superan estas tres pruebas, el operador deriva la conclusión

$h\theta$. En resumen, $\mathbf{T}_{P,N}(I)$ representa una sola «ronda de deducción»: toma lo que ya sabe (I), evalúa las reglas aplicables y genera en un solo paso todos los nuevos hechos que se desprenden directamente de ellos. Por ejemplo, si I contiene $\texttt{true}(\texttt{control}(\texttt{x}))$ y $\texttt{true}(\texttt{cell}(2,2,\texttt{b}))$, la sustitución $\theta = \{\texttt{?p} \mapsto \texttt{x}, \texttt{?r} \mapsto 2, \texttt{?c} \mapsto 2\}$ satisface el cuerpo de la regla de legalidad, y el operador produce $\texttt{legal}(\texttt{x}, \texttt{mark}(2,2))$.

Para un N fijo, $\mathbf{T}_{P,N}$ es monótono: saber más nunca permite derivar menos. En computación, la idea de un *punto fijo* ($\Psi(I) = I$) simplemente describe el instante en que el sistema se estabiliza porque ya no se puede deducir absolutamente nada nuevo: es el momento en que las reglas dejan de producir hechos adicionales. El teorema de Knaster y Tarski garantiza formalmente que sobre un retículo completo este proceso siempre converge a un resultado único y bien determinado, denominado el *menor punto fijo* $\mathrm{lfp}(\Psi)$ (*least fixed point*) [45]. Para los operadores de esta sección, ese punto fijo se obtiene aplicando el operador repetidamente a partir de los hechos conocidos:

$$I_0 = \emptyset, \qquad I_{j+1} = \Psi(I_j), \qquad \mathrm{lfp}(\Psi) = \bigcup_{j \geq 0} I_j, \tag{4.48}$$

de modo que I_1 contiene los hechos iniciales, I_2 lo que se deriva de ellos en una primera ronda, y así sucesivamente hasta que el conjunto no cambia más.

Definición 4.20: Modelo estándar

Sea D un programa estratificado con estratos $D_0, D_1, \ldots, D_m$ y sea E un conjunto finito de hechos básicos de entrada. El **modelo estándar** $M(D \cup E)$ se define por inducción sobre los estratos:

$$M_{-1} = E, \tag{4.49}$$

$$M_k = \mathrm{lfp}\left(I \mapsto M_{k-1} \cup \mathbf{T}_{D_k,I}(I)\right), \qquad k = 0, 1, \ldots, m, \tag{4.50}$$

$$M(D \cup E) = M_m, \tag{4.51}$$

donde lfp denota el menor punto fijo.

La Definición 4.20 calcula el modelo por capas, en el orden que fija la estratificación. La Ecuación (4.49) parte de los hechos de entrada E, que más adelante serán el estado y la acción conjunta consultados. La Ecuación (4.50) calcula el estrato k: aplica repetidamente las reglas de D_k sobre lo ya obtenido en el estrato anterior, M_{k-1}, hasta llegar al menor punto fijo. La notación $I \mapsto \ldots$ describe el operador que se itera, la función que a cada interpretación I le asigna $M_{k-1} \cup \mathbf{T}_{D_k,I}(I)$. La Ecuación (4.51) declara que el modelo completo es el resultado del último estrato, M_m.

La negación dentro del estrato k solo se refiere a relaciones de estratos inferiores, cuyos átomos ya están completamente decididos en M_{k-1} y no cambian durante la iteración de (4.50); por eso la negación por fallo es segura en ese punto. Apt, Blair y Walker demostraron que el resultado no depende de la estratificación elegida [46], y para programas estratificados coincide con el único modelo estable de Gelfond y Lifschitz [54]. Es, por lo tanto, *el* significado del programa.

Proposición 4.5: Modelo único y finito

Sea $D \in \mathcal{W}_{\mathrm{GDL}}$ y sea E un conjunto finito de hechos básicos de `true` y `does`. Entonces $M(D \cup E)$ existe, es único y es finito.

Demostración (esbozo). La existencia y la unicidad se siguen de la estratificación: añadir hechos de `true` y `does` no crea aristas nuevas en $\mathrm{DG}(D)$, porque estas relaciones nunca aparecen en cabezas, y el programa sigue siendo estratificado; el modelo estándar está entonces bien definido y no depende de la estratificación [46]. Para la finitud, la seguridad garantiza que cada aplicación de una regla instancia todas sus variables con términos que ya aparecen en átomos derivados, de modo que no se introducen términos "de la nada". Fuera de los ciclos de $\mathrm{DG}(D)$, cada estrato aplica un número finito de reglas sobre un conjunto finito de términos. Dentro de un ciclo, la restricción de la recursión garantiza que los argumentos de los subobjetivos recursivos provienen de la cabeza, de constantes o de relaciones ya calculadas, por lo que el conjunto de términos que circulan por el ciclo está acotado por un conjunto finito fijado antes de entrar en él, y el menor punto fijo se alcanza tras un número finito de iteraciones [10]. ■

4.5.2. Función semántica $[\![\cdot]\!]$

El segundo paso, debido en su forma precisa a Schiffel y Thielscher [47], consiste en codificar estados y acciones como hechos de entrada y leer la respuesta en el modelo estándar. Para un estado $s \subseteq U_\Lambda$ y una acción conjunta $\mathbf{a} = (a_1, \ldots, a_n)$ se definen

$$s^{\mathrm{true}} = \{\texttt{true}(f) \mid f \in s\}, \qquad \mathbf{a}^{\mathrm{does}} = \{\texttt{does}(r_i, a_i) \mid i = 1, \ldots, n\}. \tag{4.52}$$

El conjunto s^{true} envuelve cada fluent del estado en la relación `true`, y $\mathbf{a}^{\mathrm{does}}$ registra con `does` la acción que eligió cada rol. Por ejemplo, para el estado inicial del Gato, s_0^{true} contiene los diez hechos $\texttt{true}(\texttt{cell}(i, j, \texttt{b}))$ y $\texttt{true}(\texttt{control}(\texttt{x}))$, y para la acción conjunta $\mathbf{a} = (\texttt{mark}(2,2), \texttt{noop})$ se tiene $\mathbf{a}^{\mathrm{does}} = \{\texttt{does}(\texttt{x}, \texttt{mark}(2,2)), \texttt{does}(\texttt{o}, \texttt{noop})\}$. Estos hechos son las "preguntas" que el motor formula a la descripción.

Definición 4.21: Juego denotado por una descripción GDL

Sea $D \in \mathcal{W}_{\mathrm{GDL}}$. El juego $[\![D]\!] = \Gamma_D = \langle R, S, s_0, \{A_i\}, \{L_i\}, T, F, \{G_i\}\rangle$ se define por

$$R = \{r \mid \texttt{role}(r) \in M(D)\}, \tag{4.53}$$

$$s_0 = \{f \mid \texttt{init}(f) \in M(D)\}, \tag{4.54}$$

$$L_i(s) = \{a \mid \texttt{legal}(r_i, a) \in M(D \cup s^{\mathrm{true}})\}, \tag{4.55}$$

$$T(s, \mathbf{a}) = \{f \mid \texttt{next}(f) \in M(D \cup s^{\mathrm{true}} \cup \mathbf{a}^{\mathrm{does}})\}, \tag{4.56}$$

$$s \in F \iff \texttt{terminal} \in M(D \cup s^{\mathrm{true}}), \tag{4.57}$$

$$G_i(s) = v \iff \texttt{goal}(r_i, v) \in M(D \cup s^{\mathrm{true}}). \tag{4.58}$$

El espacio de estados es $S = \mathrm{Reach}(\Gamma_D)$, según la Definición 4.6. Si D contiene reglas de `base` e `input`, el universo de fluents y los catálogos de acciones son

$$\mathcal{F} = \{f \mid \texttt{base}(f) \in M(D)\}, \qquad A_i = \{a \mid \texttt{input}(r_i, a) \in M(D)\}; \tag{4.59}$$

en caso contrario se toman $\mathcal{F} = \bigcup_{s \in S} s$ y $A_i = \bigcup_{s \in S \setminus F} L_i(s)$.

Cada ecuación de la Definición 4.21 se lee de la misma manera: una componente del juego es el conjunto de objetos para los que cierto átomo pertenece a cierto modelo. La Ecuación (4.53) dice que los roles son los términos r para los que $\texttt{role}(r)$ se deriva del programa sin entradas; en el Gato, $R = \{\texttt{x}, \texttt{o}\}$. La Ecuación (4.54) dice que el estado inicial reúne los fluents marcados con `init`, también sin entradas. La Ecuación (4.55) define las acciones legales del rol r_i en el estado s: se agrega s^{true} al programa, se calcula su modelo y se recogen los a con $\texttt{legal}(r_i, a)$ en ese modelo. La Ecuación (4.56) define el sucesor: se agregan s^{true} y $\mathbf{a}^{\text{does}}$, se calcula el modelo y el nuevo estado está formado por los fluents f con $\texttt{next}(f)$ en él. Las Ecuaciones (4.57) y (4.58) usan el bicondicional $\Longleftrightarrow$, "si y solo si": un estado es terminal exactamente cuando la proposición `terminal` pertenece al modelo, y la utilidad del rol r_i vale v exactamente cuando $\texttt{goal}(r_i, v)$ pertenece al modelo.

En (4.59), el universo de fluents y los catálogos de acciones se leen de las relaciones `base` e `input` cuando la descripción las incluye. Si no las incluye, se reconstruyen a partir del juego: $\bigcup_{s \in S} s$ es la unión de todos los estados alcanzables, es decir, el conjunto de fluents que aparecen en alguno, y $\bigcup_{s \in S \setminus F} L_i(s)$ es el conjunto de acciones que el rol r_i puede ejecutar en algún estado no terminal.

Las ecuaciones (4.53) a (4.58) dicen, componente por componente, cómo se relaciona una descripción con la tupla formal de la Definición 3.4. En ellas se concreta la conexión entre el lenguaje y el modelo: cada palabra reservada de K_Γ es el nombre de una componente de Γ, y su valor en un estado se obtiene consultando el modelo estándar del programa completado con ese estado. Las cinco consultas de la interfaz computacional de la Sección 3.4 son, literalmente, las cinco expresiones (4.54) a (4.58).

Una consecuencia crucial de (4.56) es que en GDL la persistencia temporal de los objetos no es automática. El estado siguiente contiene *únicamente* aquellos fluents para los cuales el evaluador logre derivar un átomo de `next`; cualquier hecho verdadero en el estado actual que no reciba soporte de alguna regla desaparece de la memoria en la siguiente transición. Si en una partida de ajedrez un jugador mueve un caballo, las otras treinta y una piezas permanecerían en el tablero solo si existen reglas que explícitamente decreten su permanencia. De ahí que la descripción del Gato requiera obligatoriamente las reglas B y C del Código 3.4: sin ellas, todas las marcas del tablero se borrarían tras cada jugada, quedando únicamente la recién colocada. Este mecanismo encarna el tratamiento formal que GDL ofrece al clásico problema del marco (*frame problem*) formulado por McCarthy y Hayes [34]: la inercia de todo aquello que permanece inalterado debe declararse de forma explícita.

Proposición 4.6: Buena tipificación de la semántica

Para toda $D \in \mathcal{W}_{\text{GDL}}$, las ecuaciones (4.53) a (4.58) definen objetos del tipo que exige la Definición 3.4: R es un conjunto finito de constantes; s_0 es un conjunto finito de términos básicos; L_i es una función de estados en conjuntos finitos de acciones; T es una función de pares estado–acción en estados; F es un conjunto de estados; y, si para cada $s \in F$ y cada rol hay exactamente un valor de meta, G_i es una función de F en valores de meta.

Demostración. Por la Proposición 4.5, cada modelo que aparece en las ecuaciones es único y finito; por lo tanto, cada conjunto definido es finito y está determinado sin ambigüedad. Falta ver que cada componente depende solo de los argumentos que su tipo indica. Como `role` solo aparece en hechos básicos, R no depende de ningún estado. Como `init` no depende de `true` ni de `does`, sus átomos en $M(D)$ son los mismos que en $M(D \cup E)$ para cualquier entrada E, de modo que s_0 es una constante del juego. Como `legal`, `terminal` y `goal` no dependen de `does`, sus átomos en $M(D \cup s^{\text{true}} \cup \mathbf{a}^{\text{does}})$ coinciden con los de $M(D \cup s^{\text{true}})$ para toda $\mathbf{a}$; en consecuencia, $L_i(s)$, la pertenencia a F y $G_i(s)$ son funciones de s solamente. Por último, como `true` y `does` no aparecen en cabezas, ninguna regla altera los hechos de entrada, y el estado y la acción consultados son exactamente los suministrados. ■

Proposición 4.7: Determinismo

Para toda $D \in \mathcal{W}_{\text{GDL}}$, la transición T de Γ_D es una función: a cada estado y acción conjunta corresponde un único sucesor.

Demostración. Es consecuencia inmediata de la unicidad del modelo estándar (Proposición 4.5): el conjunto definido en (4.56) está determinado por D, s y $\mathbf{a}$. ■

El determinismo de GDL es, así, una propiedad de su semántica y no un supuesto adicional: GDL simplemente carece de construcciones para expresar azar, que es lo que agrega la constante `random` de GDL-II.

4.5.3. Clase de juegos objetivo y juegos bien formados

Las Proposiciones 4.5 a 4.7 muestran que toda descripción válida denota una estructura del tipo correcto, pero no que denote un *buen* juego. Nada impide escribir una descripción válida en la que un rol se quede sin jugadas o en la que la partida no termine nunca. Para eso, la especificación define una segunda capa de condiciones que se refieren al comportamiento dinámico del juego y no a la forma del texto [9], [10]. La Definición 3.7 ya recogió dos de ellas; aquí se completan.

Definición 4.22: Juego GDL bien formado

Sea $D \in \mathcal{W}_{\text{GDL}}$ y $\Gamma_D = [[D]]$. El juego es **bien formado** si cumple:

1. **Terminación.** No existe una sucesión infinita $s_0, s_1, s_2, \ldots$ de estados alcanzables con $s_k \notin F$ y $s_{k+1} = T(s_k, \mathbf{a}_k)$, $\mathbf{a}_k \in \mathbf{L}(s_k)$, para todo k.
2. **Jugabilidad.** Para todo $s \in \text{Reach}(\Gamma_D) \setminus F$ y todo $r_i \in R$: $L_i(s) \neq \emptyset$.
3. **Meta funcional.** Para todo $s \in \text{Reach}(\Gamma_D) \cap F$ y todo $r_i \in R$, existe exactamente un v con $\texttt{goal}(r_i, v) \in M(D \cup s^{\text{true}})$, y v es una de las constantes $\texttt{0}, \texttt{1}, \ldots, \texttt{100}$.
4. **Ganabilidad débil.** Para todo $r_i \in R$ existe una sucesión de acciones conjuntas legales que lleva de s_0 a un estado terminal s con $G_i(s) = 100$.

La clase objetivo $\mathfrak{G}_{\text{GDL}}$ es la de los juegos bien formados así obtenidos.

La condición de terminación reformula el axioma de la Definición 3.7: una sucesión infinita de estados no terminales, cada uno obtenido del anterior mediante una acción conjunta legal, sería una partida que nunca acaba. La jugabilidad es la del mismo axioma, restringida a los estados alcanzables, porque los demás nunca se presentan en una partida. La meta funcional usa la expresión "existe exactamente un", que en lógica se escribe $\exists!$: no basta con que haya un valor de meta, sino que no debe haber dos. La ganabilidad débil pide, para cada rol, al menos una trayectoria legal de s_0 a un estado terminal en el que ese rol obtiene la utilidad máxima, 100; en términos de la Ecuación (3.23), exige $d(s_0, F_{\text{victoria}}) < \infty$ para cada rol.

La terminación y la jugabilidad ya se justificaron en el Capítulo 3. La meta funcional es la condición que convierte la relación `goal` en las funciones G_i de la Definición 3.4; en este trabajo se exige únicamente sobre los estados terminales, en coherencia con la convención de la Sección 3.4 de no usar la meta como heurística. La ganabilidad débil, que la especificación incluye en su definición de juego bien formado [10], descarta juegos en los que algún participante no tiene ninguna forma de ganar, ni siquiera con la colaboración de los demás.

A diferencia de las condiciones de la Definición 4.18, las de la Definición 4.22 son *dinámicas*: hablan de todos los estados alcanzables. Para un juego con finitos estados alcanzables pueden comprobarse por exploración exhaustiva, pero esa exploración es exactamente lo que es inviable en el Ajedrez. En la práctica, la responsabilidad de que el juego sea bien formado recae en el autor de la descripción, y el intérprete puede a lo sumo detectar violaciones durante la partida o en pruebas acotadas.

Proposición 4.8: Finitud de los estados alcanzables

Si $D \in \mathcal{W}_{\text{GDL}}$ y Γ_D satisface la condición de terminación, entonces $\text{Reach}(\Gamma_D)$ es finito.

Demostración. Considérese el árbol cuyos nodos son las historias legales desde s_0: la raíz es s_0 y los hijos de un nodo no terminal s son los nodos $T(s, \mathbf{a})$ para $\mathbf{a} \in \mathbf{L}(s)$. Por la Proposición 4.5, cada $L_i(s)$ es finito, de modo que $\mathbf{L}(s) = \prod_i L_i(s)$ es finito, pues su cardinalidad es el producto $\prod_i |L_i(s)|$ de números finitos; el árbol es, por lo tanto, *finitamente ramificado*: cada nodo tiene un número finito de hijos. Por la condición de terminación, el árbol no tiene ramas infinitas. El lema de König afirma que todo árbol infinito y finitamente ramificado tiene una rama infinita [55]; por contraposición, el árbol es finito. Todo estado alcanzable etiqueta al menos un nodo del árbol, y por lo tanto $\text{Reach}(\Gamma_D)$ es finito. ■

Observación

La Proposición 4.8 aclara de dónde viene la finitud de S que la Definición 3.4 da por supuesta. En GDL, el universo de Herbrand puede ser infinito y nada en la sintaxis acota el tamaño de los estados a lo largo del tiempo: una regla como `(<= (next (count (s ?n))) (true (count ?n)))` es válida, porque `next` no es recursiva en el grafo de dependencias, y produce términos cada vez más grandes en cada paso. Lo que hace finito al juego es la terminación, un requisito dinámico, y no la gramática.

4.6. Análisis semántico del juego de la resta

Para ver la función semántica en acción conviene un juego lo bastante pequeño como para escribir Γ_D entero. El juego de la resta es una variante de Nim con una sola pila: hay una pila de tres fichas; los jugadores `p1` y `p2` se alternan y en su turno retiran una o dos fichas; gana quien retira la última. El Código 4.2 lo describe en GDL.

```
;; Roles y estado inicial
(role p1)
(role p2)
(init (pile 3))
(init (control p1))

;; Aritmetica finita: GDL no tiene numeros
(succ 0 1)
(succ 1 2)
(succ 2 3)

;; (minus ?n ?k ?m) significa m = n - k, para k en {1, 2}
(<= (minus ?n 1 ?m) (succ ?m ?n))
(<= (minus ?n 2 ?m) (succ ?m ?j) (succ ?j ?n))

;; Legalidad: retirar k fichas si hay suficientes
(<= (legal ?p (take ?k))
    (true (control ?p))
    (true (pile ?n))
    (minus ?n ?k ?m))
(<= (legal ?p noop)
    (role ?p)
    (not (true (control ?p))))

;; Transicion
(<= (next (pile ?m))
    (does ?p (take ?k))
    (true (pile ?n))
    (minus ?n ?k ?m))
(<= (next (control p2)) (true (control p1)))
(<= (next (control p1)) (true (control p2)))

;; Fin y utilidades: gana quien retiro la ultima ficha,
;; es decir, quien NO tiene el turno cuando la pila queda en 0
(<= terminal (true (pile 0)))
(<= (goal ?p 100) (role ?p) (true (pile 0)) (not (true (control ?p))))
(<= (goal ?p 0)   (true (control ?p)) (true (pile 0)))
```

Código 4.2: Descripción GDL del juego de la resta con una pila de tres fichas.

4.6.1. Validación estática

La signatura es $C = \{\texttt{p1}, \texttt{p2}, \texttt{0}, \texttt{1}, \texttt{2}, \texttt{3}, \texttt{noop}, \texttt{100}\}$, $\Phi = \{\texttt{pile}/1, \texttt{control}/1, \texttt{take}/1\}$ y $\Pi = K_\Gamma \cup \{\texttt{succ}/2, \texttt{minus}/3\}$. Todas las reglas son seguras: por ejemplo, en la regla de `legal` para `take`, las variables `?p`, `?n`, `?k` y `?m` aparecen en literales positivos. El grafo de dependencias

tiene una sola arista negativa relevante para `legal` y otra para `goal`, ambas desde `true`; es acíclico, y su estratificación mínima asigna el estrato 1 a `legal` y `goal` y el estrato 0 a todas las demás relaciones. No hay recursión, de modo que la restricción de la Definición 4.17 se cumple trivialmente. Las palabras reservadas se usan en las posiciones permitidas. Por lo tanto, $D_{\text{resta}} \in \mathcal{W}_{\text{GDL}}$.

4.6.2. Modelo estático

El modelo $M(D)$, sin hechos de entrada, contiene los hechos explícitos de `role`, `init` y `succ`, más los átomos de `minus` que se derivan de ellos. Al aplicar la primera regla de `minus` con cada hecho de `succ` se obtienen `minus(1, 1, 0)`, `minus(2, 1, 1)` y `minus(3, 1, 2)`; la segunda regla requiere dos hechos de `succ` encadenados y produce `minus(2, 2, 0)` y `minus(3, 2, 1)`. No hay ninguna instancia con $n = 1$ y $k = 2$, porque no existe j con `succ`$(j, 1)$ y `succ`(m, j) a la vez. Como GDL no tiene aritmética, la relación `minus` es la que la introduce: un átomo `minus`(n, k, m) afirma que $m = n - k$, y los cinco átomos derivados son exactamente las restas $1 - 1 = 0$, $2 - 1 = 1$, $3 - 1 = 2$, $2 - 2 = 0$ y $3 - 2 = 1$, las únicas con resultado no negativo y $k \in \{1, 2\}$. De (4.53) y (4.54), que leen en $M(D)$ los átomos de `role` e `init`, se obtiene

$$R = \{\texttt{p1}, \texttt{p2}\}, \qquad s_0 = \{\texttt{pile(3)},\ \texttt{control(p1)}\}.$$

4.6.3. Legalidad en el estado inicial s_0

Para calcular $L_{\texttt{p1}}(s_0)$ se agrega $s_0^{\text{true}} = \{\texttt{true(pile(3))}, \texttt{true(control(p1))}\}$ y se buscan las sustituciones que satisfacen el cuerpo de las reglas de `legal`. Para la regla de `take`, el literal `(true (control ?p))` fuerza $\theta(\texttt{?p}) = \texttt{p1}$; el literal `(true (pile ?n))` fuerza $\theta(\texttt{?n}) = \texttt{3}$; y `(minus 3 ?k ?m)` admite dos soluciones, $\{\texttt{?k} \mapsto 1, \texttt{?m} \mapsto 2\}$ y $\{\texttt{?k} \mapsto 2, \texttt{?m} \mapsto 1\}$. La regla de `noop` con $\theta(\texttt{?p}) = \texttt{p1}$ falla, porque `(true (control p1))` sí está en el modelo. Con $\theta(\texttt{?p}) = \texttt{p2}$, la regla de `take` falla y la de `noop` se cumple. Así,

$$L_{\texttt{p1}}(s_0) = \{\texttt{take(1)},\ \texttt{take(2)}\}, \qquad L_{\texttt{p2}}(s_0) = \{\texttt{noop}\},$$

y, por la Ecuación (3.16), el espacio de acciones conjuntas legales es el producto cartesiano de ambos conjuntos:

$$\mathbf{L}(s_0) = L_{\texttt{p1}}(s_0) \times L_{\texttt{p2}}(s_0) = \{(\texttt{take(1)}, \texttt{noop}),\ (\texttt{take(2)}, \texttt{noop})\}.$$

Cada par ordenado tiene en su primera posición la acción de `p1` y en la segunda la de `p2`, y su número es $|L_{\texttt{p1}}(s_0)| \cdot |L_{\texttt{p2}}(s_0)| = 2 \cdot 1 = 2$. Es la estructura de turnos alternados de la Sección 3.4.11: el rol que no tiene el turno solo dispone de `noop`, por lo que cada acción conjunta queda determinada por la jugada del rol activo.

4.6.4. Evaluación de la función de transición

Sea $\mathbf{a} = (\texttt{take(2)}, \texttt{noop})$, de modo que $\mathbf{a}^{\text{does}} = \{\texttt{does(p1, take(2))}, \texttt{does(p2, noop)}\}$. En la regla de `next` para `pile`, el literal `(does ?p (take ?k))` unifica solo con el primer hecho, con $\theta = \{\texttt{?p} \mapsto \texttt{p1}, \texttt{?k} \mapsto \texttt{2}\}$; el segundo hecho no unifica, porque `noop` no tiene la forma `(take`

`?k`). Luego `(true (pile ?n))` da `?n` $\mapsto$ `3` y `(minus 3 2 ?m)` da `?m` $\mapsto$ `1`, de donde se deriva `next(pile(1))`. La regla de control deriva `next(control(p2))`. Ninguna otra regla de `next` se aplica, y por (4.56)

$$T\big(s_0, (\texttt{take(2)}, \texttt{noop})\big) = \{\texttt{pile(1)},\ \texttt{control(p2)}\}.$$

Obsérvese que el fluent `pile(3)` desaparece sin que ninguna regla lo borre: simplemente no se deriva su `next`.

4.6.5. Juego denotado y valor Minimax

Repitiendo el cálculo desde cada estado nuevo, tal como prescribe la construcción iterativa $X_0, X_1, X_2, \ldots$ de la Definición 4.6, se obtiene $\mathrm{Reach}(\Gamma_D)$, que tiene seis estados. Todos ellos contienen exactamente dos fluents, uno de `pile` y uno de `control`; por eso se abrevian como $\langle n, p\rangle$, que denota el estado $\{\texttt{pile}(n), \texttt{control}(p)\}$, con n fichas en la pila y el turno en manos de p. La Tabla 4.9 enumera todas las transiciones y la Figura 4.5 muestra el grafo de juego.

Tabla 4.9: Representación explícita completa de $\Gamma_{D_{\text{resta}}}$: estados no terminales, acciones conjuntas legales y sucesores.

Estado s	**Acción conjunta** $\mathbf{a} \in \mathbf{L}(s)$	**Sucesor** $T(s, \mathbf{a})$
$\langle 3, \texttt{p1}\rangle = s_0$	$(\texttt{take(1)}, \texttt{noop})$	$\langle 2, \texttt{p2}\rangle$
$\langle 3, \texttt{p1}\rangle$	$(\texttt{take(2)}, \texttt{noop})$	$\langle 1, \texttt{p2}\rangle$
$\langle 2, \texttt{p2}\rangle$	$(\texttt{noop}, \texttt{take(1)})$	$\langle 1, \texttt{p1}\rangle$
$\langle 2, \texttt{p2}\rangle$	$(\texttt{noop}, \texttt{take(2)})$	$\langle 0, \texttt{p1}\rangle$
$\langle 1, \texttt{p2}\rangle$	$(\texttt{noop}, \texttt{take(1)})$	$\langle 0, \texttt{p1}\rangle$
$\langle 1, \texttt{p1}\rangle$	$(\texttt{take(1)}, \texttt{noop})$	$\langle 0, \texttt{p2}\rangle$

Con la tabla completa pueden comprobarse los requisitos de la Definición 4.22. Todas las trayectorias terminan en a lo sumo tres pasos, de modo que el juego termina. En los cuatro estados no terminales, cada rol tiene al menos una acción legal (`noop` el inactivo, alguna `take` el activo), de modo que es jugable. En los dos estados terminales cada rol tiene exactamente un valor de meta. Y ambos roles pueden ganar con alguna sucesión: `p1` gana por la trayectoria $\langle 3, \texttt{p1}\rangle \to \langle 2, \texttt{p2}\rangle \to \langle 1, \texttt{p1}\rangle \to \langle 0, \texttt{p2}\rangle$ y `p2` por $\langle 3, \texttt{p1}\rangle \to \langle 1, \texttt{p2}\rangle \to \langle 0, \texttt{p1}\rangle$. El juego es bien formado.

El universo de fluents efectivamente usado es

$$\mathcal{F} = \{\texttt{pile(0)},\ \texttt{pile(1)},\ \texttt{pile(2)},\ \texttt{pile(3)},\ \texttt{control(p1)},\ \texttt{control(p2)}\},$$

con $|\mathcal{F}| = 6$ y, por la Ecuación (4.3), $|\mathcal{P}(\mathcal{F})| = 2^6 = 64$; de esos 64 subconjuntos solo 6 son alcanzables. Por último, el valor Minimax de la Definición 3.10 puede calcularse a mano. Con $r_1 = \texttt{p1}$ en el papel de MAX y $r_2 = \texttt{p2}$ en el de MIN, la Ecuación (3.32) dice que el valor de un estado terminal es la utilidad de `p1`, que en un estado donde mueve `p1` se toma el máximo de los valores de los sucesores y que donde mueve `p2` se toma el mínimo. El cálculo procede por

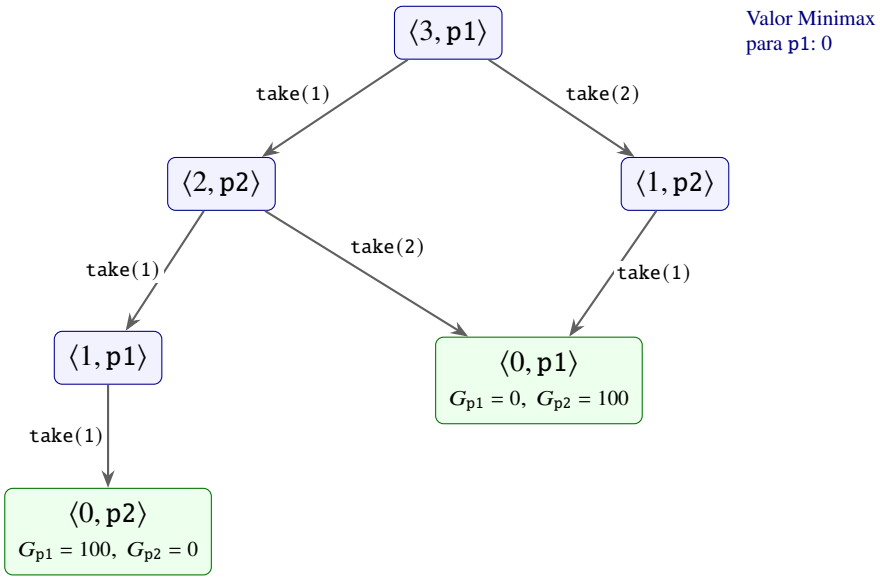


Figura 4.5: Grafo de juego de $\Gamma_{D_{\text{resta}}}$. Las aristas se etiquetan con la acción del rol activo; la del otro rol es siempre `noop`. Los nodos verdes son terminales.

inducción hacia atrás, desde los estados terminales hacia s_0:

$$\begin{aligned}
V(\langle 0, \texttt{p2}\rangle) &= G_{\texttt{p1}}(\langle 0, \texttt{p2}\rangle) = 100,\\
V(\langle 0, \texttt{p1}\rangle) &= G_{\texttt{p1}}(\langle 0, \texttt{p1}\rangle) = 0,\\
V(\langle 1, \texttt{p1}\rangle) &= \max\{V(\langle 0, \texttt{p2}\rangle)\} = 100,\\
V(\langle 1, \texttt{p2}\rangle) &= \min\{V(\langle 0, \texttt{p1}\rangle)\} = 0,\\
V(\langle 2, \texttt{p2}\rangle) &= \min\{V(\langle 1, \texttt{p1}\rangle), V(\langle 0, \texttt{p1}\rangle)\} = \min\{100, 0\} = 0,\\
V(s_0) = V(\langle 3, \texttt{p1}\rangle) &= \max\{V(\langle 2, \texttt{p2}\rangle), V(\langle 1, \texttt{p2}\rangle)\} = \max\{0, 0\} = 0.
\end{aligned}$$

En $\langle 1, \texttt{p1}\rangle$ y $\langle 1, \texttt{p2}\rangle$ el máximo y el mínimo se toman sobre un único sucesor, porque con una ficha solo es legal `take(1)`. En $\langle 2, \texttt{p2}\rangle$, el rival elige entre dejar una ficha a `p1`, que la tomaría y ganaría, o tomar las dos y ganar él mismo; minimizar el valor de `p1` significa elegir lo segundo. El primer jugador pierde con juego perfecto del rival, cualquiera que sea su primera jugada. Lo importante para este análisis es que el valor se obtuvo usando solo las consultas (4.55) a (4.58), sin ningún conocimiento previo del juego: exactamente el tipo de razonamiento que el motor del proyecto debe ser capaz de realizar.

4.7. Análisis semántico del juego del Gato

El juego de la resta permite ver todo el juego a la vez; el Gato permite ver con detalle cómo se evalúan reglas con varias variables, inercia y negación. En esta sección D_{Gato} es la descripción de los Códigos 3.2 a 3.5, que tiene 31 sentencias: 2 de `role`, 10 de `init`, 2 de `legal`, 5 de `next`, 4 de `line`, 1 de `open`, 1 de `board_full`, 3 de `goal` y 3 de `terminal`. Su validez estática se estableció en los Ejemplos 4.1 y 4.5.

4.7.1. Evaluación de la legalidad en el estado inicial

Por (4.54), s_0 es el conjunto de los diez fluents de la Ecuación (3.11). Para calcular $L_{\mathtt{x}}(s_0)$ se evalúa la regla

$$\mathtt{legal(?p, mark(?r, ?c))} \leftarrow \mathtt{true(control(?p))} \wedge \mathtt{true(cell(?r, ?c, b))}$$

sobre $M(D \cup s_0^{\text{true}})$. El primer literal solo unifica con $\mathtt{true(control(x))}$ y fija $\theta(\mathtt{?p}) = \mathtt{x}$. El segundo unifica con cada uno de los nueve hechos $\mathtt{true(cell}(i, j, \mathtt{b}))$, dando nueve sustituciones $\theta_{ij} = \{\mathtt{?p} \mapsto \mathtt{x}, \mathtt{?r} \mapsto i, \mathtt{?c} \mapsto j\}$ para $i, j \in \{1, 2, 3\}$. La segunda regla de `legal`, la de `noop`, falla para `x` porque su literal negativo exige que $\mathtt{true(control(x))}$ *no* esté en el modelo. Para `o` ocurre lo contrario: la primera regla no tiene ninguna sustitución, porque $\mathtt{true(control(o))} \notin s_0^{\text{true}}$, y la segunda se cumple con $\theta = \{\mathtt{?p} \mapsto \mathtt{o}\}$. Por lo tanto,

$$L_{\mathtt{x}}(s_0) = \{\mathtt{mark}(i, j) \mid i, j \in \{1, 2, 3\}\}, \quad L_{\mathtt{o}}(s_0) = \{\mathtt{noop}\}, \quad |\mathbf{L}(s_0)| = 9 \cdot 1 = 9.$$

La primera igualdad está escrita por comprensión: $L_{\mathtt{x}}(s_0)$ contiene un término $\mathtt{mark}(i, j)$ por cada elección de fila i y columna j, es decir, las nueve casillas, todas vacías en s_0. La tercera aplica la Ecuación (3.16): el número de acciones conjuntas legales es el producto $|L_{\mathtt{x}}(s_0)| \cdot |L_{\mathtt{o}}(s_0)|$ del número de opciones de cada rol.

4.7.2. Evaluación de la transición $s_0 \to s_1$

Sea $\mathbf{a} = (\mathtt{mark}(2, 2), \mathtt{noop})$. La Tabla 4.10 recorre las reglas del Código 3.4 y muestra qué deriva cada una sobre $M(D \cup s_0^{\text{true}} \cup \mathbf{a}^{\text{does}})$.

Tabla 4.10: Evaluación de las reglas de transición del Gato para s_0 y $\mathbf{a} = (\mathtt{mark}(2, 2), \mathtt{noop})$.

Regla	**Sustituciones que satisfacen el cuerpo**	**Hechos `next` derivados**
A. Efecto directo	Solo $\{\mathtt{?p} \mapsto \mathtt{x}, \mathtt{?r} \mapsto 2, \mathtt{?c} \mapsto 2\}$; `noop` no unifica con `(mark ?r ?c)`.	$\mathtt{cell}(2, 2, \mathtt{x})$
B. Inercia de fichas	Ninguna: en s_0 no hay casilla con marca distinta de `b`.	–
C. Inercia de vacías	Las 8 casillas $(i, j) \neq (2, 2)$; en $(2, 2)$ falla `(not (does x (mark 2 2)))`.	8 hechos $\mathtt{cell}(i, j, \mathtt{b})$
D. Alternancia	La primera regla, porque $\mathtt{control(x)} \in s_0$.	$\mathtt{control(o)}$

En total se derivan $1 + 8 + 1 = 10$ hechos de `next`, y el estado $s_1 = T(s_0, \mathbf{a})$ coincide con el de la Ecuación (3.12), que en el Capítulo 3 se había escrito como diferencia y unión de conjuntos. Aquí se ve de dónde sale esa expresión: el término $\setminus\{\mathtt{cell}(2, 2, \mathtt{b}), \mathtt{control(x)}\}$ corresponde a los fluents de s_0 para los que ninguna regla deriva `next`, y que por lo tanto desaparecen; el término $\cup\{\mathtt{cell}(2, 2, \mathtt{x}), \mathtt{control(o)}\}$ corresponde a los fluents nuevos que derivan las reglas A y D; y los ocho fluents que se conservan son los que deriva la regla de inercia C. La fórmula del Capítulo 3, que allí se presentó como descripción del efecto de la jugada, es aquí una consecuencia de la Ecuación (4.56).

4.7.3. Terminación y utilidades en estados terminales

Considérese el estado s_v en el que `x` ha completado la diagonal principal, `o` ocupa las casillas $(1,2)$ y $(2,3)$ y es turno de `o`:

$$s_v = \{\, \texttt{cell}(1,1,\texttt{x}),\ \texttt{cell}(1,2,\texttt{o}),\ \texttt{cell}(1,3,\texttt{b}),\ \texttt{cell}(2,1,\texttt{b}),\ \texttt{cell}(2,2,\texttt{x}),\ \texttt{cell}(2,3,\texttt{o}),\ \texttt{cell}(3,1,\texttt{b}),\ \texttt{cell}(3,2,\texttt{b}),\ \texttt{cell}(3,3,\texttt{x}),\ \texttt{control}(\texttt{o}) \,\}.$$

En el estrato 0 se deriva `line(x)` por la tercera regla de `line`, con $\theta = \{\texttt{?p} \mapsto \texttt{x}\}$; no se deriva `line(o)`; y se deriva `open`, porque hay casillas vacías. En el estrato 1, `board_full` es falso porque `open` es verdadero; `terminal` se deriva por la regla `(<= terminal (line x))`; `goal(x, 100)` se deriva directamente; y `goal(o, 0)` se deriva con $\theta = \{\texttt{?p} \mapsto \texttt{o}, \texttt{?other} \mapsto \texttt{x}\}$, pues $\texttt{o} \neq \texttt{x}$. La regla de `goal` con valor 0 *no* se aplica a `x`, porque requeriría `line(?other)` con $\texttt{?other} \neq \texttt{x}$, y `line(o)` es falso. Así, $s_v \in F$, $G_{\texttt{x}}(s_v) = 100$ y $G_{\texttt{o}}(s_v) = 0$, con un único valor de meta por rol.

Considérese ahora el tablero lleno sin líneas, representado como una matriz en la que la entrada de la fila i y la columna j es la marca m del fluent $\texttt{cell}(i, j, m)$:

x	o	x
x	o	o
o	x	x

con turno de `o`. Ninguna de las ocho líneas está completa, así que no se deriva `line` para ningún rol; como no hay casillas `b`, tampoco se deriva `open`, y en el estrato 1 la regla `(<= board_full (not open))` deriva `board_full`. Entonces `terminal` se cumple por la tercera regla y las dos reglas de empate derivan `goal(x, 50)` y `goal(o, 50)`. Este ejemplo muestra por qué `open` debe estar en un estrato inferior al de `board_full`: la negación `(not open)` solo tiene sentido una vez que se ha comprobado *todo* el tablero.

4.7.4. Dimensión del juego denotado

Explorando exhaustivamente $\Gamma_{D_{\text{Gato}}}$ desde s_0 con las ecuaciones (4.55) a (4.58) se obtienen las cifras de la Tabla 4.11. Se calcularon con un evaluador auxiliar de programas GDL estratificados, escrito únicamente para verificar los ejemplos de este capítulo y distinto del intérprete que se desarrollará en el proyecto; coinciden con los valores conocidos del Gato, cuyo valor Minimax con juego perfecto de ambos es el empate.

La proporción entre estados alcanzables y representables, $|\,\text{Reach}(\Gamma_D)|/|\mathcal{P}(\mathcal{F})| = 5\,478/2^{29} \approx 1.02 \times 10^{-5}$, indica que solo uno de cada 98 000 subconjuntos de $\mathcal{F}$, aproximadamente, es un estado al que puede llegarse jugando. Esta proporción ilustra que la mayoría de los subconjuntos de $\mathcal{F}$ no son estados con sentido: tableros con dos fichas en la misma casilla, sin indicador de turno o con ambos turnos a la vez. El conjunto S de la Definición 3.4 debe entenderse, por lo tanto, como $\text{Reach}(\Gamma_D)$ y no como $\mathcal{P}(\mathcal{F})$, y el motor nunca debe enumerar este último.

4.8. Clasificación de descripciones defectuosas

Cada restricción de las Secciones 4.3 a 4.5 se entiende mejor a través de lo que ocurre cuando se viola. La Tabla 4.12 reúne un contraejemplo por restricción, el nivel de la Figura 4.3 en el que

Tabla 4.11: Cifras del juego denotado por D_{Gato}. La intersección $F \cap \text{Reach}(\Gamma_D)$ es el conjunto de estados que son a la vez terminales y alcanzables; la suma de la penúltima fila cuenta las aristas del grafo de juego, como en la Ecuación (4.2).

Magnitud	**Valor**
Universo de fluents $\lvert\mathcal{F}\rvert$ (Código 4.1)	29
Estados representables $\lvert\mathcal{P}(\mathcal{F})\rvert = 2^{29}$	536 870 912
Estados alcanzables $\lvert\text{Reach}(\Gamma_D)\rvert$	5 478
Estados terminales alcanzables $\lvert F \cap \text{Reach}(\Gamma_D)\rvert$	958
con victoria de `x` / victoria de `o` / empate	626 / 316 / 16
Transiciones $\sum_{s \in \text{Reach} \setminus F} \lvert\mathbf{L}(s)\rvert$	16 167
Partidas $\lvert\text{Part}(\Gamma_D)\rvert$ (Proposición 4.3)	255 168
Nodos del árbol de juego (historias)	549 946
Valor Minimax de s_0 para `x`	50

se detecta y su consecuencia sobre el juego. Todos son variaciones mínimas de las descripciones del Gato y de la resta, de modo que podrán reutilizarse como casos de prueba del intérprete.

Los contraejemplos anteriores se detectan sin jugar. Hay, sin embargo, una segunda familia de defectos que pasan todas las comprobaciones estáticas y solo se manifiestan al explorar el juego: las violaciones de la Definición 4.22. Se describen a continuación porque su detección exige estrategias diferentes.

Ejemplo 4.8: Violación de la jugabilidad

Si se elimina de D_{Gato} la regla de `noop`, la descripción sigue siendo válida, pero en s_0 se tiene $L_{\mathtt{o}}(s_0) = \emptyset$ y, por lo tanto, $\mathbf{L}(s_0) = L_{\mathtt{x}}(s_0) \times \emptyset = \emptyset$. No existe ninguna acción conjunta legal en un estado no terminal: el juego se bloquea en su primer paso. El error es visible en la primera consulta, pero solo si el intérprete comprueba explícitamente que $L_i(s) \neq \emptyset$ para cada rol.

Ejemplo 4.9: Violación de la terminación y su reparación

Agréguese al juego de la resta una acción para pasar el turno sin retirar fichas:

```
(<= (legal ?p pass) (true (control ?p))).
```

Como `pass` no deriva ningún hecho de `next` para `pile`, la pila desaparecería del estado; supóngase que se añade también la inercia `(<= (next (pile ?n)) (does ?p pass) (true (pile ?n)))`. La descripción es válida, pero ahora existe la trayectoria $\langle 3, \mathtt{p1}\rangle \to \langle 3, \mathtt{p2}\rangle \to \langle 3, \mathtt{p1}\rangle \to \cdots$, que nunca alcanza un estado terminal. La reparación habitual consiste en añadir un contador de pasos: un fluent `step`(k) que se incrementa con hechos de sucesor en cada transición y una regla `(<= terminal (true (step 10)))`, junto con una asignación de metas para ese caso. El juego resultante es una variante del original con una cota de duración; esta es también la forma de garantizar la terminación en juegos como el Ajedrez, cuyas reglas de repetición y de cincuenta movimientos dependen de la historia y no solo de la posición.

Tabla 4.12: Contraejemplos de cada restricción, nivel de detección y consecuencia.

Fragmento defectuoso	**Restricción violada**	**Consecuencia**
`(<= (legal ?p noop) (role ?p)`	Sintaxis	Paréntesis no balanceados; la secuencia no pertenece a $L(\mathcal{G})$.
`(cell 1 1)` junto a `(cell 1 1 b)`	Signatura	`cell` con dos aridades; la signatura no está definida.
`(role not)`	No redefinición	Una conectiva usada como constante.
`(<= (legal ?p noop) (not (true (control ?p))))`	Seguridad	Infinitos hechos de `legal` (Ejemplo 4.4).
`(<= (diff ?a ?b) (distinct ?a ?b))`	Seguridad	La desigualdad se cumple para infinitos pares.
`(<= turno_x (not turno_o))` `(<= turno_o (not turno_x))`	Estratificación	Dos modelos estables; significado ambiguo (Ejemplo 4.6).
`(<= (num (s ?x)) (num ?x))`	Recursión	Modelo infinito (Ejemplo 4.7).
`(<= (legal o noop) (does x (mark 2 2)))`	Uso de `does`	L_i dependería de **a**; tipo incorrecto.
`(<= (init (control o))` `(true (control x)))`	Uso de `init`	s_0 dependería de un estado previo inexistente.
`(<= (true (control o))` `(true (control x)))`	Uso de `true`	Se alteraría el estado dentro del mismo instante.
`(sees x (cell 1 1 o))`	Fuera del dialecto	Palabra de GDL-II; fuera de la clase objetivo del proyecto.

Ejemplo 4.10: Violación de la meta funcional

Si en la regla de derrota del Gato se omite `(distinct ?p ?other)`,

```
(<= (goal ?p 0) (role ?p) (line ?other))
```

en el estado s_v de la sección anterior se derivan $\texttt{goal}(\texttt{x}, \texttt{100})$ y también $\texttt{goal}(\texttt{x}, \texttt{0})$, porque `?other` puede tomar el valor `x`. El ganador recibe dos utilidades, $G_{\texttt{x}}$ deja de ser una función y la condición (3.19) de suma constante pierde sentido. La descripción es estáticamente válida; el defecto solo aparece al evaluar un estado terminal.

4.9. Expresividad y no inyectividad de la semántica

La definición de la semántica se complementa con dos resultados: GDL puede describir cualquier juego de la clase de interés, y cada juego admite muchas descripciones distintas.

Proposición 4.9: Universalidad para juegos finitos de información perfecta

Sea Γ un juego finito, determinista y de información perfecta en el sentido de la Definición 3.4, con utilidades enteras en $\{0, \dots, 100\}$, que satisface la terminación. Entonces existe $D_\Gamma \in \mathcal{W}_{\mathrm{GDL}}$ tal que $[\![D_\Gamma]\!] \cong \Gamma$.

Demostración. Se construye una representación explícita del juego dentro de GDL. Para cada estado alcanzable s se elige una constante nueva $\ulcorner s \urcorner$; para cada acción a, una constante $\ulcorner a \urcorner$; para cada rol, una constante $\ulcorner r_i \urcorner$. La notación $\ulcorner x \urcorner$, habitual en lógica, se lee "el nombre de x": es un símbolo del lenguaje que representa al objeto matemático x, y se eligen nombres distintos para objetos distintos. Se usa un único símbolo de función `at` de aridad 1, de modo que el fluent $\texttt{at}(\ulcorner s \urcorner)$ significa "el juego está en el estado s", y se definen las sentencias:

$\texttt{(role}\ \ulcorner r_i \urcorner\texttt{)}$	para cada $r_i \in R$,
$\texttt{(init (at}\ \ulcorner s_0 \urcorner\texttt{))}$,	
$\texttt{(<= (legal}\ \ulcorner r_i \urcorner\ \ulcorner a \urcorner\texttt{) (true (at}\ \ulcorner s \urcorner\texttt{)))}$	para $s \notin F$, $a \in L_i(s)$,
$\texttt{(<= (next (at}\ \ulcorner T(s,\mathbf{a}) \urcorner\texttt{)) (true (at}\ \ulcorner s \urcorner\texttt{))}$	para $s \notin F$,
$\texttt{(does}\ \ulcorner r_1 \urcorner\ \ulcorner a_1 \urcorner\texttt{)} \dots \texttt{(does}\ \ulcorner r_n \urcorner\ \ulcorner a_n \urcorner\texttt{))}$	$\mathbf{a} \in \mathbf{L}(s)$,
$\texttt{(<= terminal (true (at}\ \ulcorner s \urcorner\texttt{)))}$	para $s \in F$,
$\texttt{(<= (goal}\ \ulcorner r_i \urcorner\ G_i(s)\texttt{) (true (at}\ \ulcorner s \urcorner\texttt{)))}$	para $s \in F$.

Cada línea traduce una componente de Γ a la Definición 4.21. La primera declara un rol por cada elemento de R. La segunda fija como estado inicial el fluent que nombra a s_0. La tercera escribe, para cada estado no terminal s y cada acción $a \in L_i(s)$, una regla que hace legal a a para el rol r_i exactamente cuando el juego está en s. La cuarta escribe, para cada estado no terminal s y cada acción conjunta legal $\mathbf{a} = (a_1, \dots, a_n) \in \mathbf{L}(s)$, una regla que lleva al fluent del sucesor $T(s, \mathbf{a})$ cuando el juego está en s y cada rol r_i ejecuta a_i. Las dos últimas declaran terminales los estados de F y asignan a cada rol su utilidad $G_i(s)$ en ellos.

D_Γ es válido: todas las reglas son básicas y, por lo tanto, seguras; no hay negación, así que es estratificado; no hay recursión; y cada palabra reservada aparece en una posición permitida, sin que `legal`, `terminal` ni `goal` dependan de `does`. En Γ_{D_Γ}, cada estado alcanzable es un conjunto unitario $\{\texttt{at}(\ulcorner s \urcorner)\}$: así es s_0 y, en cada transición, la única regla de `next` cuyo cuerpo se satisface es la que corresponde al estado y a la acción conjunta actuales, que deriva un único fluent. Defínase $h(s) = \{\texttt{at}(\ulcorner s \urcorner)\}$, $\rho(r_i) = \ulcorner r_i \urcorner$ y $\alpha_i(a) = \ulcorner a \urcorner$. Por construcción, las reglas de `legal`, `next`, `terminal` y `goal` reproducen exactamente las condiciones (4.30) a (4.34), de modo que $(\rho, h, \{\alpha_i\})$ es un isomorfismo. ∎

La Proposición 4.9 es un caso particular, con demostración elemental, del resultado de Thielscher, quien probó que GDL-II es universal para la clase mucho más amplia de juegos finitos en forma extensiva con información imperfecta y azar [56]; Soemers et al. obtuvieron un resultado análogo para el lenguaje de Ludii [28]. Para este proyecto la consecuencia práctica es que la elección de GDL, o de un subconjunto que conserve $K_\Gamma \uplus K_{\log}$, no restringe en principio los juegos que pueden plantearse dentro del alcance.

Proposición 4.10: No inyectividad de la semántica

La función $[[\cdot]]_{\mathrm{GDL}}$ no es inyectiva, ni siquiera módulo isomorfismo: existen $D \neq D'$ en $\mathcal{W}_{\mathrm{GDL}}$ con $[[D]] \cong [[D']]$.

Demostración. Basta tomar $D = D_{\mathrm{Gato}}$ y $D' = D_\Gamma$ con $\Gamma = [[D_{\mathrm{Gato}}]]$, construida como en la Proposición 4.9. Ambas son válidas, son distintas y, por esa misma proposición, $[[D']] \cong [[D]]$. Ejemplos más modestos se obtienen renombrando de manera consistente cualquier símbolo del usuario, por ejemplo `cell` por `casilla`, o reordenando las sentencias. ∎

La demostración contiene, además, una medida de la concisión de GDL. Basta contar las sentencias de D_Γ para $\Gamma = [[D_{\mathrm{Gato}}]]$ con las cifras de la Tabla 4.11. Hay 2 sentencias de `role`, una por rol, y 1 de `init`. Las de `legal` son de dos tipos. En cada estado no terminal, el rol activo tiene tantas acciones legales como acciones conjuntas hay, porque el rol inactivo solo dispone de `noop`; sumando sobre todos los estados no terminales, eso da 16 167 reglas, el mismo número que las transiciones. A ellas se suma una regla de `noop` para el rol inactivo en cada uno de los $5\,478 - 958 = 4\,520$ estados no terminales, y en total son $16\,167 + 4\,520 = 20\,687$. Se agregan 16 167 sentencias de `next`, una por transición, 958 de `terminal`, una por estado terminal, y $2 \cdot 958 = 1\,916$ de `goal`, una por rol y estado terminal. En total,

$$2 + 1 + 20\,687 + 16\,167 + 958 + 1\,916 = 39\,731$$

sentencias, frente a las 31 de la descripción factorizada. La razón, $39\,731/31 \approx 1\,282$, crece sin límite con el tamaño del juego, y en el Ajedrez la descripción explícita sería físicamente imposible de escribir. Esta diferencia es la justificación cuantitativa del enfoque declarativo frente a la enumeración.

4.10. Complejidad computacional de las consultas

La efectividad de GDL está garantizada, pero no su eficiencia. Cada consulta del motor equivale a calcular un modelo estándar, y el costo de hacerlo determina cuántos nodos por segundo

podrá explorar la búsqueda. La teoría de la complejidad de la programación lógica permite acotar ese costo [57].

Conviene separar dos medidas. En la *complejidad de datos*, el programa D se considera fijo y el tamaño de la entrada es el de los hechos $s^{\text{true}} \cup \mathbf{a}^{\text{does}}$. En la *complejidad combinada*, también el programa forma parte de la entrada. Para Datalog, con o sin negación estratificada, la complejidad de datos es P-completa y la combinada es EXPTIME-completa [51], [57]. P es la clase de problemas que pueden resolverse en tiempo polinomial en el tamaño de la entrada, es decir, en tiempo acotado por $c \cdot x^k$ para una entrada de tamaño x y constantes c y k; EXPTIME es la de los problemas que pueden resolverse en tiempo exponencial, acotado por $2^{p(x)}$ con p un polinomio. Que un problema sea "completo" para una clase significa que pertenece a ella y que es de los más difíciles que contiene, de modo que no se espera resolverlo con recursos sensiblemente menores. La primera es la relevante durante una partida, porque las reglas no cambian entre consultas: cada consulta se resuelve en tiempo polinomial en el tamaño del estado. La segunda explica por qué una descripción puede ser costosa aunque el estado sea pequeño: el exponente del polinomio depende del número de variables de las reglas.

La fuente concreta de ese exponente es la instanciación. Una regla con v variables, evaluada sobre un universo efectivo de u términos, tiene hasta

$$\underbrace{u \cdot u \cdots u}_{v \text{ veces}} = u^v \tag{4.60}$$

instancias básicas, porque cada una de las v variables puede tomar, de manera independiente, cualquiera de los u valores, y por el principio multiplicativo las elecciones se multiplican. El número crece de forma exponencial con el número de variables de la regla. En el Gato, la regla C del Código 3.4 tiene dos variables, `?r` y `?c`, que recorren $\{1, 2, 3\}$, y produce $3^2 = 9$ instancias, una por casilla; la regla de derrota de `goal` tiene dos variables, `?p` y `?other`, que recorren los dos roles, y produce $2^2 = 4$. Una descripción de Ajedrez, en cambio, necesita reglas que relacionan una casilla de origen, una de destino, una dirección, una pieza y un rol, con universos de 8 filas, 8 columnas y varias direcciones; el número de instancias crece con rapidez, y se evalúa en cada nodo del árbol de búsqueda. Por eso la eficiencia del intérprete pesa tanto como la del algoritmo de búsqueda, como ya se anticipó al mencionar las redes proposicionales [31]. El análisis sugiere tres medidas para el diseño: ordenar los literales de cada cuerpo para que los más restrictivos se evalúen primero, calcular el modelo estático $M(D)$ una sola vez al cargar el juego y reutilizarlo en todas las consultas, y separar en la representación interna los estratos que no dependen de `does` de los que sí, para no recalcular la legalidad al consultar una transición.

4.11. Análisis comparativo de lenguajes de descripción de juegos

La Definición 4.5 permite situar en un mismo plano lenguajes que, a primera vista, tienen poco en común. La diferencia esencial entre ellos está en dos componentes: la clase objetivo $\mathfrak{G}$, es decir, qué juegos pueden describir, y la forma de $[\![\cdot]\!]$, es decir, cómo se obtiene el juego a partir del texto.

En la familia GDL, $[\![\cdot]\!]$ es una semántica *declarativa*: el significado se define mediante modelos lógicos, con independencia de cualquier implementación, y las palabras reservadas

nombran componentes de Γ. GDL-II y GDL-III conservan esta forma y amplían K y $\mathfrak{G}$, como se resumió en la Tabla 4.6 [26], [53].

Ludii adopta un enfoque opuesto en el nivel del vocabulario. Su unidad es el *ludema*, un concepto de alto nivel propio de los juegos, como un tablero cuadrado, un movimiento deslizante o una condición de alineación [27]. Su gramática no se escribe a mano: se deriva automáticamente de la jerarquía de clases del código que implementa cada ludema, lo que Browne llama una gramática de clases [58]. En términos de la Definición 4.5, K_{Ludii} es mucho mayor que K_{GDL}, con cientos de ludemas, cada uno con un significado específico, y $[\![\cdot]\!]_{\text{Ludii}}$ es *operacional*: el significado de un ludema lo fija el código que lo ejecuta. La ventaja es una descripción mucho más breve y legible; el costo es que la semántica depende de la implementación de referencia.

Regular Boardgames (RBG) ocupa una posición intermedia [29]. Describe el tablero como un grafo dirigido con aristas etiquetadas por direcciones y expresa las reglas como una expresión regular sobre un alfabeto de acciones elementales: desplazamientos por el grafo, comprobaciones y modificaciones del contenido de las casillas, operaciones sobre variables acotadas y cambios de turno. El conjunto de movimientos legales es, por lo tanto, un lenguaje regular, y su semántica puede calcularse mediante autómatas finitos, lo que permite implementaciones muy eficientes manteniendo una definición formal precisa.

El lenguaje PDDL de planificación automática comparte con GDL la representación factorizada y la descripción de acciones mediante precondiciones y efectos [59], pero su clase objetivo es distinta: problemas con un único agente que busca alcanzar una meta, sin adversarios ni utilidades por rol. Por último, OpenSpiel no es un lenguaje de descripción en el sentido de la Definición 4.5: los juegos se programan directamente en C++ o Python contra una interfaz común [30]. En su caso, la descripción vive en el código fuente, y no hay Σ^* de reglas que un agente pueda leer; es el extremo opuesto del desacoplamiento que persigue este Trabajo Terminal. La Tabla 4.13 resume la comparación.

4.12. Requisitos derivados para el intérprete

El análisis anterior se traduce directamente en obligaciones para el intérprete de la Figura 5.1. Cualquiera que sea la decisión final sobre el lenguaje (GDL completo, un subconjunto o un DSL inspirado en él), la estructura de la Definición 4.5 debe conservarse, porque es la que garantiza que cada archivo aceptado denote un juego y que el motor pueda consultarlo. La Tabla 4.14 enumera los requisitos de análisis que se derivan, con su nivel de verificación y su fundamento formal; servirán como punto de partida para la especificación de requisitos y para el diseño de pruebas.

Del análisis se desprende, además, el subconjunto mínimo de palabras reservadas que el proyecto necesita: las ocho relaciones de K_Γ, sin las cuales no puede expresarse ninguna componente de Γ, y las conectivas <=, `not` y `distinct`. La disyunción `or` puede admitirse sin costo semántico, porque π la elimina por normalización, y las relaciones `base` e `input` son recomendables porque permiten fijar de antemano el tamaño de las estructuras internas. Para el Ajedrez, el análisis anticipa tres necesidades que no aparecen en el Gato: fluents que conserven la información histórica que las reglas exigen, como los derechos de enroque y la casilla de captura al paso (Sección 3.4); un contador de pasos que garantice la terminación, como en el Ejemplo 4.9; y un intérprete lo bastante eficiente para que el costo de la Sección 4.10 no domine el tiempo de cada jugada. Estas necesidades, junto con los requisitos de la Tabla 4.14, constituyen el punto de

Tabla 4.13: Comparación de lenguajes y entornos de descripción de juegos según las componentes de la Definición 4.5.

Lenguaje	**Vocabulario reservado**	**Clase $\mathfrak{G}$**	**Semántica**	**Dónde vive la descripción**
GDL	8 relaciones del juego y 4 conectivas	Finitos, deterministas, información perfecta	Declarativa: modelo estándar	Archivo de reglas lógicas
GDL-II	GDL + `sees`, `random`	Agrega información imperfecta y azar	Declarativa	Archivo de reglas lógicas
GDL-III	GDL-II + `knows`	Juegos epistémicos	Declarativa	Archivo de reglas lógicas
Ludii	Cientos de ludemas	Muy amplia; universal para juegos finitos	Operacional: implementación de ludemas	Archivo de ludemas
RBG	Acciones elementales y operadores regulares	Juegos de tablero finitos	Autómatas finitos	Archivo con grafo y expresión regular
PDDL	Predicados, acciones, precondiciones y efectos	Planificación de un agente	Sistema de transición de estados	Archivos de dominio y problema
OpenSpiel	No aplica	Muy amplia	La del programa	Código fuente

Tabla 4.14: Requisitos derivados del análisis del lenguaje de descripción.

ID	Requisito	Nivel	Fundamento
RA-01	Segmentar el archivo en tokens y rechazar cadenas sin segmentación válida, indicando línea y columna.	Léxico	Tabla 4.4
RA-02	Rechazar secuencias que no pertenezcan a $L(\mathcal{G})$ con un diagnóstico localizado.	Sintáctico	Tabla 4.7
RA-03	Inferir la signatura y rechazar aridades inconsistentes y redefiniciones de palabras reservadas.	Estático	Definición 4.13
RA-04	Rechazar palabras de dialectos no soportados (`sees`, `random`, `knows`).	Estático	Tabla 4.6
RA-05	Verificar la seguridad de cada regla.	Estático	Definición 4.14
RA-06	Construir el grafo de dependencias, verificar la estratificación y conservar los estratos para la evaluación.	Estático	Definición 4.16
RA-07	Verificar la restricción de la recursión.	Estático	Definición 4.17
RA-08	Verificar las restricciones de uso de las relaciones reservadas.	Estático	Tabla 4.8
RA-09	Calcular R, s_0 y, si existen, $\mathcal{F}$ y A_i una sola vez al cargar el juego.	Semántico	Ecuaciones (4.53), (4.54) y (4.59)
RA-10	Responder las consultas $L_i(s)$, $T(s, \mathbf{a})$, $s \in F$ y $G_i(s)$ mediante el modelo estándar.	Semántico	Ecuaciones (4.55) a (4.58)
RA-11	Detectar durante la partida estados no terminales con algún $L_i(s) = \emptyset$.	Dinámico	Ejemplo 4.8
RA-12	Detectar estados terminales sin valor de meta o con más de uno por rol.	Dinámico	Ejemplo 4.10
RA-13	Ofrecer un modo de verificación exhaustiva para juegos pequeños, que compruebe terminación, jugabilidad, meta funcional y ganabilidad.	Dinámico	Definición 4.22
RA-14	Incluir como casos de prueba las descripciones de las Secciones 4.6 a 4.8 y comprobar las cifras de la Tabla 4.11.	Pruebas	Secciones 4.6–4.8

partida del resto del análisis del sistema.

Capítulo 5

Diseño

5.1. Arquitectura conceptual propuesta

La arquitectura de la Figura 5.1 separará reglas, interpretación, representación, generación de sucesores y decisión mediante el modelo formal de la Sección 3.4. El contorno delimitará el núcleo reutilizable; las entradas y la coordinación de la partida quedarán fuera. Se trata de un diseño conceptual, sin fijar tecnología ni acreditar módulos implementados.

1. **Descripción externa.** Cada archivo expresará roles, estado inicial, acciones, legalidad, transiciones, terminalidad y utilidades. La elección del lenguaje requerirá justificar expresividad, restricciones y costo de interpretación.

2. **Intérprete.** Analizará la estructura y las restricciones semánticas acordadas. La descripción aceptada originará el modelo; el rechazo producirá un diagnóstico y detendrá la carga.

3. **Modelo y generador.** El modelo conservará las reglas y la representación de estados; el generador resolverá consultas y construirá sucesores a solicitud del motor. La carga proporcionará s_0, sin materializar por anticipado todo el espacio de Ajedrez.

4. **Motor y ciclo de decisión.** Recibirá estado actual, rol y presupuesto. Si el estado actual es terminal, reportará utilidades y finalizará la partida; en otro caso explorará sucesores y elegirá una acción legal. Los terminales hipotéticos solo servirán para valorar ramas.

La coordinación solicitará al generador la transición real $T(s, \mathbf{a}^*)$ del mismo modelo y recibirá el sucesor para actualizar el turno. Al cargar una descripción aceptada recibirá s_0; en los turnos posteriores utilizará el sucesor recibido. Para formar $\mathbf{a}^*$ se conservará la convención de acciones inactivas. Esta transición real se distinguirá de los sucesores hipotéticos de la búsqueda.

Se estudiarán Minimax con alfa-beta y MCTS/UCT, con heurísticas generales y selección pendiente. El núcleo conservará sus componentes al cambiar el archivo, sin valores de piezas ni procedimientos particulares del dominio. La evaluación de la Sección 1.6.5 registrará contadores y contrastará resultados con referencias externas, sin proporcionar sus respuestas al agente.

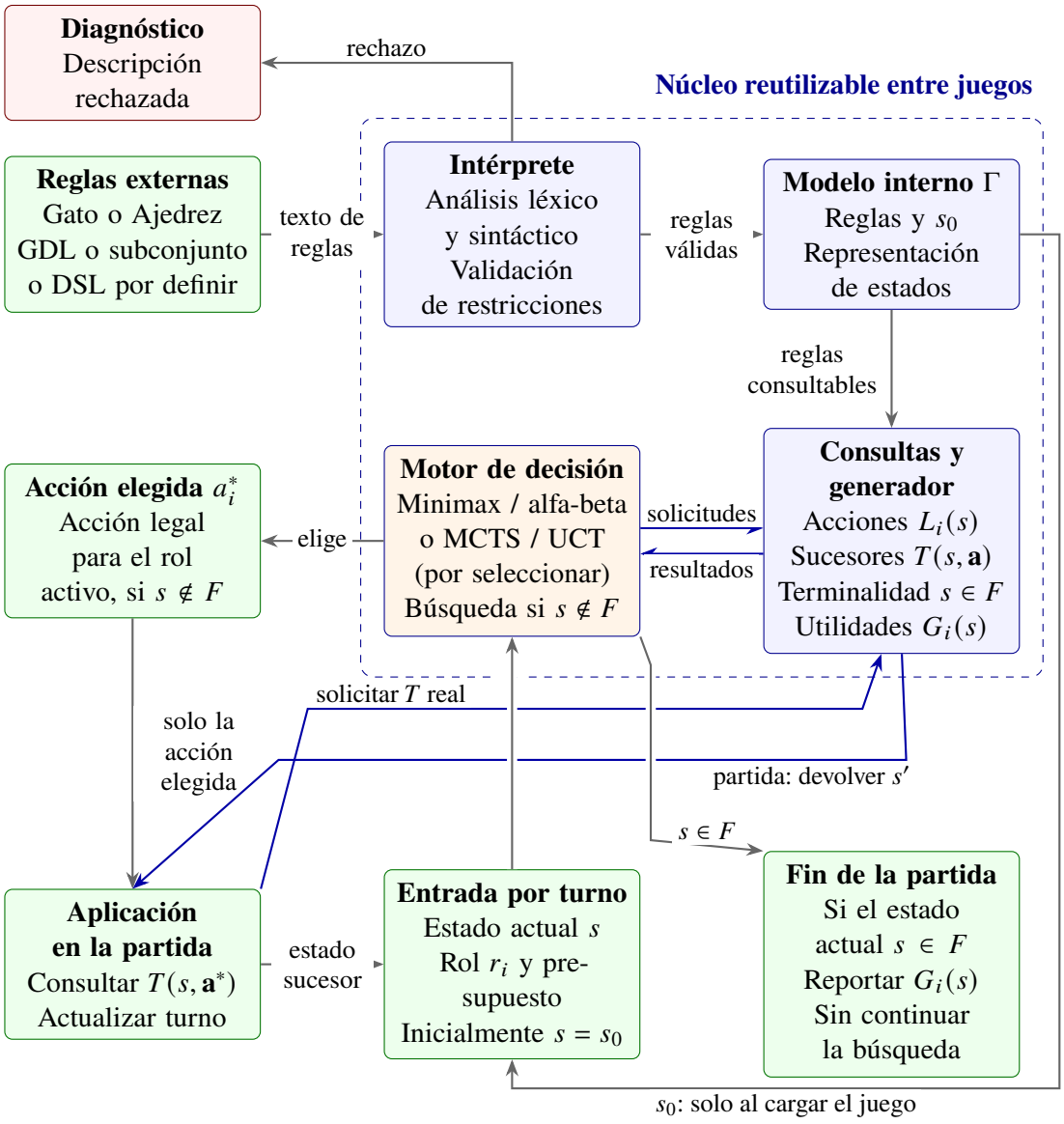


Figura 5.1: Arquitectura propuesta: carga de reglas, consultas de búsqueda y actualización de la partida. Las dos filas principales forman un único flujo; las entradas, salidas y condiciones de terminación se muestran alrededor del núcleo.

5.2. Prototipos de interfaz aprobados

Las Figuras 5.2–5.6 documentan prototipos visuales aprobados para comunicar cómo podría organizarse la interacción con el agente. Retoman los límites de responsabilidad de la arquitectura de la Figura 5.1: la interfaz recibe una descripción externa, muestra el estado de una partida y presenta la preparación o el reporte de una evaluación. No describen una interfaz implementada ni fijan una tecnología, la sintaxis definitiva de las reglas o el algoritmo de decisión. Los controles, estados y valores visibles son ilustrativos y no constituyen evidencia de ejecución.

Carga y validación de reglas. La Figura 5.2 ilustra la entrada de una descripción externa mediante un archivo de ejemplo de Gato. La vista semántica hace explícitos los roles, el estado inicial, las acciones y la condición de fin; el panel lateral representa el resultado que se esperaría de validar roles, transición, terminalidad y utilidades. La leyenda "descripción aceptada" es un estado propuesto de la interfaz, no una afirmación de que exista ya un intérprete ni de que se haya elegido GDL, un subconjunto de GDL o un DSL propio.

Partida de Gato. La Figura 5.3 propone una vista para una partida entre dos agentes. Presenta el tablero, el turno activo, las acciones legales, un historial y el espacio reservado para la siguiente decisión. Con ello separa la visualización de la partida de las consultas que el motor deberá resolver sobre legalidad y transición. La posición pausada y sus acciones se muestran solo como ejemplo de interacción, sin acreditar que se generen a partir de reglas ejecutables.

Partida de Ajedrez. La Figura 5.4 reutiliza la misma organización para Ajedrez: tablero, turno, acciones legales, historial y próxima decisión. Su propósito es hacer visible que la interfaz no queda ligada a la geometría ni a las piezas de un único dominio; el cambio de juego corresponderá a la descripción externa y al modelo interno. La posición inicial y las veinte acciones mostradas son datos visuales de ejemplo, no una prueba de que el agente ya soporte las reglas completas de Ajedrez.

Preparación de la evaluación. La Figura 5.5 representa la configuración previa de una comprobación de primera jugada en Ajedrez. Distingue el banco de casos, las condiciones de ejecución y el criterio de comparación, de acuerdo con la propuesta de métricas de la Sección 1.6.5. Las 30 posiciones, una ejecución por posición y el presupuesto de 1000 ms son valores de ejemplo; el tamaño definitivo del banco, las repeticiones, el presupuesto y el protocolo de Ajedrez continúan sujetos a acuerdo. La indicación de que las respuestas pertenecen al evaluador preserva la separación entre referencia independiente y agente evaluado.

Resultados de evaluación. La Figura 5.6 propone cómo reportar los indicadores definidos para la validación: acierto, tiempo por decisión, nodos expandidos, posiciones distintas, ejecuciones y detalle por caso. También conserva el motivo de terminación para que los tiempos agotados y otras incidencias permanezcan visibles en el registro. Los valores numéricos, los movimientos y el resumen de 4 de 5 casos son datos simulados; la figura no informa resultados experimentales ni selecciona un algoritmo de búsqueda.



Figura 5.2: Prototipo aprobado de carga y revisión de una descripción externa de Gato. La validación y el contenido del archivo son ilustrativos; la sintaxis del lenguaje permanece por definir.



Figura 5.3: Prototipo aprobado de una partida de Gato entre agentes. El tablero, historial, acciones legales y próxima decisión representan un estado ilustrativo.



Figura 5.4: Prototipo aprobado de una partida de Ajedrez entre agentes. La posición y las acciones son ilustrativas.

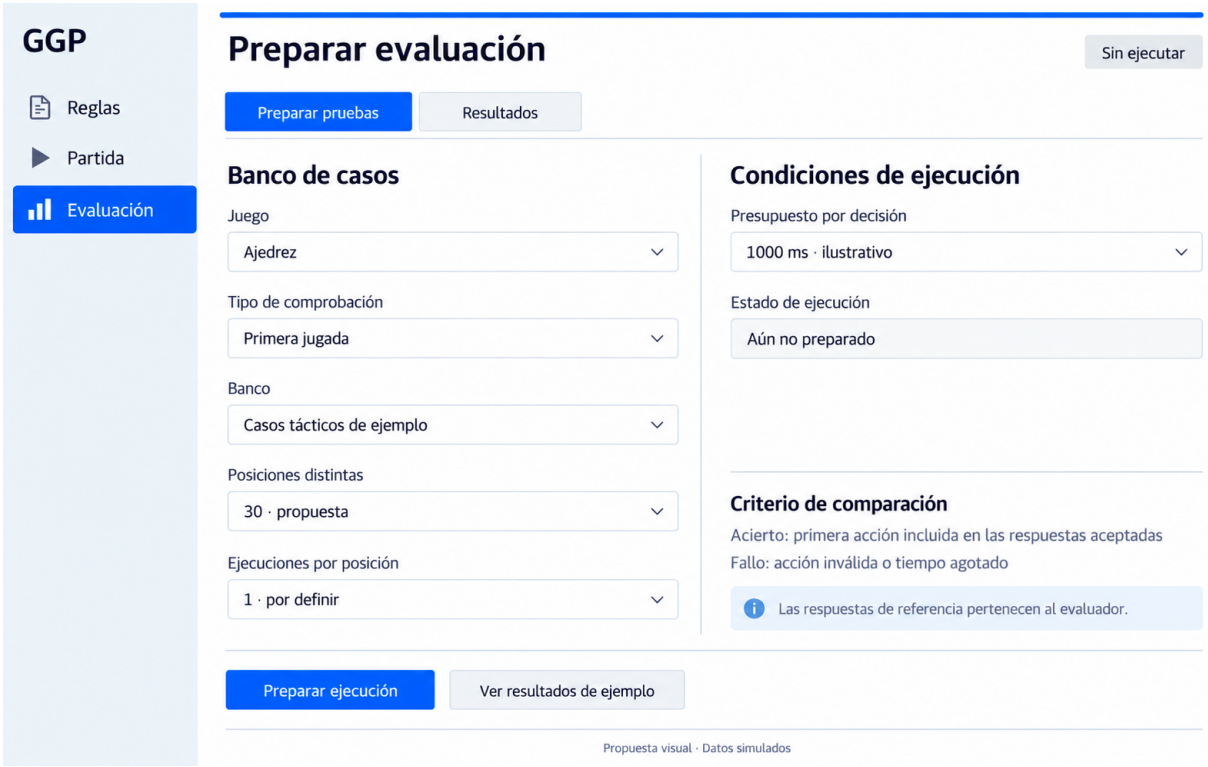


Figura 5.5: Prototipo aprobado de preparación de una evaluación táctica de primera jugada en Ajedrez. Las cantidades, el presupuesto y el estado de ejecución son ilustrativos y no describen un experimento realizado.



Figura 5.6: Prototipo aprobado del reporte de resultados de una evaluación. Las métricas y el detalle por caso muestran datos simulados, sin atribuir una ejecución real ni desempeño al agente.

Capítulo 6

Desarrollo

Capítulo 7

Pruebas

Capítulo 8

Conclusiones y trabajo a futuro

Referencias

[1] S. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 3.ª ed. Prentice Hall, 2010.

[2] A. L. Samuel, «Some Studies in Machine Learning Using the Game of Checkers,» *IBM Journal of Research and Development*, vol. 3, n.º 3, págs. 210-229, 1959. DOI: 10.1147/rd.33.0210.

[3] A. M. Turing, «Computing Machinery and Intelligence,» *Mind*, vol. 59, n.º 236, págs. 433-460, 1950. DOI: 10.1093/mind/LIX.236.433.

[4] M. Campbell, A. J. Hoane y F.-h. Hsu, «Deep Blue,» *Artificial Intelligence*, vol. 134, n.º 1–2, págs. 57-83, 2002. DOI: 10.1016/S0004-3702(01)00129-1.

[5] J. Schaeffer, N. Burch, Y. Björnsson, A. Kishimoto, M. Müller, R. Lake, P. Lu y S. Sutphen, «Checkers Is Solved,» *Science*, vol. 317, n.º 5844, págs. 1518-1522, 2007. DOI: 10.1126/science.1144079.

[6] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel y D. Hassabis, «Mastering the Game of Go with Deep Neural Networks and Tree Search,» *Nature*, vol. 529, n.º 7587, págs. 484-489, 2016. DOI: 10.1038/nature16961.

[7] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan y D. Hassabis, «A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go Through Self-Play,» *Science*, vol. 362, n.º 6419, págs. 1140-1144, 2018. DOI: 10.1126/science.aar6404.

[8] M. Genesereth, N. Love y B. Pell, «General Game Playing: Overview of the AAAI Competition,» *AI Magazine*, vol. 26, n.º 2, págs. 62-72, 2005. dirección: https://logic.stanford.edu/publications/genesereth/aaai.pdf.

[9] M. Genesereth y M. Thielscher, *General Game Playing*. Morgan & Claypool Publishers, 2014. dirección: https://logic.stanford.edu/ggp/chapters/cover.html.

[10] N. Love, T. Hinrichs, D. Haley, E. Schkufza y M. Genesereth, «General Game Playing: Game Description Language Specification,» Stanford Logic Group, inf. téc. LG-2006-01, 2008. dirección: https://logic.stanford.edu/ggp/.

[11] Google. «The N-queens Problem.» (), dirección: https://developers.google.com/optimization/cp/queens (visitado 25-10-2026).

[12] C. E. Shannon, «Programming a Computer for Playing Chess,» *Philosophical Magazine*, vol. 41, n.º 314, págs. 256-275, 1950. DOI: 10.1080/14786445008521796.

[13] The Stockfish developers. «Stockfish: A free and strong UCI chess engine.» (), dirección: https://github.com/official-stockfish/Stockfish (visitado 25-10-2026).

[14] M. Schmid, M. Moravcik, N. Burch, R. Kadlec, J. Davidson, K. Waugh, N. Bard, F. Timbers, M. Lanctot, G. Z. Holland, E. Davoodi, A. Christianson y M. Bowling, «Student of Games: A unified learning algorithm for both perfect and imperfect information games,» *Science Advances*, vol. 9, n.º 46, eadg3256, 2023. DOI: 10.1126/sciadv.adg3256.

[15] E. Sánchez Catarino y G. Linares Villegas, «Desarrollo de un General Game Player (GGP) utilizando un lenguaje de descripción de reglas para juegos de información perfecta,» Protocolo de Trabajo Terminal 2027-A178, Escuela Superior de Cómputo, Instituto Politécnico Nacional. Documento de trabajo proporcionado por los autores.

[16] N. Fiekas. «Core — python-chess documentation.» (), dirección: https://python-chess.readthedocs.io/en/latest/core.html (visitado 17-10-2026).

[17] Lichess. «Lichess open database: Puzzles.» (), dirección: https://database.lichess.org/#puzzles (visitado 17-10-2026).

[18] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap y D. Silver, «Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model,» *Nature*, vol. 588, n.º 7839, págs. 604-609, 2020. DOI: 10.1038/s41586-020-03051-4.

[19] J. Clune, «Heuristic Evaluation Methods for General Game Playing,» en *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence*, 2007, págs. 1133-1139.

[20] S. Schiffel y M. Thielscher, «Fluxplayer: A Successful General Game Player,» en *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence*, 2007, págs. 1191-1196.

[21] H. Finnsson e Y. Björnsson, «Simulation-Based Approach to General Game Playing,» en *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence*, 2008, págs. 259-264.

[22] H. Finnsson, «Generalized Monte-Carlo Tree Search Extensions for General Game Playing,» en *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*, 2012, págs. 1550-1556.

[23] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis y S. Colton, «A Survey of Monte Carlo Tree Search Methods,» *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, n.º 1, págs. 1-43, 2012. DOI: 10.1109/TCIAIG.2012.2186810.

[24] S. Draper y A. Rose. «Sancho.» (), dirección: https://sanchoggp.github.io/sancho-ggp/ (visitado 25-10-2026).

[25] S. Gelly y D. Silver, «Monte-Carlo tree search and rapid action value estimation in computer Go,» *Artificial Intelligence*, vol. 175, n.º 11, págs. 1856-1875, 2011. DOI: 10.1016/j.artint.2011.03.007.

[26] M. Thielscher, «A General Game Description Language for Incomplete Information and Stochastic Games,» en *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*, 2010, págs. 994-999. DOI: 10.1609/aaai.v24i1.7677.

[27] É. Piette, D. J. N. J. Soemers, M. Stephenson, C. F. Sironi, M. H. M. Winands y C. Browne, «Ludii – The Ludemic General Game System,» en *ECAI 2020*, 2020, págs. 411-418. DOI: 10.3233/FAIA200120. dirección: https://arxiv.org/abs/1905.05013.

[28] D. J. N. J. Soemers, E. Piette, M. Stephenson y C. Browne, «The Ludii Game Description Language is Universal,» en *Proceedings of the 2024 IEEE Conference on Games (CoG)*, 2024, págs. 1-8. DOI: 10.1109/CoG60054.2024.10645550.

[29] J. Kowalski, M. Mika, J. Sutowicz y M. Szykuła, «Regular Boardgames,» en *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence*, 2019, págs. 1699-1706. DOI: 10.1609/aaai.v33i01.33011699.

[30] M. Lanctot, E. Lockhart, J.-B. Lespiau, V. Zambaldi, S. Upadhyay, J. Pérolat, S. Srinivasan, F. Timbers, K. Tuyls, S. Omidshafiei, D. Hennes, D. Morrill, P. Feng, T. Biro, J. Hou, F. Bazzana, M. Schmid, D. Geiger, S. Liu, J. Schrittwieser, I. Antonoglou, T. Graepel y M. Bowling, «OpenSpiel: A Framework for Reinforcement Learning in Games,» *arXiv preprint arXiv:1908.09453*, 2019. DOI: 10.48550/arXiv.1908.09453.

[31] A. Goldwaser y M. Thielscher, «Deep Reinforcement Learning for General Game Playing,» en *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence*, 2020, págs. 1701-1708. DOI: 10.1609/aaai.v34i02.5533.

[32] H. W. Kuhn, «Extensive Games and the Problem of Information,» en *Contributions to the Theory of Games (AM-28), Volume II*, Princeton University Press, 1953, págs. 193-216. DOI: 10.1515/9781400881970-012.

[33] M. J. Osborne y A. Rubinstein, *A Course in Game Theory*. Cambridge, MA: MIT Press, 1994.

[34] J. McCarthy y P. J. Hayes, «Some Philosophical Problems from the Standpoint of Artificial Intelligence,» en *Machine Intelligence 4*, B. Meltzer y D. Michie, eds., Edinburgh University Press, 1969, págs. 463-502.

[35] R. Reiter, «On Closed World Data Bases,» en *Logic and Data Bases*, H. Gallaire y J. Minker, eds., Plenum Press, 1978, págs. 55-76. DOI: 10.1007/978-1-4684-3384-5_3.

[36] B. Bonet y H. Geffner, «Planning as Heuristic Search,» *Artificial Intelligence*, vol. 129, n.º 1–2, págs. 5-33, 2001. DOI: 10.1016/S0004-3702(01)00108-4.

[37] J. Hoffmann y B. Nebel, «The FF Planning System: Fast Plan Generation Through Heuristic Search,» *Journal of Artificial Intelligence Research*, vol. 14, págs. 253-302, 2001. DOI: 10.1613/jair.855. dirección: https://arxiv.org/abs/1106.0675.

[38] A. Cropper, R. Evans y M. Law, «Inductive General Game Playing,» *Machine Learning*, vol. 109, págs. 1393-1434, 2020. DOI: 10.1007/s10994-019-05843-w.

[39] J. von Neumann, «Zur Theorie der Gesellschaftsspiele,» *Mathematische Annalen*, vol. 100, n.º 1, págs. 295-320, 1928. DOI: 10.1007/BF01448847.

[40] D. E. Knuth y R. W. Moore, «An Analysis of Alpha-Beta Pruning,» *Artificial Intelligence*, vol. 6, n.º 4, págs. 293-326, 1975. DOI: 10.1016/0004-3702(75)90019-3.

[41] R. Coulom, «Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search,» en *Computers and Games (CG 2006)*, ép. Lecture Notes in Computer Science, vol. 4630, Springer, 2007, págs. 72-83. DOI: 10.1007/978-3-540-75538-8_7.

[42] L. Kocsis y C. Szepesvári, «Bandit Based Monte-Carlo Planning,» en *Machine Learning: ECML 2006*, ép. Lecture Notes in Computer Science, vol. 4212, Springer, 2006, págs. 282-293. DOI: 10.1007/11871842_29.

[43] P. Auer, N. Cesa-Bianchi y P. Fischer, «Finite-time Analysis of the Multiarmed Bandit Problem,» *Machine Learning*, vol. 47, n.º 2–3, págs. 235-256, 2002. DOI: 10.1023/A:1013689704352.

[44] J. E. Hopcroft, R. Motwani y J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3.ª ed. Addison-Wesley, 2006.

[45] J. W. Lloyd, *Foundations of Logic Programming*, 2.ª ed. Springer, 1987.

[46] K. R. Apt, H. A. Blair y A. Walker, «Towards a Theory of Declarative Knowledge,» en *Foundations of Deductive Databases and Logic Programming*, J. Minker, ed., Morgan Kaufmann, 1988, págs. 89-148.

[47] S. Schiffel y M. Thielscher, «A Multiagent Semantics for the Game Description Language,» en *Agents and Artificial Intelligence (ICAART 2009)*, ép. Communications in Computer and Information Science, J. Filipe, A. Fred y B. Sharp, eds., vol. 67, Springer, 2010, págs. 44-55.

[48] M. R. Genesereth y R. E. Fikes, «Knowledge Interchange Format, Version 3.0: Reference Manual,» Computer Science Department, Stanford University, inf. téc. Logic-92-1, 1992.

[49] J. McCarthy, «Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I,» *Communications of the ACM*, vol. 3, n.º 4, págs. 184-195, 1960.

[50] S. Ceri, G. Gottlob y L. Tanca, «What You Always Wanted to Know About Datalog (And Never Dared to Ask),» *IEEE Transactions on Knowledge and Data Engineering*, vol. 1, n.º 1, págs. 146-166, 1989.

[51] S. Abiteboul, R. Hull y V. Vianu, *Foundations of Databases*. Addison-Wesley, 1995.

[52] S. Schiffel y M. Thielscher, «Representing and Reasoning About the Rules of General Games With Imperfect Information,» *Journal of Artificial Intelligence Research*, vol. 49, págs. 171-206, 2014.

[53] M. Thielscher, «GDL-III: A Description Language for Epistemic General Game Playing,» en *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI)*, 2017, págs. 1276-1282.

[54] M. Gelfond y V. Lifschitz, «The Stable Model Semantics for Logic Programming,» en *Proceedings of the Fifth International Conference and Symposium on Logic Programming*, MIT Press, 1988, págs. 1070-1080.

[55] D. Kőnig, «Über eine Schlussweise aus dem Endlichen ins Unendliche,» *Acta Scientiarum Mathematicarum (Szeged)*, vol. 3, págs. 121-130, 1927.

[56] M. Thielscher, «The General Game Playing Description Language Is Universal,» en *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence (IJCAI)*, 2011, págs. 1107-1112.

[57] E. Dantsin, T. Eiter, G. Gottlob y A. Voronkov, «Complexity and Expressive Power of Logic Programming,» *ACM Computing Surveys*, vol. 33, n.º 3, págs. 374-425, 2001.

[58] C. Browne, «A Class Grammar for General Games,» en *Computers and Games (CG 2016)*, ép. Lecture Notes in Computer Science, vol. 10068, Springer, 2016, págs. 167-182.

[59] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld y D. Wilkins, «PDDL – The Planning Domain Definition Language,» Yale Center for Computational Vision y Control, inf. téc. CVC TR-98-003, 1998.

Anexos

Cronogramas

Los siguientes cronogramas reproducen las actividades, los responsables y los periodos del protocolo del TT 2027-A178 [15]. Las celdas sombreadas indican el periodo programado; no representan actividades concluidas ni porcentajes de avance. Se conservan los meses de agosto a junio sin asignar años o fechas adicionales.

Sánchez Catarino Edgar

Actividad	Ago	Sep	Oct	Nov	Dic	Ene	Feb	Mar	Abr	May	Jun
Fundamentación teórica del motor y representación matemática	■	■	■								
Programación de reglas, validación y generación de movimientos				■	■	■					
Implementación del agente inteligente y algoritmos de optimización de búsqueda.							■	■			
Integración del motor base							■	■			
Desarrollo del archivo de descripción del juego ajedrez									■		
Redacción de los capítulos técnicos y manuales del documento de Trabajo Terminal.									■	■	
Revisión final del documento y preparación del material para la defensa.											■

Linares Villegas Gustavo

Actividad	Ago	Sep	Oct	Nov	Dic	Ene	Feb	Mar	Abr	May	Jun
Investigación del estado del arte en agentes lúdicos y definición del problema.											
Diseño algorítmico de los árboles de búsqueda y propuesta de la función heurística.											
Diseño y ejecución de experimentos para evaluar el rendimiento algorítmico.											
Análisis estadístico de resultados y redacción de conclusiones en el Trabajo Terminal.											
Redacción de los capítulos técnicos y manuales del documento de Trabajo Terminal.											
Revisión final del documento y preparación del material para la defensa.											

Dependencia por revisar. Los experimentos están programados para febrero y marzo, mientras que el archivo de descripción de Ajedrez está programado para abril. Antes de ejecutar las pruebas, se deberá revisar la disponibilidad de ese archivo y delimitar los experimentos que dependan de él. Se conserva el calendario original y no se supone que las actividades ya hayan sido realizadas.